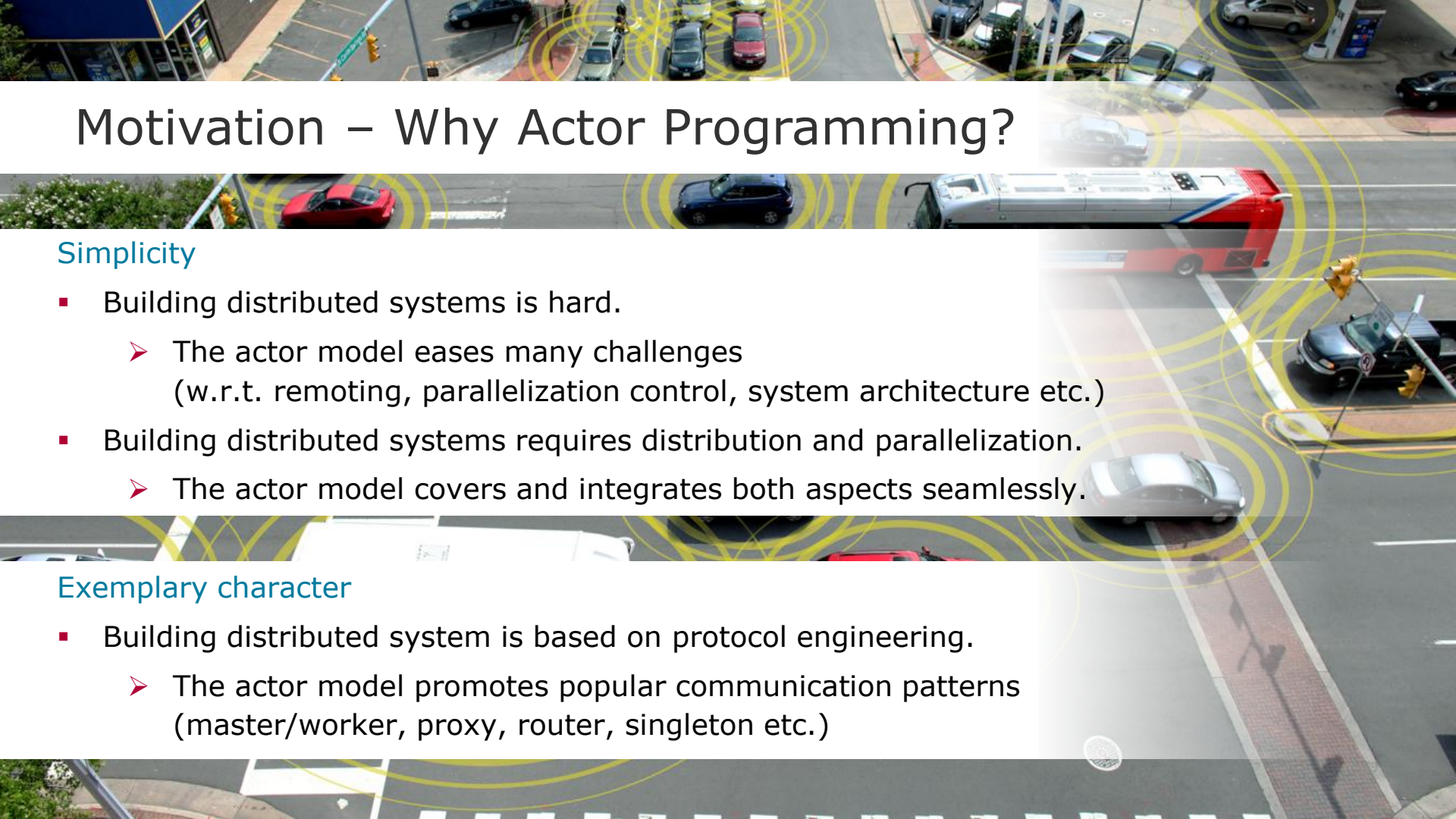




Big Data Systems Akka Actor Programming

Thorsten Papenbrock



Motivation – Why Actor Programming?

Simplicity

- Building distributed systems is hard.
 - The actor model eases many challenges (w.r.t. remoting, parallelization control, system architecture etc.)
- Building distributed systems requires distribution and parallelization.
 - The actor model covers and integrates both aspects seamlessly.

Exemplary character

- Building distributed system is based on protocol engineering.
 - The actor model promotes popular communication patterns (master/worker, proxy, router, singleton etc.)

Actor Model

Motivation – Why Akka?

Popularity

- Akka is the most popular actor framework
- Akka is/was used within popular Big Data tools (Spark, Flink etc.)
- Akka is build for the omnipresent JVM with Java and Scala APIs
 - Akka is a valuable technology to know about!

Coverage

- Akka uses state-of-the-art distributed system techniques (e.g. for failure detection, leader election, gossip, sharding etc.)
- Akka offers an easy to learn toolkit with many pre-build protocols (e.g. with routers, adapters, supervision, clustering, streaming, ...)
 - Akka is an easy and educative toolkit for building distributed systems!

[1] <https://awesomeopensource.com/projects/actor-model>

[2] <https://stackshare.io/stackups/akka-vs-orleans-vs-actix>

[1]

Akka

★ 11,790

Orleans

★ 7,871

Actix

★ 6,616

[2]



Akka

+ Follow

+ I use this

Stacks	Followers	Votes
799	798	83



Orleans

+ Follow

+ I use this

Stacks	Followers	Votes
41	83	26

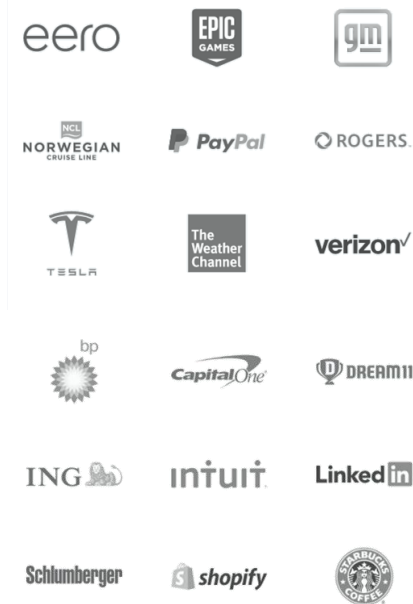


Actix

+ Follow

+ I use this

Stacks	Followers	Votes
84	126	5



Self marketing:
<https://akka.io/>



Coding Preparation

Coding Preparation

Development Environment

Java installation

- https://java.com/en/download/help/download_options.html
- Use Java 21 or higher

Maven installation

- <https://maven.apache.org/install.html>

IntelliJ IDEA installation

- <https://www.jetbrains.com/help/idea/installation-guide.html>
- Use your student email for a free ultimate license
- Select the Scala and sbt plugins to be installed during the installation; otherwise you need to install them later
- Install the lombok plugin within IntelliJ
<https://projectlombok.org/setup/intellij>

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide 5

Coding Preparation

Development Environment

Project Lombok



```
public static class MyMessage implements Serializable {  
    private static final long serialVersionUID = -6823011111281387872L;
```

```
    private MyMessage() {}
```

Default constructor for serialization

```
    private MyMessage(String myText, int myValue) {  
        this.myText = myText;  
        this.myValue = myValue;  
    }
```

All-args constructor for instantiation

```
    private String myText;  
    private int myValue;
```

The "real" message attributes

```
    public String getMyText() {  
        return this.myText;  
    }
```

Getters for private message data;
no need for setters due to message immutability

```
    public int getMyValue() {  
        return this.myValue;  
    }  
}
```

So much shorter in Scala...

```
case class MyMessage(myText: String, myValue: Int)
```

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide 6

Coding Preparation

Development Environment

Project Lombok



Lombok provides aspects for generating most Java boilerplate code

```
import lombok.AllArgsConstructor;  
import lombok.NoArgsConstructor;  
import lombok.Getter;
```

```
@Getter @AllArgsConstructor @NoArgsConstructor  
public static class MyMessage implements Serializable {  
    private static final long serialVersionUID = -6823011111281387872L;  
  
    private String myText;  
    private int myValue;  
}
```

@Data for getters and setters
@Getter for only getters
@Setter for only setters

No **final** due to serialization

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide 7

Tasks

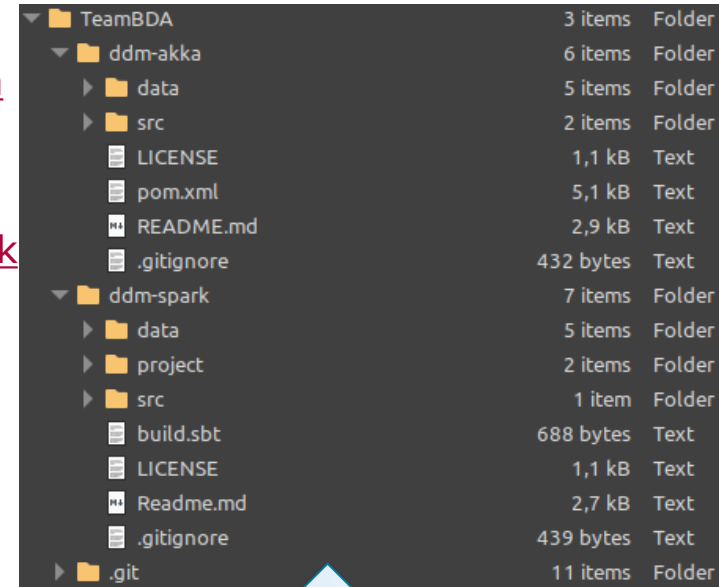
1. Form teams of **one or two** students.
 - A team size of two is highly recommended, but not enforced.
 - Teams with more than two participants are rejected.
2. Create a new **private** GitHub repository.
 - The name of your repository will be your team name; choose whatever name you like.
 - Invite **ilnazTayebi** and **thorsten-Papenbrock** to your new repository.
3. Create two folders in your repository: **ddm-akka** and **ddm-spark**
 - The two folders will host the two exercises that need to be completed for this course.
 - The Akka exercise is the first assignment; the Spark exercise will later be our second.

Coding Preparation

Repository Setup

Tasks

4. **Copy** the ddm-akka project
<https://github.com/UMR-Big-Data-Analytics/ddm-akka>
into your repository.
5. **Copy** the ddm-spark project
<https://github.com/UMR-Big-Data-Analytics/ddm-spark>
into your repository.
 - Copy the contents and do not fork the original repositories, because your repository needs to remain private!
6. Download the homework data from our Ilias course (Homework Data -> TPCB.zip) and unzip the data into a folder **ddm-akka/data/** of your Akka project.



TeamBDA	3 items	Folder
ddm-akka	6 items	Folder
data	5 items	Folder
src	2 items	Folder
LICENSE	1,1 kB	Text
pom.xml	5,1 kB	Text
README.md	2,9 kB	Text
.gitignore	432 bytes	Text
ddm-spark	7 items	Folder
data	5 items	Folder
project	2 items	Folder
src	1 item	Folder
build.sbt	688 bytes	Text
LICENSE	1,1 kB	Text
Readme.md	2,7 kB	Text
.gitignore	439 bytes	Text
.git	11 items	Folder

Your file structure should now look like this



Inhalt

- Virtual Meeting Room
BigBlueButton streams of lectures and exercises; BigBlueButton
- Forum
A place to ask questions, spark discussions and form exercise groups
Beiträge (Ungelesen): 0 (0)
- Lecture Slides
Slides of all completed lectures and exercises
- Homework Data
Data and resources for the practical exercises
- Exercise Exams
Exercises and their solutions of past DDM iterations
- Literature
Reference publications and books for the topics discussed in the lecture
- Coding Assignments
Submit your GitHub repository
Lernfortschritt:
- Lecture Recordings
Recordings of the BigBlueButton streams of the lecture

Submit your GitHub repository!

Neigkeiten

Die Datei wurde hinzugefügt.
Datei: BD5_Re_Exam_WS2023_Musterloesu...
Termin: Heute, 11:06

Die Datei wurde hinzugefügt.
Datei: BD5_Re_Exam_WS2023_2.pdf
Termin: Heute, 11:06

Die Datei wurde hinzugefügt.
Datei: ExamDDM-5521 solution.pdf
Termin: Heute, 11:06

Die Datei wurde hinzugefügt.
Datei: ExamDDM-WS1920 solution.pdf
Termin: Heute, 11:06

Die Datei wurde hinzugefügt.
Datei: ExamDDA-WS1718.pdf
Termin: Heute, 11:06

Kalender

So	Mo	Di	Mi
			1
5	6	7	8
12	13	14	15
19	20	21	22
26	27	28	29

Tasks

7. Submit you GitHub repository URL.

- Every student needs to submit the URL of his/her GitHub repository.
- Students with the same URL are considered to be team members.
- Example of a GitHub URL:
<https://github.com/GitHubProgrammer/TeamBDA>
- Provide only the GitHub URL without any further text.

Deadline: 15.11.2025

 ddm-akka Public

✂ Edit Pins

👁 Unwatch 1

🍴 Fork 13

☆ Star 3

👤 main

🌿 2 Branches

🏷 0 Tags

🔍 Go to file

t

➕ Add file

<> Code

 thorsten-papenbrock Update README.md	053ad80 · 3 hours ago	🕒 15 Commits
📁 src	Added a flag for the enterShutdown, increased the logger st...	9 months ago
📄 .gitignore	Updated gitignore to ignore the solution files	8 months ago
📄 LICENSE	Migrated the ddm-akka projekt into this new repository.	last year
📄 README.md	Update README.md	3 hours ago
📄 pom.xml	Update libs and extend readme for token usage of akka	4 days ago

📖 README

📄 MIT license

✎

☰

ddm-akka

Akka example and homework code for the "Distributed Data Management" lecture.

Requirements

- Java Version >= 21
- Maven Compiler Version >= 3.14.1

Getting started

1. Clone repo

```
git clone https://github.com/UMR-Big-Data-Analytics/ddm-akka.git
```

2. Get the Akka libraries

About

Akka example and homework code for the "Distributed Data Management" lecture.

📖 README

📄 MIT license

🔄 Activity

📖 Custom properties

☆ 3 stars

👁 1 watching

🍴 13 forks

Report repository

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Contributors 2

 **thorsten-papenbrock** Thorsten Pape...

 **SeegerM** Seeger

<https://github.com/UMR-Big-Data-Analytics/ddm-akka>

Coding Preparation

Repository Setup

Tasks

8. Setup your settings.xml

- Get an Akka access token.
- Create/Update your settings.xml

9. Follow the README instructions.

- Build the code.
- Perform a test run.

Getting started

1. Clone repo

```
git clone https://github.com/UMR-Big-Data-Analytics/ddm-akka.git
```

2. Get the Akka libraries

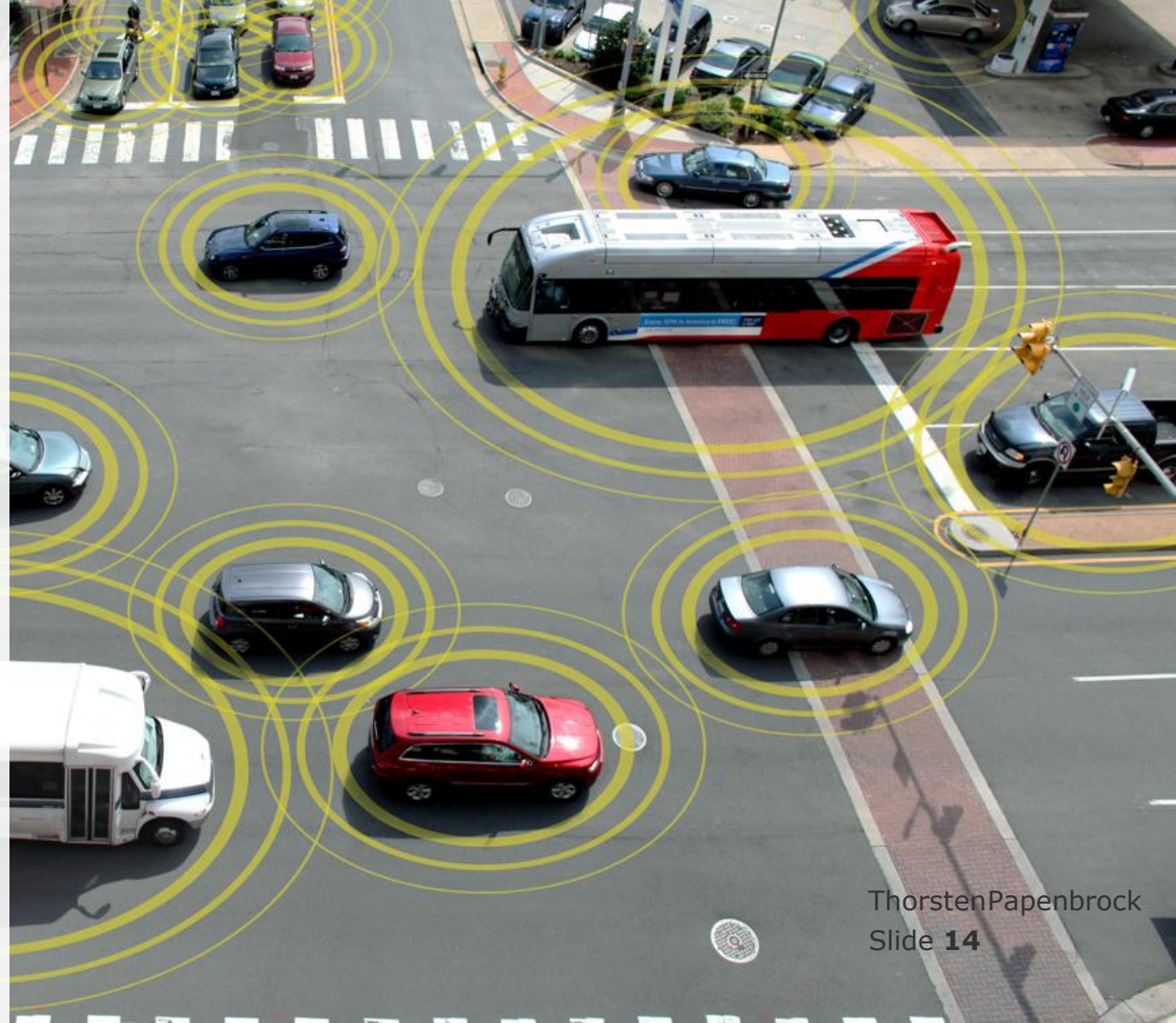
The license of Akka requires the user to obtain an access token (free for development) to checkout the libraries. For this, create an account for [Akka](#) and get your token from [here](#). Then, we need to add this token to the `~/.m2/settings.xml` file (Linux/Mac), `c:\Users\<username>\.m2` . If this file does not exist, create it. Add the following content to this file, replacing `#TOKEN` with your actual token:

```
<settings xmlns="http://maven.apache.org/SETTINGS/1.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://maven.apache.org/xsd/settings-1.0.0.xsd">
  <activeProfiles>
    <activeProfile>akka</activeProfile>
  </activeProfiles>
  <profiles>
    <profile>
      <id>akka</id>
      <repositories>
        <repository>
          <id>akka-secure</id>
          <name>Akka Secure</name>
          <url>https://repo.akka.io/#TOKEN/secure</url>
        </repository>
      </repositories>
    </profile>
  </profiles>
</settings>
```

Do not add the repository to the `pom.xml` file, as this would expose your token to everyone who has access to the `pom.xml` file.

Akka Actor Programming Hands-on

- **Actor Model**
- Basic Concepts
- Runtime Architecture
- Demo
- Messaging
- Parallelization
- Remoting
- Clustering
- Patterns
- Homework



Actor Model

Reactive Systems

Responsiveness

- The system responds in a timely manner (if possible).
- To user interaction, failures, events, state changes, data characteristics, ...

Resilience

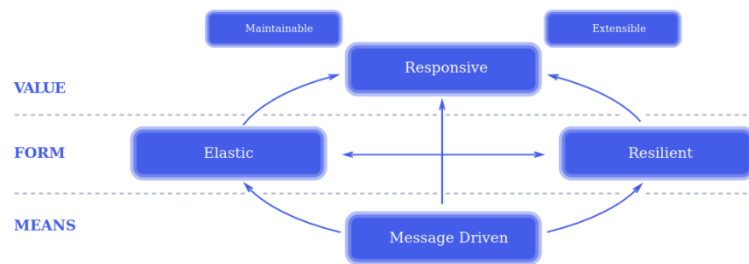
- The system stays responsive in the face of failures.
- Achieved by replication, containment, isolation and delegation.

Elasticity

- The system stays responsive under varying workloads.
- Achieved by dynamic resource (de-)allocation, sharding, replication and decentralization.

Messaging

- The system relies on asynchronous message-passing between loosely coupled, isolated and location transparent components.
- Enables workload distribution, parallelization, failure delegation, load management, and flow control.



<https://www.reactivemanifesto.org/>

Actor Model

Actor Programming

Object-oriented programming

- Objects encapsulate state and behavior.
- Objects communicate with each other.
- Separation of concerns makes applications easier to build and maintain.

Actor programming

- Actors encapsulate state and behavior.
- Actors communicate with each other.
- Actor activities are scheduled and executed transparently.
- Combines the advantages of object- and task-oriented programming.

Task-oriented programming

- Application split down into task graph.
- Tasks are scheduled and executed transparently.
- Decoupling of tasks and resources allows for asynchronous and parallel programming.

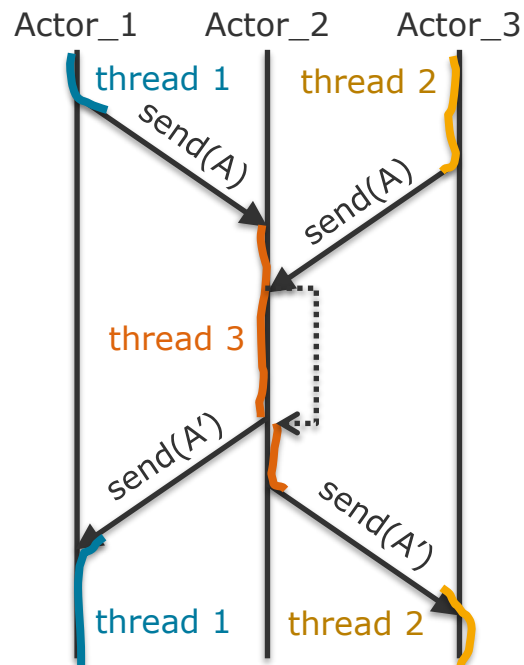
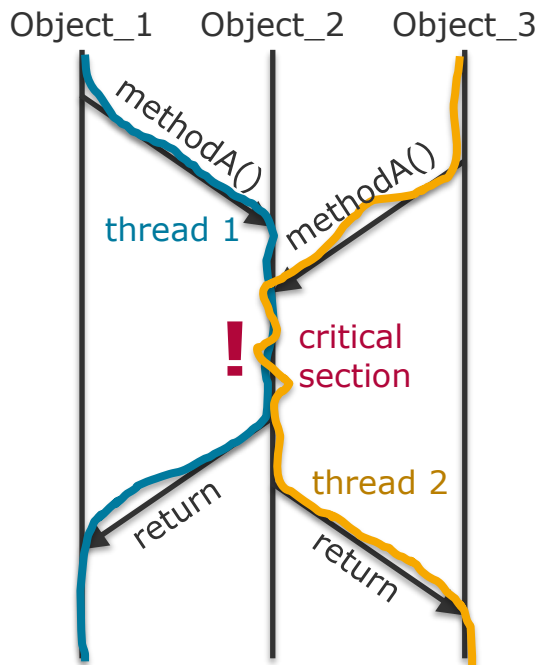
Big Data Systems

Akka Actor Programming

ThorstenPapenbrock
Slide **16**

Actor Model

Actor Programming



Note: Actors might actually be put on different threads from one message to the next!

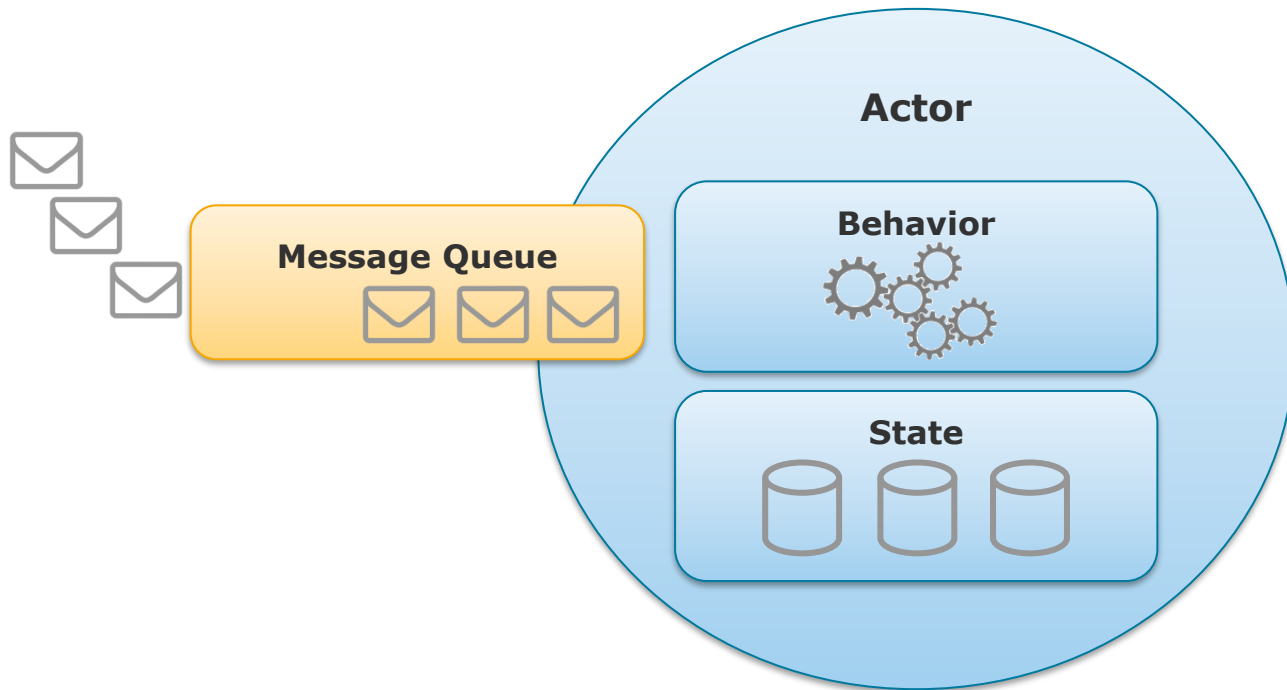
Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide 17

Actor Model

Actor



**Distributed Data
Analytics**

Akka Actor
Programming

ThorstenPapenbrock
Slide **18**

Actor Model

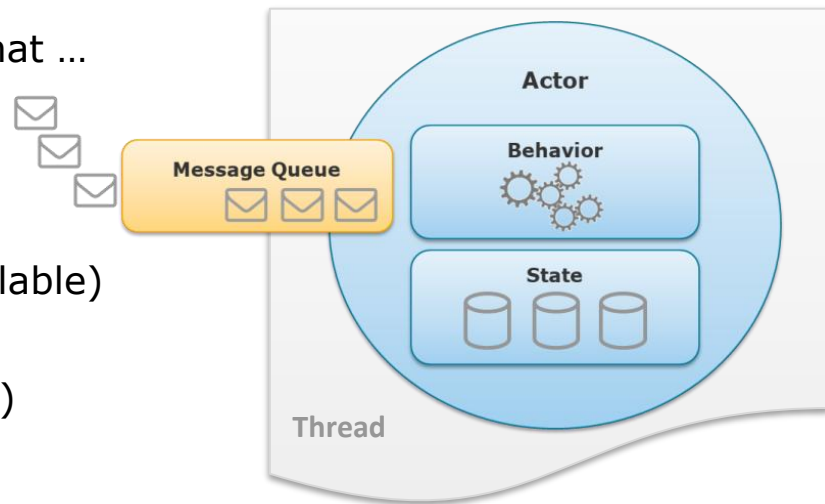
Actor

An object with strictly private **state** and **behavior** that ...

- owns exactly one **message queue**
(cannot subscribe to more or less queues)
- is dynamically scheduled on threads
(if messages are queued and resources are available)
- constitutes the smallest unit of parallelization
(execution within an actor is strongly sequential)
- reacts to incoming messages
(is passive like any object otherwise)

Reactions (on one message at a time):

- Send a finite number of messages to (other) actors
- Change own state and/or behavior for next message
- Create a finite number of new actors



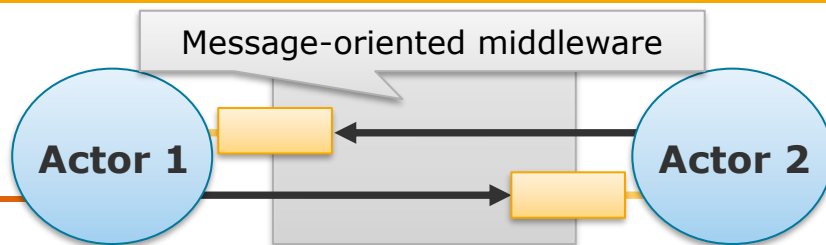
**Distributed Data
Analytics**

Akka Actor
Programming

ThorstenPapenbrock
Slide **19**

Actor Model

Actor Model



The **actor model** is a mathematical, object-oriented message-passing model that treats actors as the universal primitives of concurrent computation.

- Actors in the actor model ...
 - are concurrent, fully encapsulated, self-contained entities;
 - address one another via location-transparent references that identify an actor object (pointer) inside a process (ID) on a node (IP + port).
- Message-oriented middleware required to i.a. ...
 - resolve abstract addresses;
 - deliver messages from sender to receiver actors;
 - schedule actors on threads.



Carl Eddie Hewitt,
designer of the Actor Model in 1973.
[Carl Hewitt, Peter Bishop and Richard Steiger,
"A Universal Modular Actor Formalism for
Artificial Intelligence", 1973, IJCAI]

Actor Model

Actor Model Principles



The actor model follows certain programming principles:

- **Asynchronicity**
 - Actors fire-and-forget messages
- **Encapsulation**
 - Actors have strictly private state and behavior
- **Distribution**
 - Actor locations are transparent
- **Parallelization**
 - Actors execute concurrently
- **Synchronization**
 - Actors synchronize explicitly

➤ Actor model helps to prevent many parallel programming issues (concurrent memory access, race conditions, locking, deadlocks, ...)



Alan Kay,
pioneer of object orientation:
“The actor model retained more of what
I thought were good features of the
object idea”

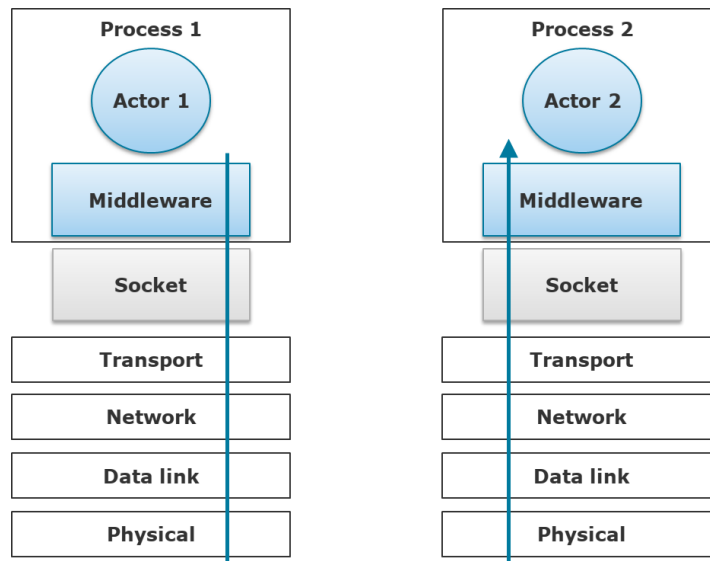
Actor Model

Actor Model Principles



Asynchronicity

- Actor-to-Actor communication is asynchronous (fire-and-forget)
- Actor-to-Middleware communication can be synchronous or asynchronous (depending on the actor model implementation)
 - Note: Actor-to-Middleware communication is within the same process and can therefore use shared memory (middleware implementation hides this from developer)
- Middleware communication is usually based on synchronous TCP or UDP sockets



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **22**

Actor Model

Actor Model Principles



Encapsulation

- Communicating entities have private state and private behavior.
- Communication means sending messages and reacting on received messages.
- Communicate “what” is to be done not “how”!
 - The recipient decides how and if it handles a certain message.
- Communication protocols define the etiquette:
 - Commonly agreed message formats
 - Patterns for message exchanges

It is the developers task to
define actor-based
application protocols that
form viable applications!

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **23**

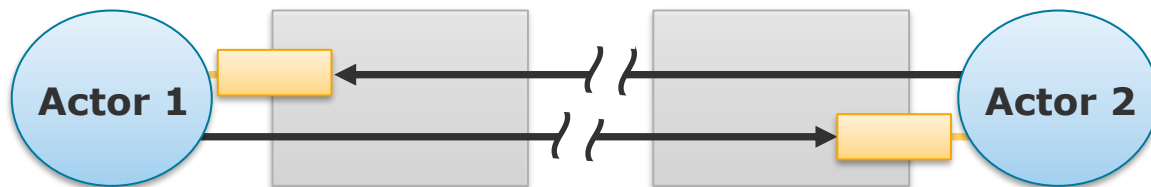
Actor Model

Actor Model Principles



Distribution

- Messages can pass through busses, channels, networks, ...
- Actors use abstract references to identify each other.
- Message-passing system i.e. the middleware resolves addresses and automatically routes messages from senders to receivers.
 - Allows actors to be transparently distributed, i.e., actors do not need to know where their communication partners actually are.



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **24**

Actor Model

Actor Model Principles



Parallelization

- Actors process one message at a time but different actors operate independently (parallelization between actors not within an actor).
- Actors may spawn new actors if needed (**dynamic parallelization**).
- Task parallelism:**
 - Different actors execute different tasks.
- Data parallelism:**
 - Different actors execute the same task on different data elements.

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **25**

Actor Model

Actor Model Principles



Synchronization

Message passing $\neq$ no Synchronization

- Synchronization happens explicitly via messaging and state changes.
- Immutable messages and private actor states prevent concurrency conflicts (e.g. concurrent memory access).
- Deadlocks, starvation and live-locks are easier to avoid with message passing, because resource ownership and waiting behavior is more explicit; still all are possible if communication protocols are faulty implemented.
- Actor-Middleware and Middleware-Actor communication still requires synchronization primitives or lock-free data structures.
 - Message passing can be implemented via locking/atomic operations on the message queue(s) or buffer(s).
 - Vice versa: Locks can be implemented via exchange of messages (e.g. the request/response pattern introduces waiting behavior).

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **26**

See also <http://web.mit.edu/6.031/www/fa17/classes/22-queues>
and <https://stackoverflow.com/questions/7140544/message-passing-vs-locking>

Actor Model

Actor Model



Advantages

- **Fault-tolerance:**
 - “Let it crash!” philosophy to heal from unexpected errors
 - Lifecycle management with automatic restarts of failed actors
 - Dead-letters mailbox to capture messaging faults
 - Errors are expected to happen and implemented into the model.
- **Fault-prevention:**
 - Asynchronous messaging helps to prevent deadlocks.
 - Private states and immutability prevent concurrent state modifications.
- **Parallelization:**
 - Implicit parallelization: Actors process one message at a time but different actors always operate independently, i.e., in parallel.
 - Dynamic parallelization: Actors may spawn new actors if needed.

Big Data Systems

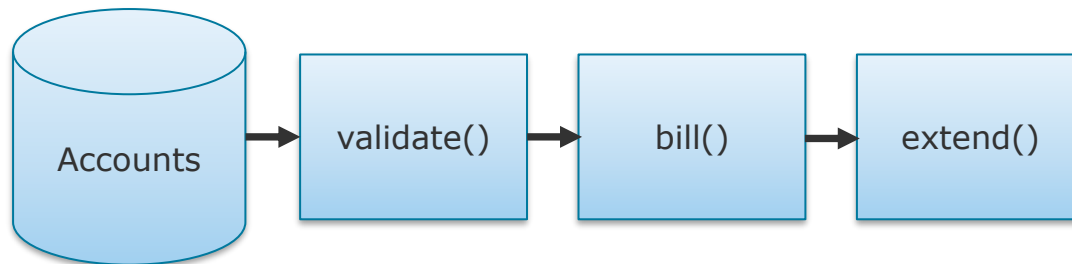
Akka Actor
Programming

ThorstenPapenbrock
Slide **27**

Actor Model

Actor Model vs. Multithreading

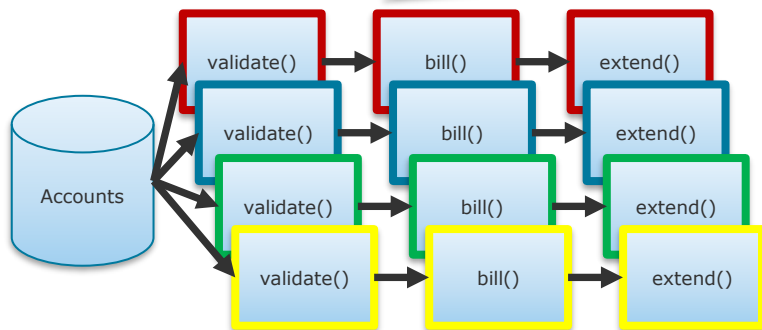
Example:



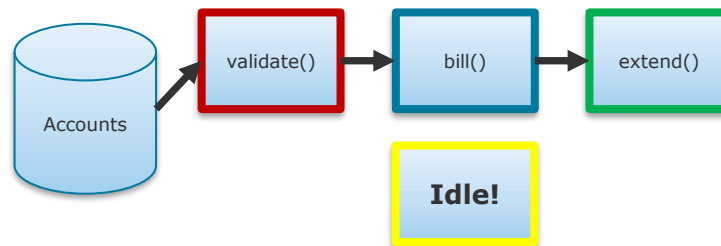
Multithreading
(4 threads):

If partitions are **skewed** or
if partitions **cannot be evenly divided** over threads,
threads might idle around!

If tasks **have different execution times** or
if tasks **cannot be evenly divided** over threads,
threads will idle around!



data-parallel



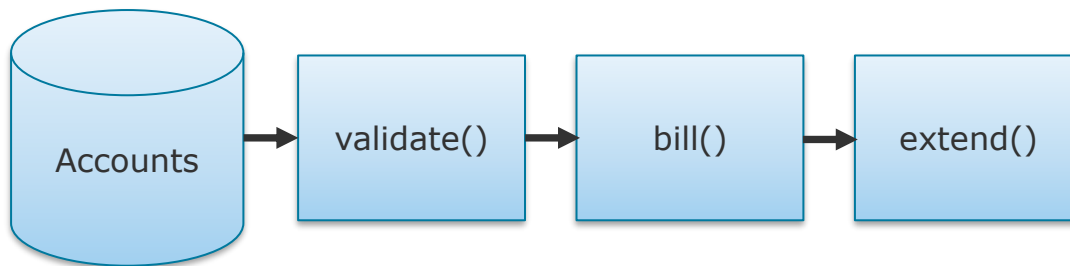
task-parallel

Big Data Systems

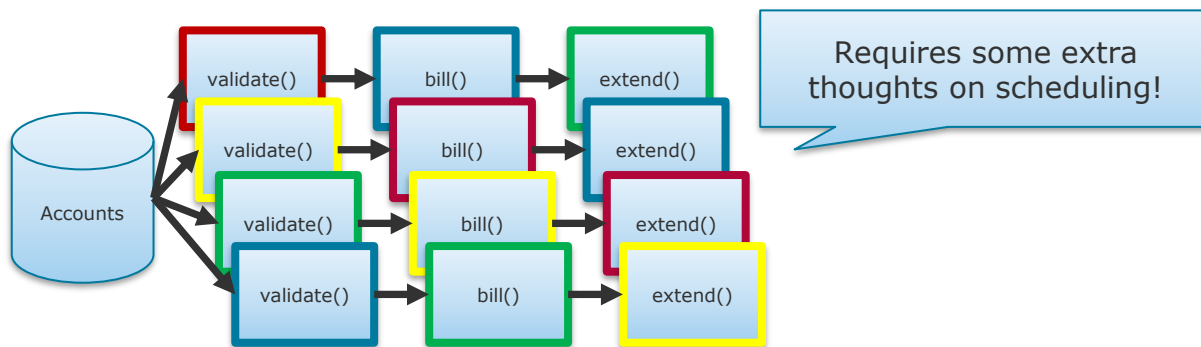
Akka Actor
Programming

ThorstenPapenbrock
Slide **28**

Example:



Multithreading
(4 threads):



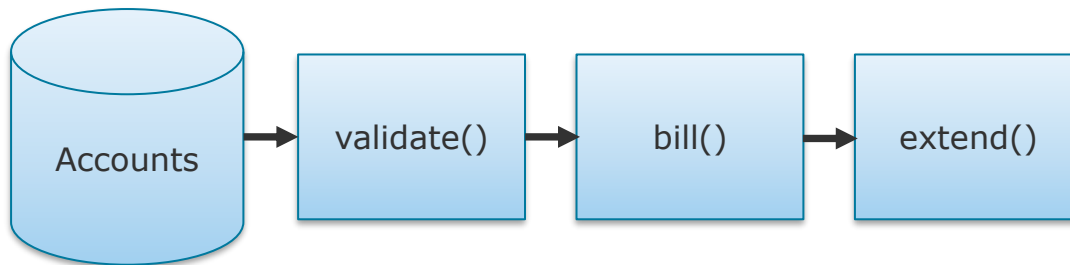
data- and task-parallel

Big Data Systems

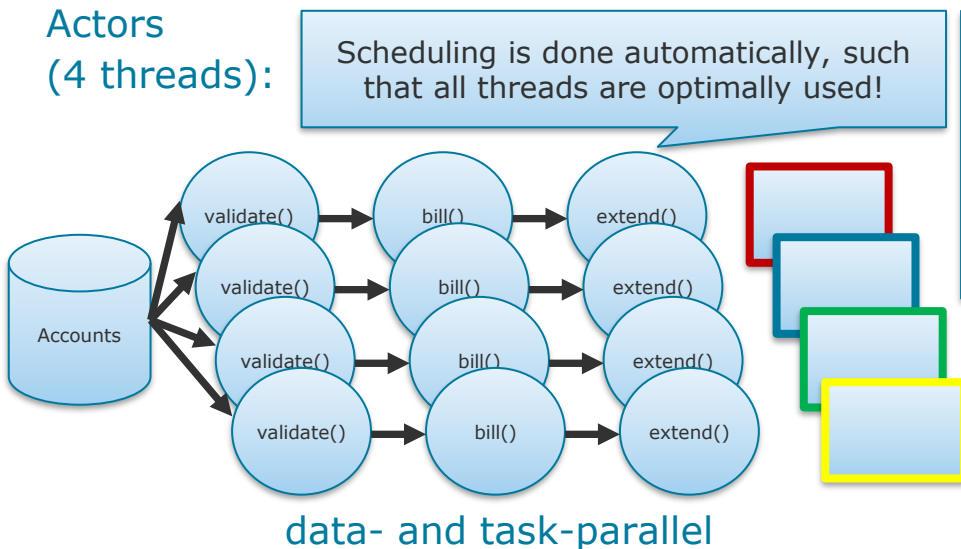
Akka Actor
Programming

ThorstenPapenbrock
Slide 29

Example:



Actors
(4 threads):



Disadvantages here:

- Messaging adds some extra costs (routing, memory management).
- If message data is larger than actor data, cache lines might not be used optimally.

Big Data Systems

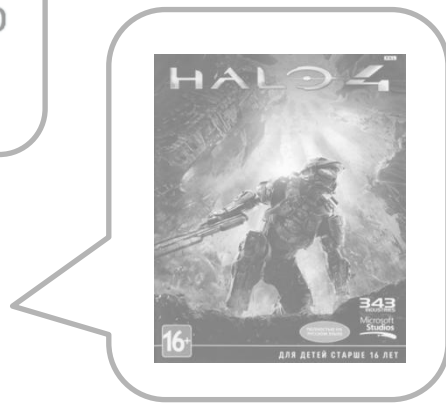
Akka Actor
Programming

ThorstenPapenbrock
Slide **30**

Actor Model

Actor Model Implementations

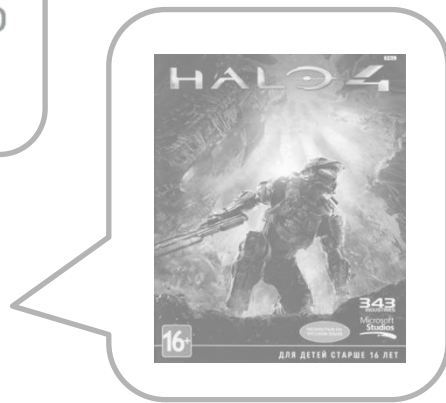
- **Erlang:** 
 - Actor library natively included in the Erlang language
 - First popular actor implementation
 - Special: **Native language support** and strong actor isolation
- **Akka:** 
 - Actor library for the JVM (Java und Scala)
 - Most popular actor implementation (atm.)
 - Special: **Actor hierarchies** and typed actors
- **Orleans:**  Orleans
 - Actor library for Microsoft .NET/C#
 - Very popular in research and industry (due to Microsoft)
 - Special: **Virtual actors** (persistent state and transparent location)



Actor Model

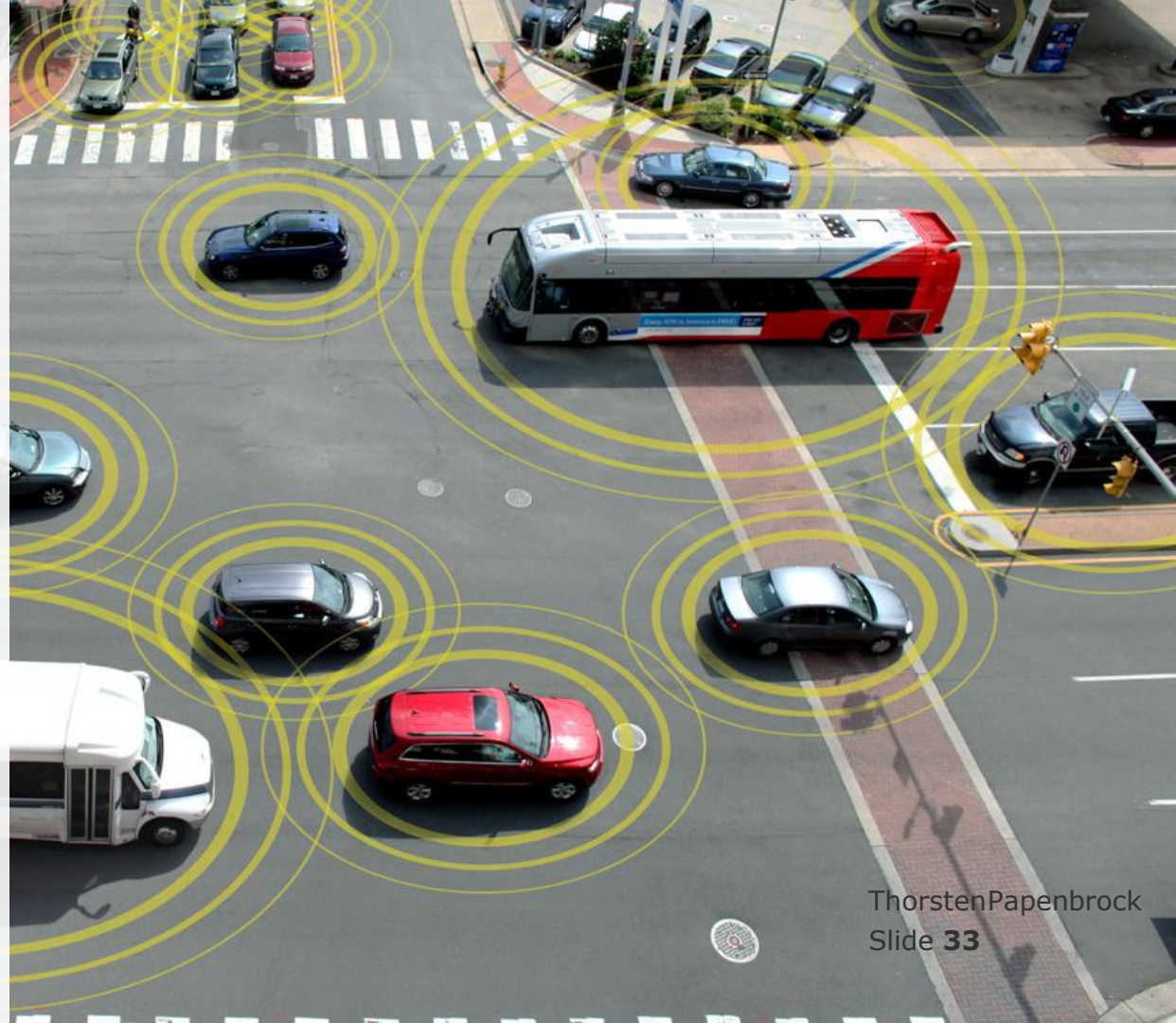
Actor Model Implementations

- **Erlang:** 
 - Actor library natively included in the Erlang language
 - First popular actor implementation
 - Special: **Native language support** and strong actor isolation
- **Akka:** 
 - Actor library for the JVM (Java und Scala)
 - Most popular actor implementation (atm.)
 - Special: **Actor hierarchies** and typed actors
- **Orleans:**  Orleans
 - Actor library for Microsoft .NET/C#
 - Very popular in research and industry (due to Microsoft)
 - Special: **Virtual actors** (persistent state and transparent location)



Akka Actor Programming Hands-on

- Actor Model
- **Basic Concepts**
- Runtime Architecture
- Demo
- Messaging
- Parallelization
- Remoting
- Clustering
- Patterns
- Homework



Basic Concepts

Akka Toolkit and Runtime



- Implements the **actor model**
- A free and open-source **toolkit and runtime** for building concurrent and distributed applications on the JVM (<https://akka.io/>)
- Written in Scala (<https://scala-lang.org/>)
- Offers APIs for **Java und Scala**
- Invented by Jonas Bonér
- Maintained by Lightbend (<https://lightbend.com/>)

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **34**

Basic Concepts

Akka Modules

Actors

Core actor
model classes
for concurrency
and distribution

Cluster

Classes for the
resilient and
elastic
distribution over
multiple nodes

Streams

Asynchronous,
non-blocking,
backpressured,
reactive stream
classes

Http

Asynchronous,
streaming-first
HTTP server and
client classes

Cluster Sharding

Classes to
decouple actors
from their
locations
referencing
them by identity

Persistence

Classes to
persist actor
state for fault
tolerance and
state restore
after restarts

Alpakka

Stream
connector
classes to other
technologies

...

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide 35

Java

Search

We focus on **Java** in this class.

Security Announcements

Receiving Security Advisories

Reporting Vulnerabilities

Security Related Documentation

Fixed Security Vulnerabilities

Java Serialization, Fixed in Akka 2.4.17

Camel Dependency, Fixed in Akka 2.5.4

Getting Started Guide

Introduction to Akka

Why modern systems need a new programming model

How the Actor Model Meets the Needs of Modern, Distributed Systems

Overview of Akka libraries and modules

Introduction to the Example

Akka 2.5.16

Part 3: Working with Device Actors

Part 4: Working with Device Groups

Part 5: Querying Device Groups

The modern **Akka** API with typed actors

General Concepts

Terminology, Concepts

Actor Systems

What is an Actor?

Supervision and Monitoring

Actor References, Paths and Addresses

Location Transparency

Akka and the Java Memory Model

Message Delivery Reliability

Configuration

Default configuration

Security Announcements

Getting Started Guide

General Concepts

Actors

Cluster

Persistence (Event Sourcing)

Persistence (Durable State)

Streams

Discovery

Utilities

Other Akka modules

Package, Deploy and Run

Project Information

Akka Classic

The old **Akka Classic** API with untyped actors

Cluster

Introduction to Actors

Actor lifecycle

Interaction Patterns

Fault Tolerance

Actor discovery

Routers

Stash

Cluster Usage

Cluster Specification

Cluster Membership Service

Phi Accrual Failure Detector

Distributed Data

Cluster Singleton

Cluster Sharding

Persistence (Event Sourcing)

Event Sourcing

Replicated Event Sourcing

CQRS

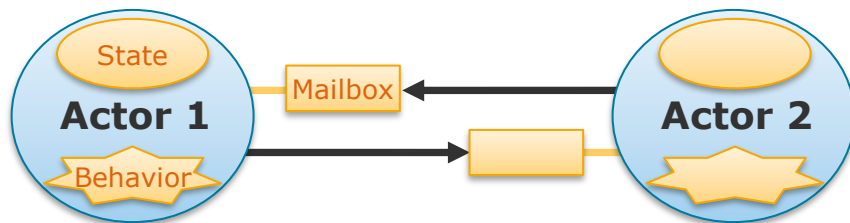
Style Guide

Snapshotting

Testing

Basic Concepts

Akka Actors



- Actor = State + Behavior + Mailbox
- Communication:
 - Sending messages to mailboxes
 - Non-blocking, fire-and-forget
- Messages:
 - Immutable, serializable objects
 - Object classes are known to both sender and receiver
 - Receiver interprets a message via pattern matching

Mutable messages are possible,
but don't use them!

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **37**

```
public class Worker extends AbstractBehavior<Worker.Message> {

    public interface Message {
```

Type for actor interface

Inherit default actor properties
(i.a. mailbox)

```
@Getter @NoArgsConstructor @AllArgsConstructor
```

Java lombok code generation aspects

```
public static class WorkMessage implements Message, Serializable {
    private static final long serialVersionUID = 7041548741032696121L;
    int task;
}
```

Messages that the actor can receive

```
public static Behavior<Message> create() {
    return Behaviors.setup(context -> new Worker(context));
}
```

Actor factory method(s)

```
private Worker(ActorContext<Message> context) {
    super(context);
}
```

Actor constructor(s)

Calls the createReceive() and sets
result as the actor's behavior

```
private final List<WorkMessage> work;
```

The Receive class performs
pattern matching and
(de-)serialization

Actor state

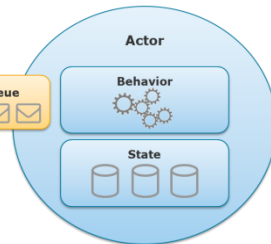
```
@Override
```

```
public Receive<Worker.Message> createReceive() {
    return newReceiveBuilder()
        .onMessage(WorkMessage.class, w -> {this.work.add(w); return this;})
        .build();
}
```

A builder pattern
for constructing a
Receive object

Action that is called
every time a
WorkMessage is received

Actor behavior



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide 38

```
public class Worker extends AbstractBehavior<Worker.Message> {
```

```
    public interface Message {  
    }
```

```
    @Getter @NoArgsConstructor @AllArgsConstructor
```

```
    public static class AddWorkMessage implements Message, Serializable {  
        private static final long serialVersionUID = 7041548741032696121L;  
        int task;  
    }
```

```
    @Getter @NoArgsConstructor @AllArgsConstructor
```

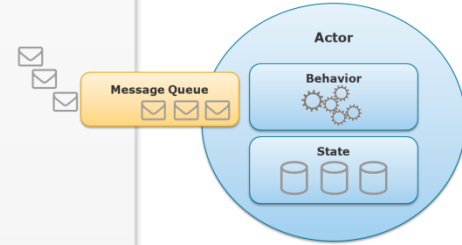
```
    public static class RemoveWorkMessage implements Message, Serializable {  
        private static final long serialVersionUID = 3481542741072693190L;  
        float task;  
    }
```

```
    ...
```

```
    @Override
```

```
    public Receive<Message> createReceive() {  
        return new ReceiveBuilder()  
            .onMessage(AddWorkMessage.class, w -> {this.work.add(w); return this;})  
            .onMessage(RemoveWorkMessage.class, w -> {this.work.remove(w); return this;})  
            .build();  
    }  
}
```

The message types (= classes)
define how the actor reacts



We can add more
messages if needed.

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide 39

```
public class Worker extends AbstractBehavior<Worker.Message> {
```

```
...
```

```
@Override
```

```
public Receive<Message> createReceive() {  
    return new ReceiveBuilder()  
        .onMessage(WorkMessage.class, this::handle)  
        .onMessage(RewardMessage.class, this::handle)  
        .build();  
}
```

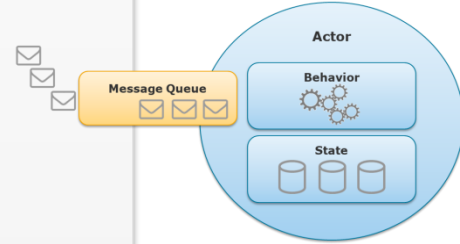
Reference to longer
behavior functions

```
private Behavior<Message> handle(WorkMessage message) {  
    for (int i = 0; i < message.getTask(); i++)  
        this.results.add(i * i);  
  
    this.getContext().getLog().info("Did my work!");  
  
    return this;  
}
```

Context offers access to logging facility, own ActorRef,
actor hierarchy, actor system, actor schedulers, ...

```
private Behavior<Message> handle(RewardMessage message) {  
    this.getContext().getLog().info("Thanks for the " + message.getReward() + " reward!");  
  
    return this;  
}
```

Return provides the Behavior of this actor;
return a new behavior, if the behavior should change.



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide 40

```
public class Worker extends AbstractBehavior<Worker.Message> {

    public static final String DEFAULT_NAME = "worker";

    ...

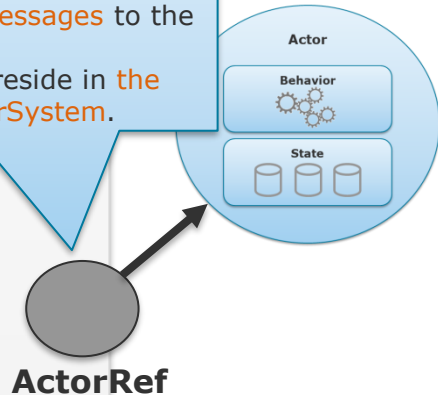
    @Override
    public Receive<Message> createReceive() {
        return newReceiveBuilder()
            .onMessage(SpawnMessage.class, this::handle)
            .build();
    }

    private Behavior<Message> handle(SpawnMessage message) {
        ActorRef<Message> childWorker = this.getContext()
            .spawn(Worker.create(), Worker.DEFAULT_NAME);

        childWorker.tell(new WorkMessage(42));

        return this;
    }
}
```

- An **immutable and serializable handle** to an actor (kind of a symbolic link)
- **Decouples actors** from each other and from their locations still allowing them to target one another.
- Can be **used to send messages** to the referenced actor.
- Referenced actor may reside in **the same or a remote ActorSystem**.



Create a new actor
(as a child of this actor)

ActorRef is a typed, abstract,
location-transparent pointer to an
Akka actor

Send a message to the new worker actor via its ActorRef
(asynchronously, non-blocking)

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **41**

```
public class Worker extends AbstractBehavior<Worker.Message> {
```

```
    public static Behavior<Message> create(ActorRef<Master.Message> master) {  
        return Behaviors.setup(context -> new Worker(context, master));  
    }
```

```
    private Worker(ActorContext<Message> context, ActorRef<Master.Message> master) {  
        super(context);  
        this.contacts.add(master);  
    }
```

Receive an ActorRef in constructor

@Getter @NoArgsConstructor @AllArgsConstructor

```
    public static class ContactMessage implements Message, Serializable {  
        private static final long serialVersionUID = 3481542741072693190L;  
        ActorRef<Master.Message> contact;  
    }
```

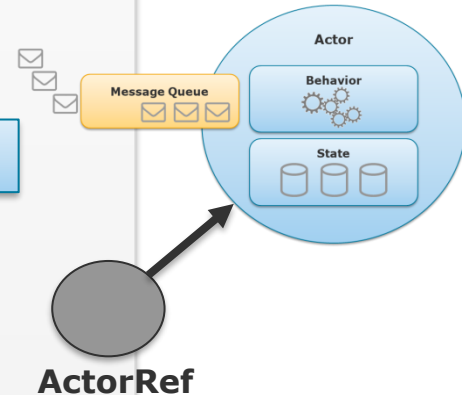
All further example codes omit the "...";
they are implicitly not complete!

```
    private Behavior<Message> handle(ContactMessage message) {  
        ActorRef<Master.Message> master = message.getContact();  
        this.contacts.add(master);
```

Receive an ActorRef in a message

```
        ActorRef<Worker.Message> me = this.getContext().getSelf();  
        master.tell(new Master.ContactMessage(me));  
        return this;  
    }
```

Send an ActorRef



Big Data Systems

```

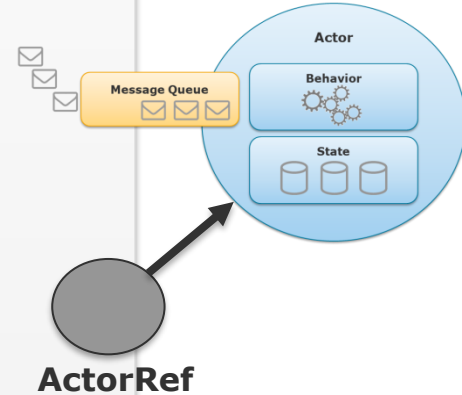
public class Master extends AbstractBehavior<Master.Message> {

    public static final ServiceKey<Master.Message> masterService =
        ServiceKey.create(Master.Message.class, "masterService");

    private Master(ActorContext<Message> context) {
        super(context);
        context.getSystem().receptionist().tell(
            Receptionist.register(masterService, context.getSelf()));
    }
}

```

Offer own ActorRef
via Akka's **receptionist**



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **43**


```
public class Master extends
```

```
    public static final ServiceKey  
    ServiceKey.create(Master
```

```
private Master(ActorContext  
    super(context);  
    context.getSystem().receptionist().register(this)  
}
```

Offer own ActorRef
via Akka's **receptionist**

Receive an ActorRef
via Akka's **receptionist**

```
public class Worker extends AbstractBehavior<Worker.Message> {
```

```
    @Getter @NoArgsConstructor @AllArgsConstructor
```

```
    public static class ReceptionistListingMessage implements Message {  
        Receptionist.Listing listing;  
    }
```

```
private Worker(ActorContext<Message> context) {  
    super(context);
```

```
    final ActorRef<Receptionist.Listing> listingResponseAdapter =  
        context.messageAdapter(Receptionist.Listing.class, ReceptionistListingMessage::new);  
    context.getSystem().receptionist().tell(  
        Receptionist.subscribe(Master.masterService, listingResponseAdapter));  
}
```

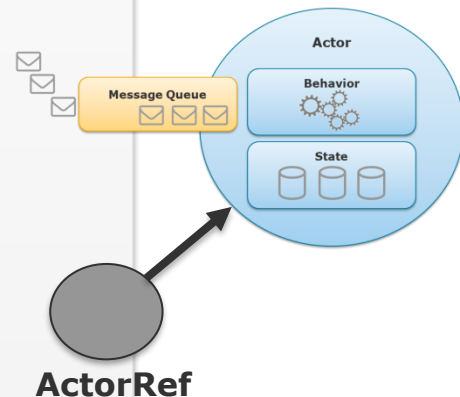
```
    @Override
```

```
    public Receive<Message> createReceive() {  
        return new ReceiveBuilder()  
            .onMessage(ReceptionistListingMessage.class, this::handle)  
            .build();  
    }
```

```
private Behavior<Message> handle(ReceptionistListingMessage message) {  
    Set<ActorRef<Master.Message>> masters =  
        message.getListing().getServiceInstances(Master.masterService);  
    return this;  
}
```

```
public class Worker extends AbstractBehavior<Worker.Message> {  
  
    @Getter @NoArgsConstructor  
    public static class TerminateMessage implements Message, Serializable {  
        private static final long serialVersionUID = 8483542741052698192L;  
    }  
  
    ...  
  
    private Behavior<Message> handle(TerminateMessage message) {  
        return Behaviors.stopped();  
    }  
}
```

Terminate an actor by returning Behaviors.stopped()



Big Data Systems

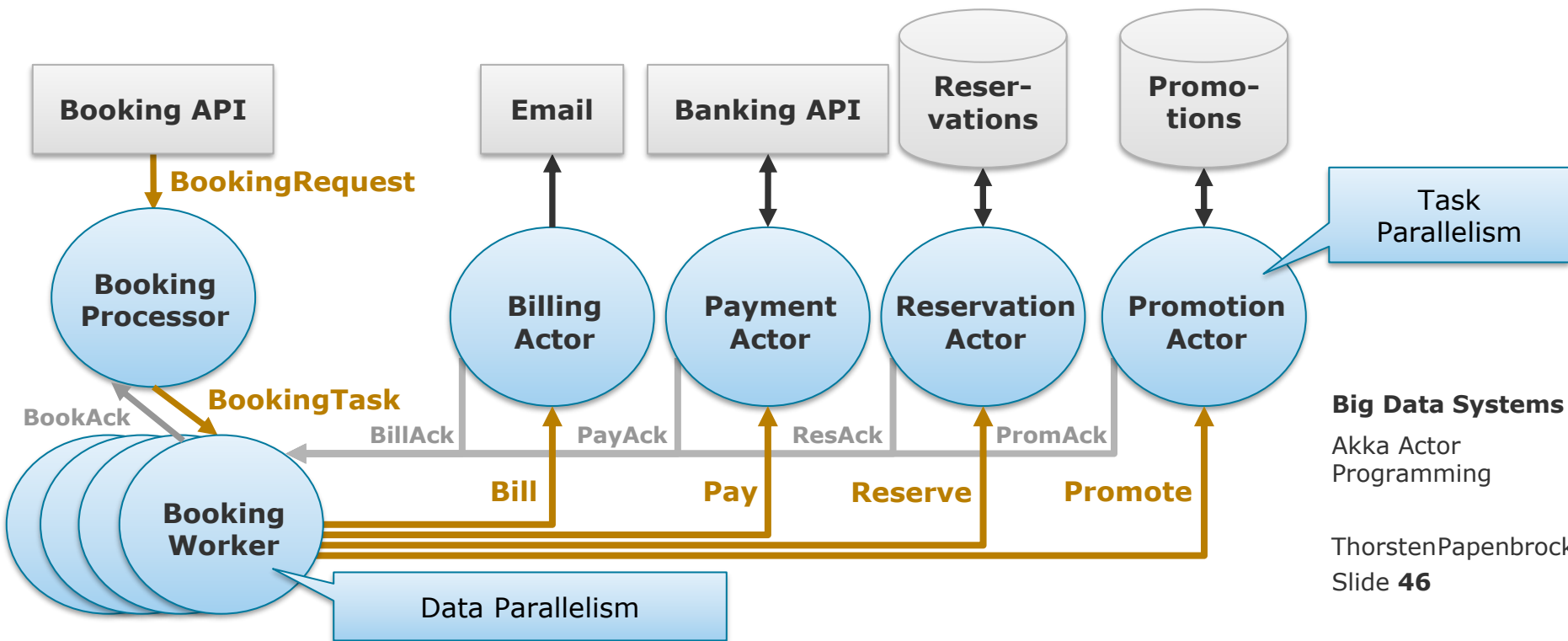
Akka Actor
Programming

ThorstenPapenbrock
Slide **45**

Actor Model

Actor Model Example

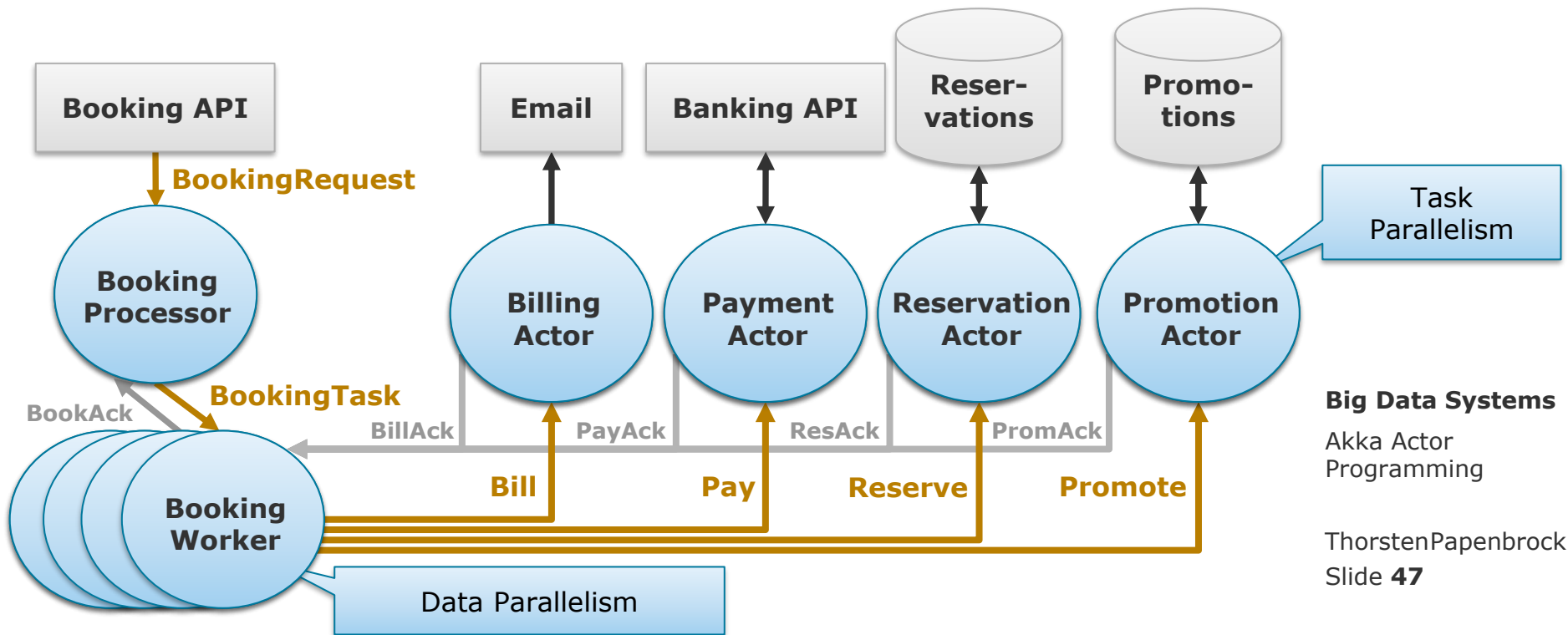
A flight booking system



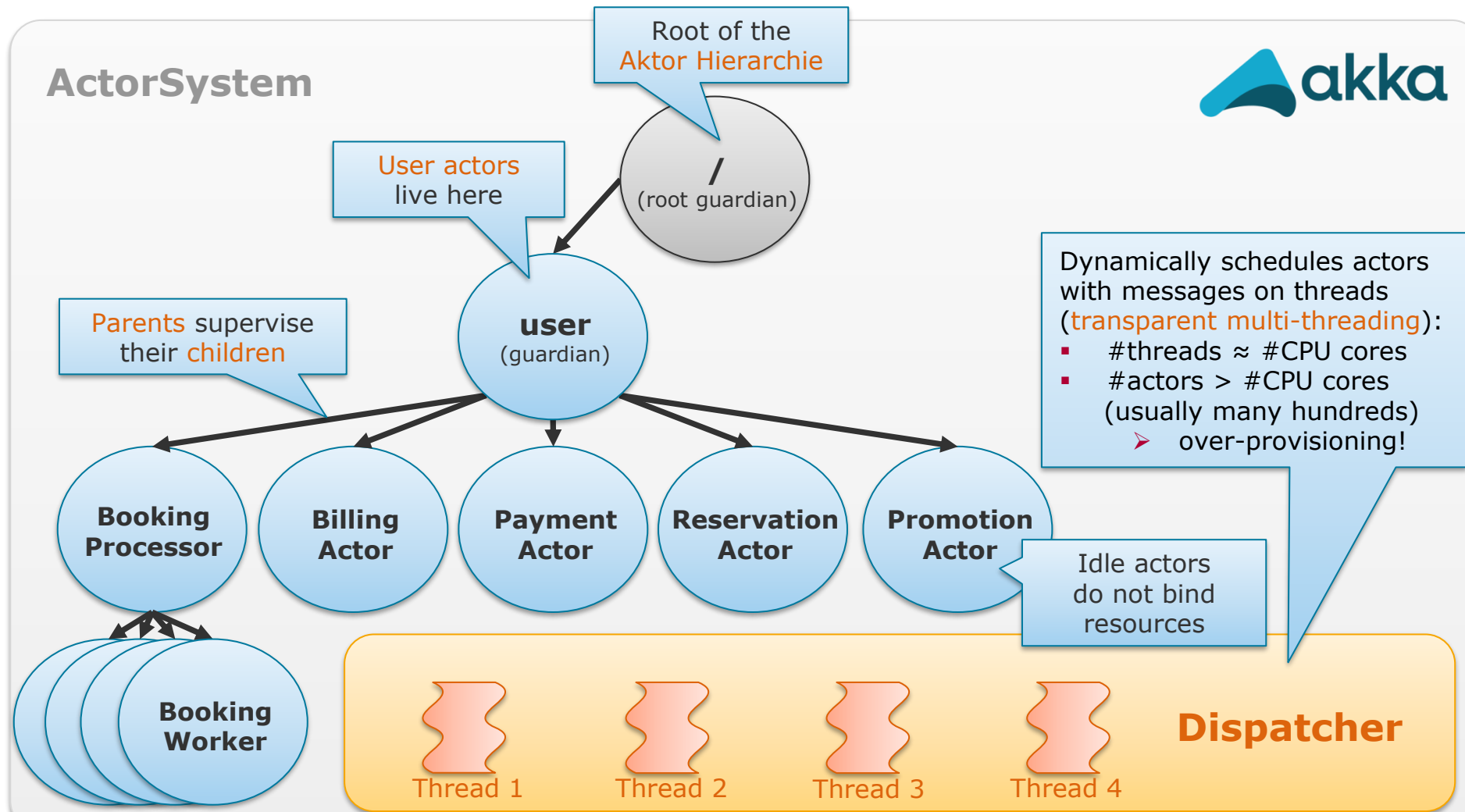
Actor Model

Actor Model Example

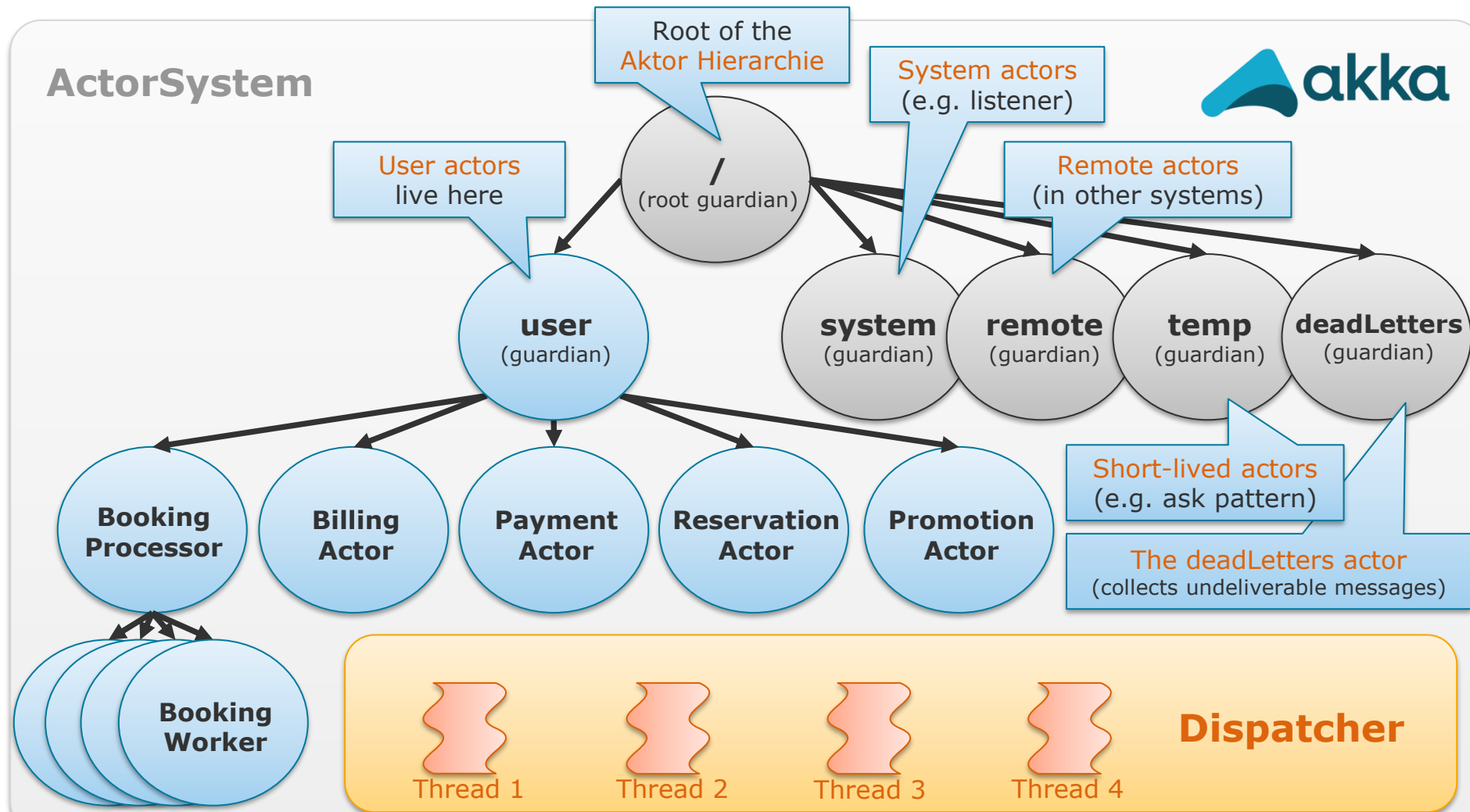
A flight booking system



ActorSystem



ActorSystem

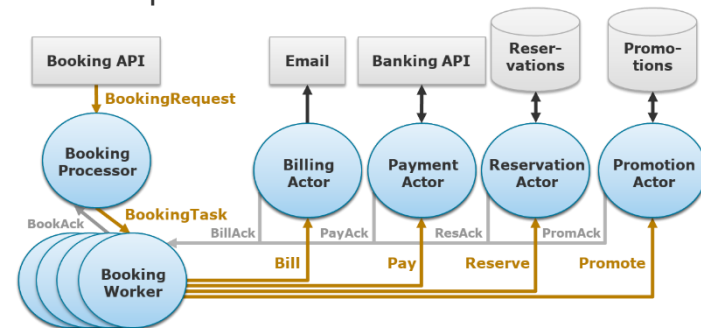


Actor Model

Actor Programming

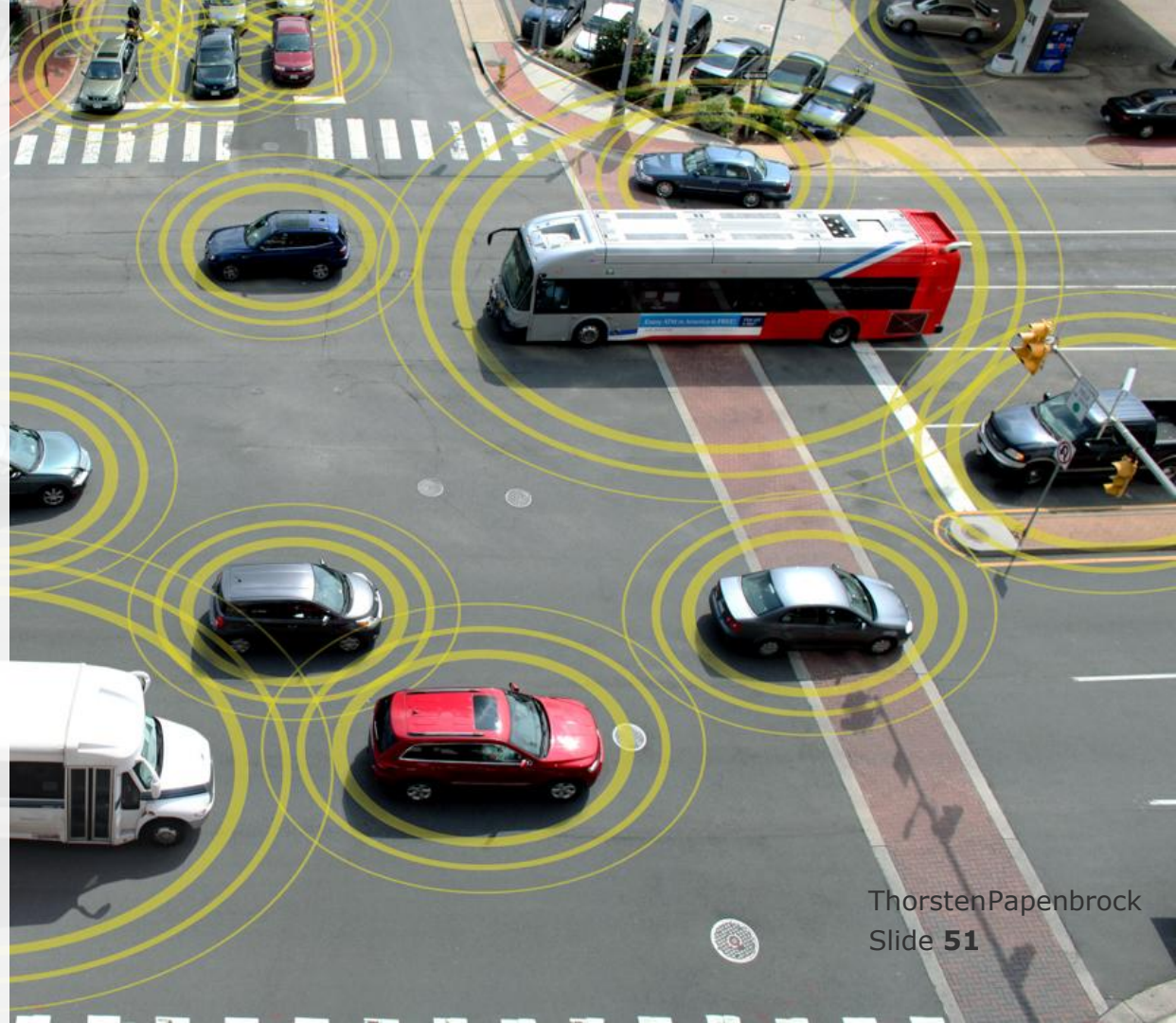
Reactive Programming

- A declarative programming paradigm based on event and message flows.
- Program components (= actors) declare how they react on certain messages.
- Instead of following a fixed calculation plan, components behave dynamically and independent of each other.
 - Writing a reactive algorithm is more like declaring rules for how to react on certain input changes rather than defining a step-by-step execution plan.
- Reactions are triggered by data or changes in the environment.
- Reaction:
 - changing state (= private variables)
 - changing behavior (= private code)
 - changing algorithm topology (= child actors)
 - sending further messages
- Reactivity helps to optimize resource utilization, robustness and elasticity.



Akka Actor Programming Hands-on

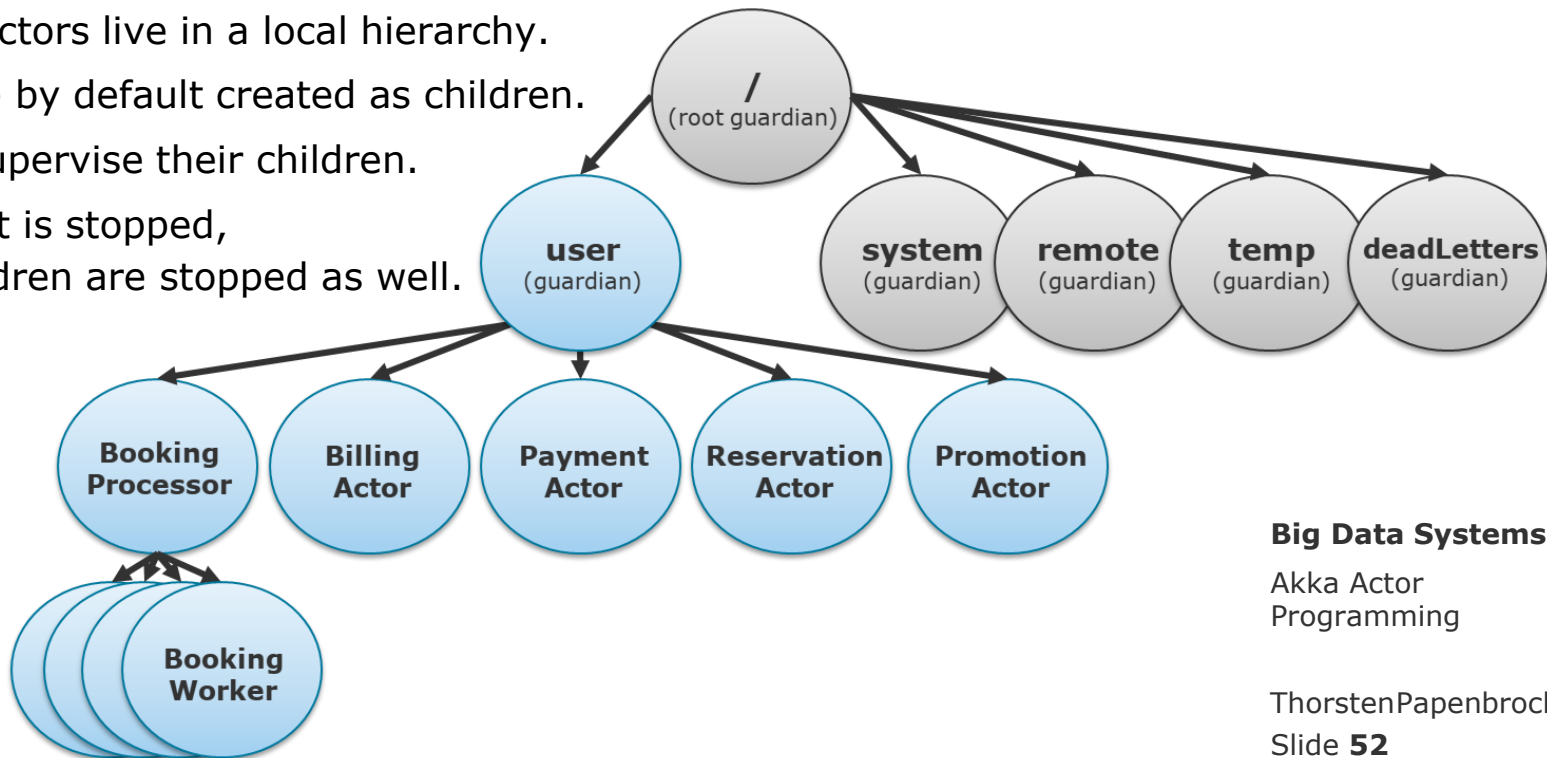
- Actor Model
- Basic Concepts
- **Runtime Architecture**
- Demo
- Messaging
- Parallelization
- Remoting
- Clustering
- Patterns
- Homework



Runtime Architecture

Actor Hierarchies

- All Akka actors live in a local hierarchy.
- Actors are by default created as children.
- Parents supervise their children.
- If a parent is stopped, all its children are stopped as well.



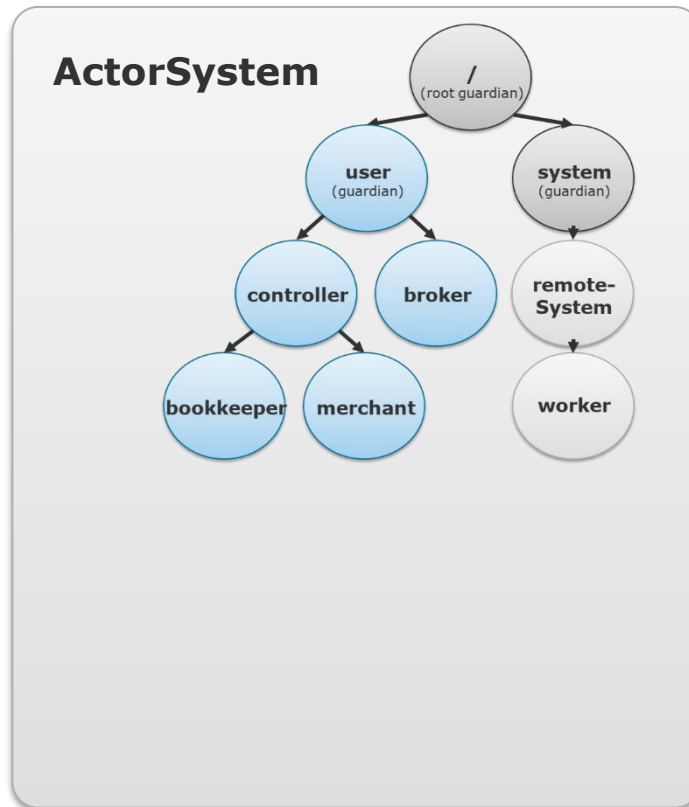
Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **52**

Runtime Architecture Actor Systems

- A named hierarchy of actors
- Runs within one JVM process
- Configures:
 - Global actor settings
(e.g. mailbox types)
 - Actor dispatchers
(that schedule actors on threads)
 - Remote actor access
(e.g. addresses)
 - ...



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **53**

Runtime Architecture Actor Systems

Event stream

- Reacts on errors, new nodes, message sends, message loss, ...

Dispatcher

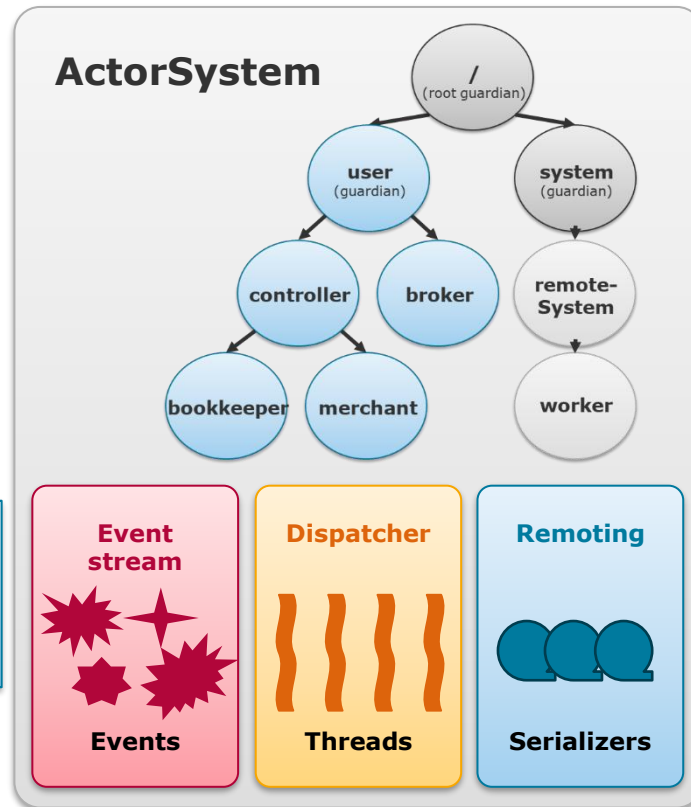
- Assigns threads dynamically to actors.
- Transparent multi-threading
 - # Threads $\approx$ # CPU cores
 - # Actors > # CPU cores (usually many hundreds)

- Over-provisioning!

Remoting

- Resolves remote actor addresses.
- Sends messages over network.
 - serialization + de-serialization

Idle actors bind
~400 bytes of
heap space but
no CPU resources



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **54**

Runtime Architecture

Actor Hierarchies

Task- and data-parallelism

Delegate work!

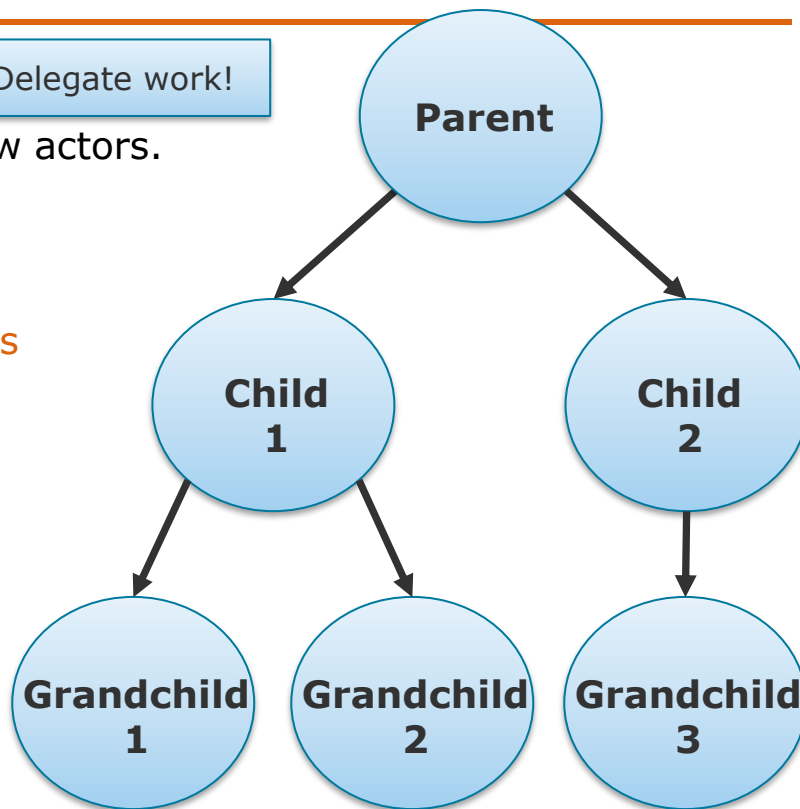
- Actors can dynamically create new actors.

Supervision hierarchy

- Creating actor (parent) **supervises** created actor (child).

Fault-tolerance

- If child fails, parent can choose:
 - restart**, **resume**, **stop**, or **escalate**



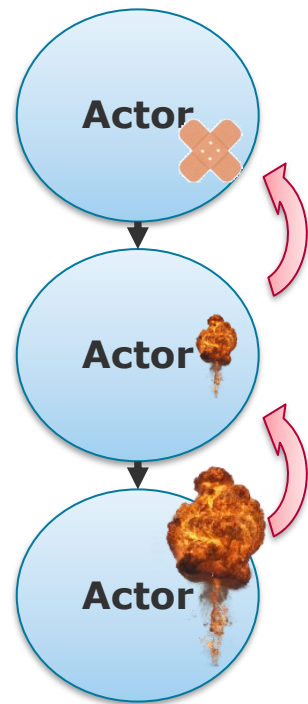
Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **55**

“Let it crash” philosophy

- Distributed systems are inherently prone to errors (because there is simply more to go wrong/break).
 - Message loss, unreachable mailboxes, crashing actors ...
- Make sure that critical code is supervised by some entity that knows how errors can be handled.
 - Actor hierarchies
- Then, if an error occurs, do not (desperately) try to fix it: let it crash!
 - Errors are propagated to supervisors that can deal better with them.
- Example: Actor loses a database connection due to a DB restart.
 - It decides to crash.
 - Its supervisor restarts the actor, which re-creates the DB connection.

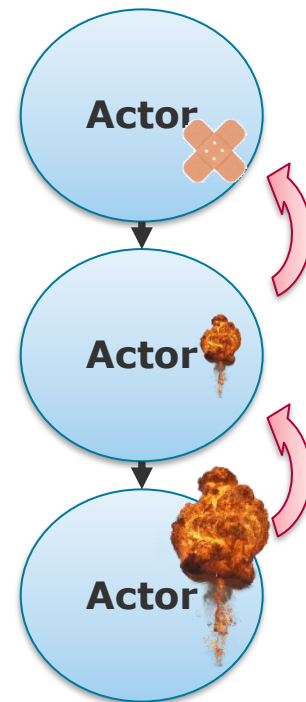
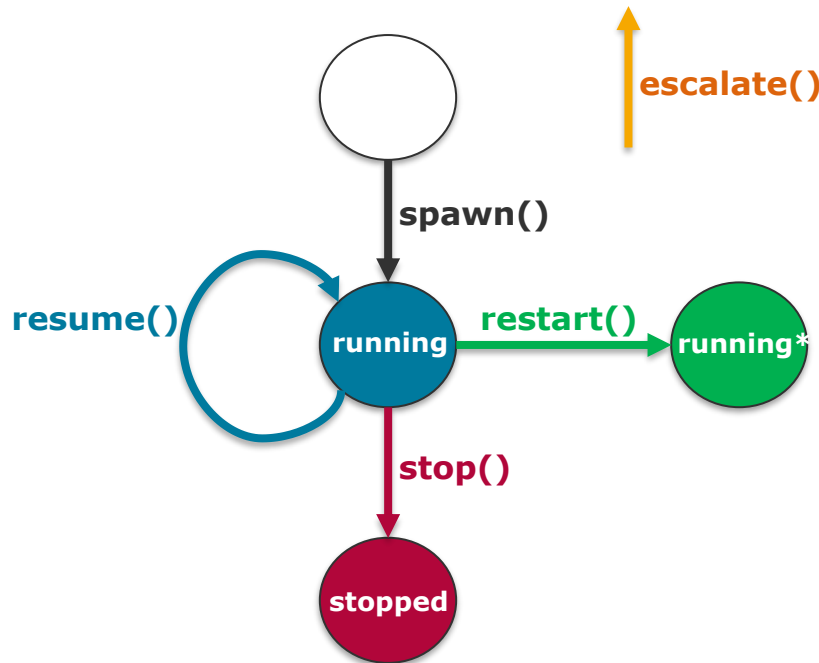


Runtime Architecture

Actor Hierarchies and Fault Tolerance

Lifecycle management

- Supervisor actions:
 - resume()**
 - Resume actor with same state.
 - restart()**
 - Restart actor with fresh state.
 - stop()**
 - Stop actor and remove object.
 - escalate()**
 - Escalate exception to parent if local supervisor actions fail.

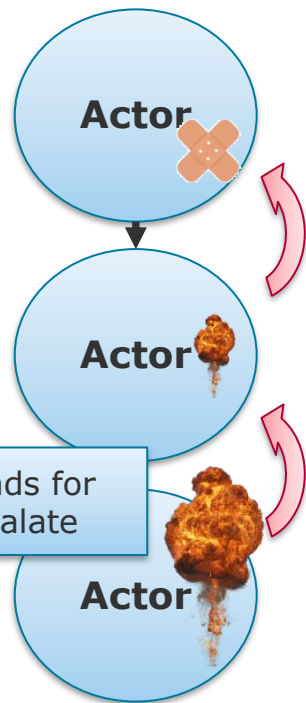



```
private ActorRef<Worker.Message> startWorker(int id) {  
    Behavior<Worker.Message> workerBehavior = Worker.create();  
    ActorRef<Worker.Message> worker = this.getContext()  
        .spawn(workerBehavior, Worker.DEFAULT_NAME + id);  
    return worker;  
}
```

```
private ActorRef<Worker.Message> startSupervisedWorker(int id) {  
    Behavior<Worker.Message> workerBehavior = Worker.create();  
    Behavior<Worker.Message> supervisedWorkerBehavior = Behaviors.supervise(workerBehavior)  
        .onFailure(IOException.class, SupervisorStrategy.restart()  
            .withLimit(3, Duration.ofSeconds(9)));  
    ActorRef<Worker.Message> worker = this.getContext()  
        .spawn(supervisedWorkerBehavior, Worker.DEFAULT_NAME + id);  
    return worker;  
}
```

Try 3 restarts in 9 seconds for
IOExceptions; then escalate

```
SupervisorStrategy.restart()  
SupervisorStrategy.resume()  
SupervisorStrategy.stop()
```




```
public class Worker extends AbstractBehavior<Worker.Message> {
```

```
...
```

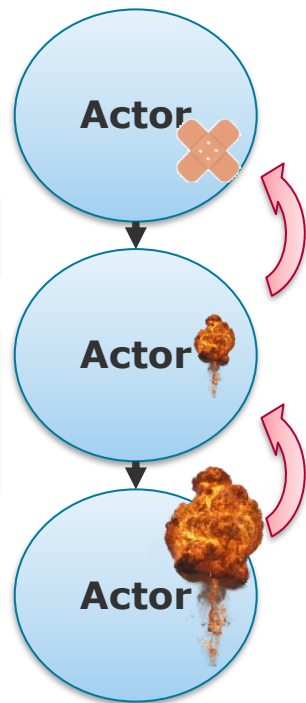
```
@Override
```

```
public Receive<Message> createReceive() {  
    return newReceiveBuilder()  
        .onMessage(WorkMessage.class, this::handle)  
        .onMessage(RewardMessage.class, this::handle)  
        .onSignal(PostStop.class, this::handle)  
        .onSignal(PreRestart.class, this::handle)  
        .build();  
}
```

```
private Behavior<Message> handle(PostStop signal) throws IOException {  
    this.fileReader.close();  
    return this;  
}  
}
```

Signal = "Environment Message"

PostStop and PreRestart allow for some housekeeping when lifecycle changes occur.



Runtime Architecture

Actor Hierarchies and Fault Tolerance

```
public class Worker extends AbstractBehavior<Worker.Message> {  
  
    @Override  
    public Receive<Message> createReceive() {  
        return newReceiveBuilder()  
            .onMessage(ContactMessage.class, this::handle)  
            .onSignal(Terminated.class, this::handle)  
            .build();  
    }  
  
    private Behavior<Message> handle(ContactMessage message) {  
        ActorRef<Master.Message> master = message.getContact();  
        this.contacts.add(master);  
        this.getContext().watch(master);  
        return this;  
    }  
  
    private Behavior<Message> handle(Terminated signal) {  
        ActorRef<Master.Message> master = signal.getRef().unsafeUpcast();  
        this.contacts.remove(master);  
        return this;  
    }  
}
```

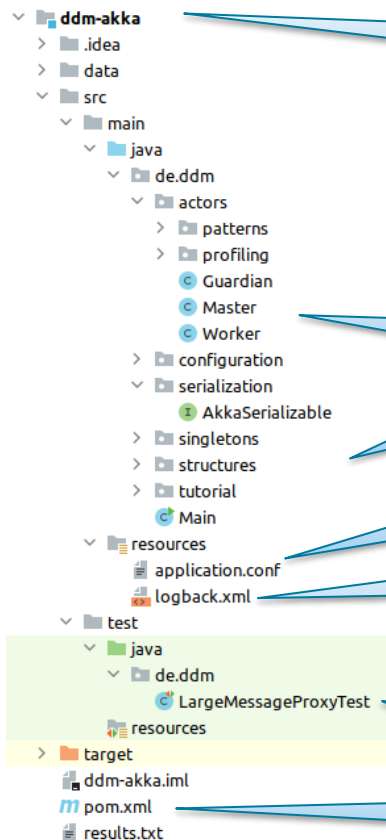
Watching vs. Supervision

By watching an actor, the ActorSystem will send a Terminated signal, if the watched actor becomes **terminated** or **unreachable**.

Any actor can watch any other actor to assure that the ActorRefs still work.

Runtime Architecture

ActorSystem Setup



A common Java maven project

Akka actors (and other classes)

```
public static void main(String[] args)
```

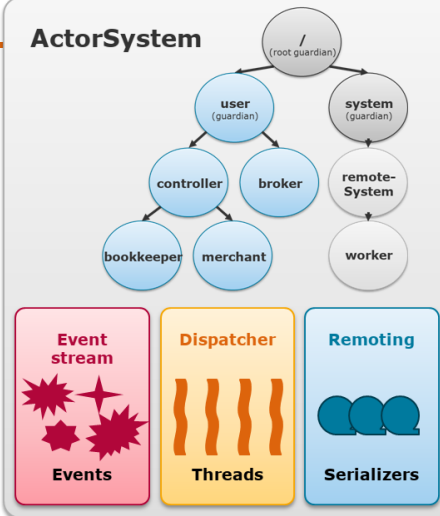
Akka application configuration

Logging configuration

Unit testing for akka actors and their protocols

Project dependencies (i.a. [akka](#) and [lombok](#)) in maven

ActorSystem



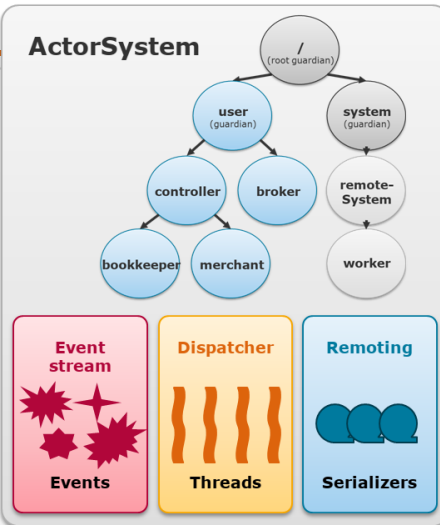
Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **61**

Runtime Architecture

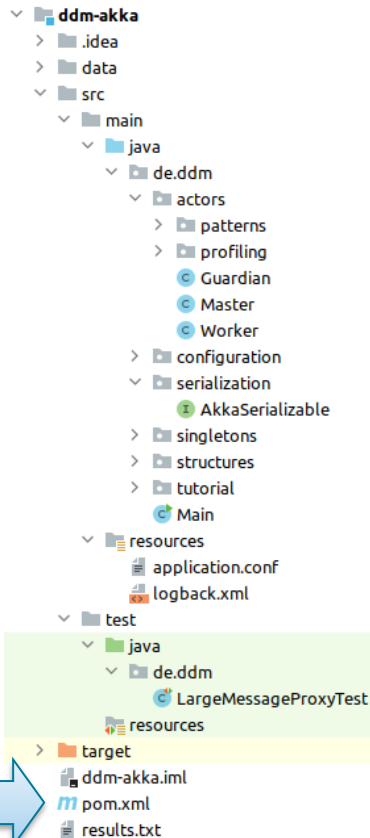
ActorSystem Setup



The Scala Build Tool (**sbt**)
is an alternative to **maven**

Thorsten Papenbrock
Slide 62

pom.xml

[illegible]

Runtime Architecture

ActorSystem Setup

Base typed actor library
actors, supervision, scheduling, ...

Remoting and clustering library
remote actors, heartbeats ...

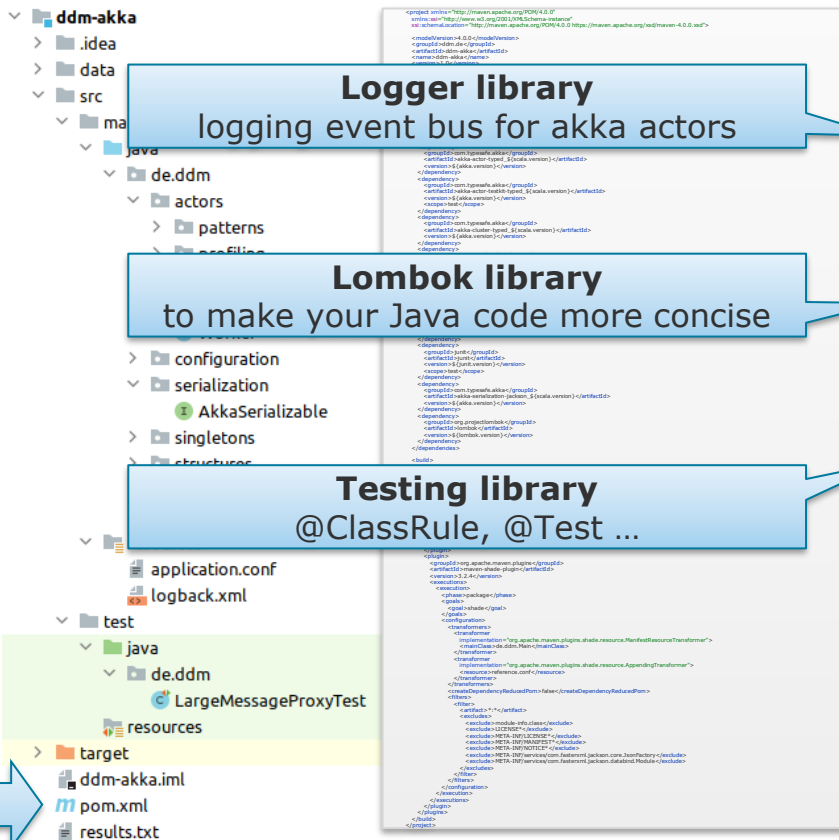
Streaming library
actor-backed streams, backpressure ...

Serialization library
Custom serialization with jackson

Testing library
TestKit class, expecting messages, ...

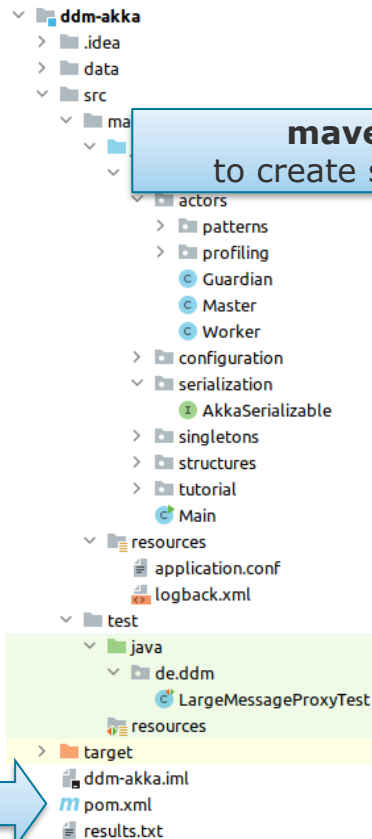
```
<dependency>
  <groupId>com.typesafe.akka</groupId>
  <artifactId>akka-actor-typed_${scala.version}</artifactId>
  <version>${akka.version}</version>
</dependency>
<dependency>
  <groupId>com.typesafe.akka</groupId>
  <artifactId>akka-remote_${scala.version}</artifactId>
  <version>${akka.version}</version>
</dependency>
<dependency>
  <groupId>com.typesafe.akka</groupId>
  <artifactId>akka-cluster-typed_${scala.version}</artifactId>
  <version>${akka.version}</version>
</dependency>
<dependency>
  <groupId>com.typesafe.akka</groupId>
  <artifactId>akka-stream-typed_${scala.version}</artifactId>
  <version>${akka.version}</version>
</dependency>
<dependency>
  <groupId>com.typesafe.akka</groupId>
  <artifactId>akka-serialization-jackson_${scala.version}</artifactId>
  <version>${akka.version}</version>
</dependency>
<dependency>
  <groupId>com.typesafe.akka</groupId>
  <artifactId>akka-actor-testkit-typed_${scala.version}</artifactId>
  <version>${akka.version}</version>
  <scope>test</scope>
</dependency>
```

Runtime Architecture ActorSystem Setup



```
<dependency>
  <groupId>ch.qos.logback</groupId>
  <artifactId>logback-classic</artifactId>
  <version>${logback.version}</version>
</dependency>
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <version>${lombok.version}</version>
</dependency>
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>${junit.version}</version>
  <scope>test</scope>
</dependency>
```

Runtime Architecture ActorSystem Setup



maven-shade-plugin
to create self-contained jar-files

```
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-shade-plugin</artifactId>
<version>3.2.1</version>
<configuration>
  <transformer>org.apache.maven.plugins.shade.resource.ManifestResourceTransformer</transformer>
</configuration>
</plugin>
```

```
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-shade-plugin</artifactId>
<version>3.2.1</version>
<configuration>
  <transformer>org.apache.maven.plugins.shade.resource.ManifestResourceTransformer</transformer>
</configuration>
</plugin>
```

<build>

<plugins>

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-compiler-plugin</artifactId>

...

</plugin>

<plugin>

<groupId>org.apache.maven.plugins</groupId>

<artifactId>maven-shade-plugin</artifactId>

<executions>

<execution>

<phase>package</phase>

<goals>

<goal>shade</goal>

</goals>

<configuration>

<transformers>

<transformer implementation="...">

<mainClass>de.ddm.Main</mainClass>

</transformer>

<transformer implementation="...">

<resource>reference.conf</resource>

</transformer>

</transformers>

...

</configuration>

</execution>

</executions>

</plugin>

</plugins>

</build>

pom.xml

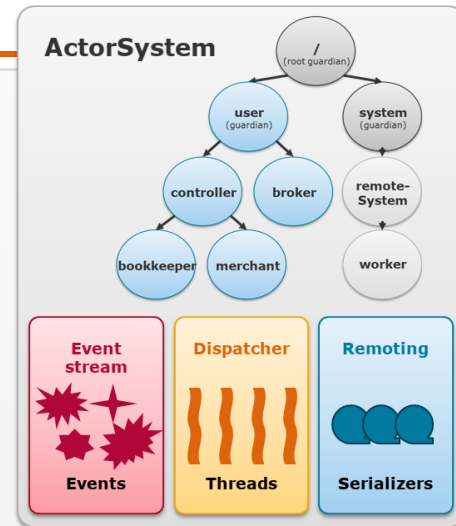
See [reference configuration](https://doc.akka.io/docs/akka/current/general/configuration-reference.html) for many more options:
<https://doc.akka.io/docs/akka/current/general/configuration-reference.html>



or local or remote

or udp	hostname or IP
---------------	----------------

} ... of this actor system



Big Data Systems

Akka Actor Programming

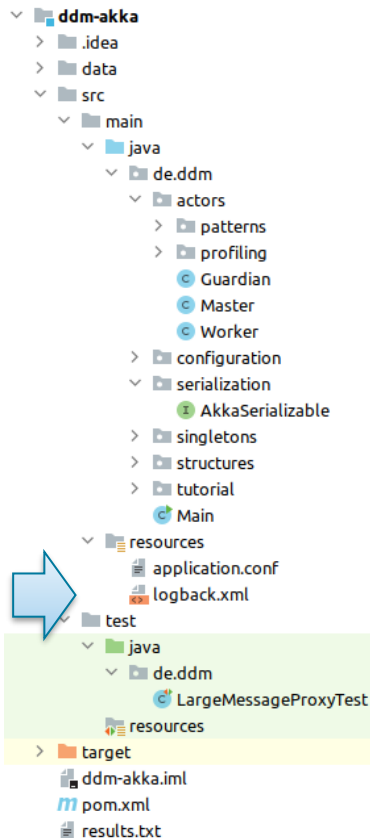
Thorsten Papenbrock
Slide 66

application.conf

Runtime Architecture

ActorSystem Setup

Try omitting the
logback.xml to see very
verbose log outputs ;-)



```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<configuration>
```

```
  <appender name="STDOUT"
```

```
    class="ch.qos.logback.core.ConsoleAppender">
```

```
    <encoder>
```

```
      <pattern>
```

```
        [%d{HH:mm:ss.SSS} %-5level]
```

```
        %50.50X{akkaSource:-local}|
```

```
        %msg%n
```

```
      </pattern>
```

```
    </encoder>
```

```
  </appender>
```

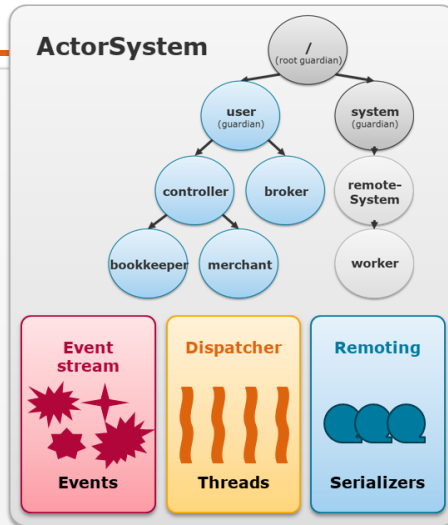
```
  <root level="INFO">
```

```
    <appender-ref ref="STDOUT" />
```

```
  </root>
```

```
</configuration>
```

ActorSystem

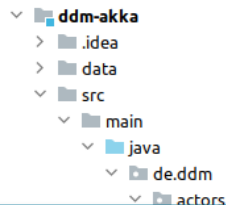


For optional formatting
and filtering of log
messages

logback.xml

Runtime Architecture

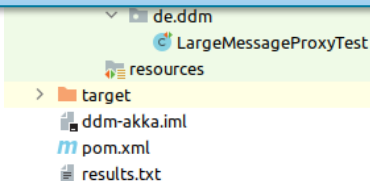
ActorSystem Setup



Create an ActorSystem with a specific actor as user guardian.

Start some action; ActorSystem is passive otherwise.

Main thread could end here and the ActorSystem would continue running.



Write config programmatically with application.conf as fallback

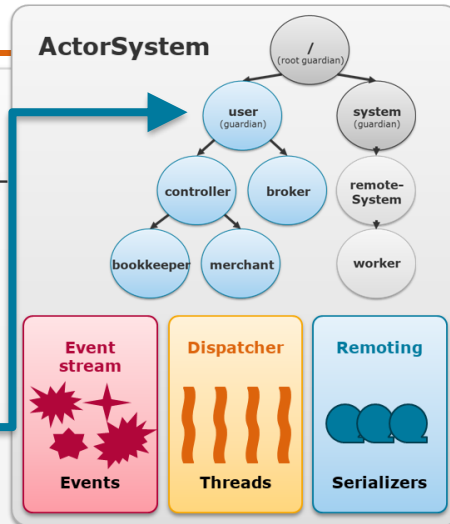
```
public static void main(String[] args) {  
    Config conf = ConfigFactory.parseString(""  
        "akka.remote.artery.canonical.hostname = 192.168.0.2\n" +  
        "akka.remote.artery.canonical.port = 9778\n" +  
        "akka.cluster.roles = [worker]\n" +  
        "akka.cluster.seed-nodes = \n" +  
        "\"[\"akka://MyAkkaApp@192.168.0.1:9777\"]\"")  
        .withFallback(ConfigFactory.load("application"));
```

```
    final ActorSystem<Guardian.Message> guardian =  
        ActorSystem.create(Guardian.create(), "MyAkkaApp", conf);
```

```
    guardian.tell(new Guardian.StartMessage());
```

```
    try {  
        System.out.println(">>> Press ENTER to exit <<<");  
        System.in.read();  
    } catch (IOException ignored) {}  
    finally {  
        guardian.tell(new Guardian.ShutdownMessage());  
    }  
    a) return Behaviors.stopped();  
    b) this.getContext().getSystem().terminate();  
}
```

ActorSystem

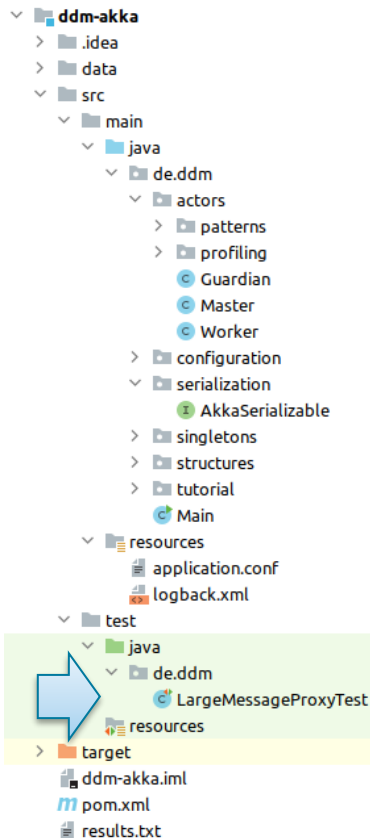


This message tells the user guardian to terminate the ActorSystem.

Main.java

Runtime Architecture

ActorSystem Testing



```
public class LargeMessageProxyTest {
```

Wrapps an ActorSystem into a TestKit.

@ClassRule

```
public static final TestKitJunitResource testKit =
    new TestKitJunitResource(Configuration.getMyConf());
```

@Test

```
public void testProtocol() {
    TestProbe<LargeMessageProxy.LargeMessage> probe =
        testKit.createTestProbe();
```

A probe to expect certain actor reactions.

Actor under test.

```
ActorRef<LargeMessageProxy.Message> largeMessageProxy =
    testKit.spawn(LargeMessageProxy.create(probe.getRef()),
        LargeMessageProxy.DEFAULT_NAME);
```

```
LargeMessageProxy.LargeMessage message = new MyMessage();
```

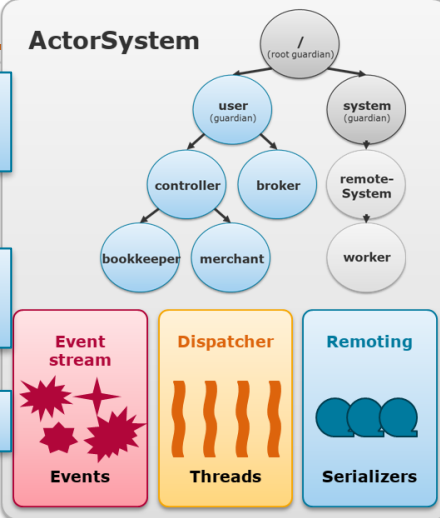
```
largeMessageProxy.tell(message);
```

The largeMessageProxy is supposed to simply loop the received message back.

```
probe.expectMessage(message);
probe.expectNoMessage();
```

LargeMessageProxyTest.java

ActorSystem



Big Data Systems

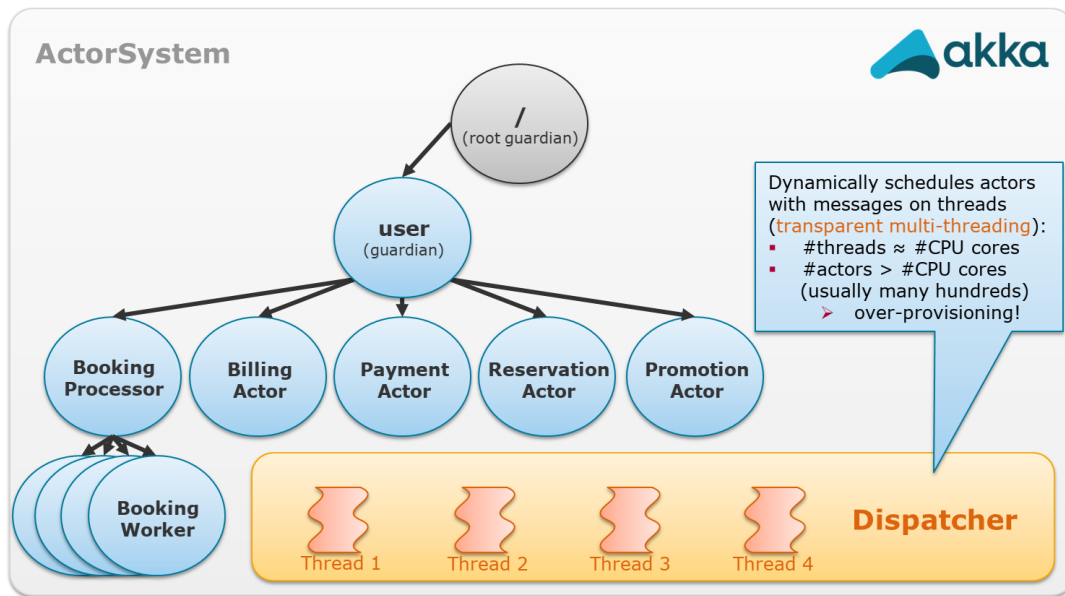
Akka Actor Programming

ThorstenPapenbrock
Slide 69

Runtime Architecture

Dispatching

- The ActorSystem schedules its actors on threads that live inside a dispatcher.
 - With no further care, all user actors are scheduled on the same **default dispatcher** (system actors have their own dispatcher).
 - But we might want to ...
 - control actor prioritization;
 - configure dispatching logic;
 - avoid starvation;
 - separate worker actors from application critical actors e.g. master, leader, shepherd;
 - pin actors to threads.
- Use custom dispatchers!



Runtime Architecture

Dispatching

- Configure a custom dispatcher in the [application.conf](#):

alternatives:

- `Dispatcher`
- `PinnedDispatcher`

alternatives:

- `fork-join-executor`
- `thread-pool-executor`

```
my-dispatcher {  
  type = Dispatcher  
  executor = "fork-join-executor"  
  fork-join-executor {  
    # Parallelism (threads) ... ceil(available processors * factor)  
    parallelism-factor = 2.0  
    # Min number of threads to cap factor-based parallelism number to  
    parallelism-min = 2  
    # Max number of threads to cap factor-based parallelism number to  
    parallelism-max = 10  
  }  
  # Throughput defines the maximum number of messages  
  # processed per actor before the thread jumps to the next actor  
  # Set to 1 for as fair as possible.  
  throughput = 100  
}
```

= number of
threads per core

```
my-other-dispatcher {  
  type = Dispatcher  
  executor = "thread-pool-executor"  
  thread-pool-executor {  
    fixed-pool-size = 32  
  }  
  throughput = 1}
```

- Dispatch actors on your custom dispatcher:
 - Option A: via code

```
ActorRef<Worker.Message> worker = this.getContext().spawn(  
    DependencyWorker.create(), "worker",  
    DispatcherSelector.fromConfig("my-dispatcher"));
```

- Option B: via configuration

```
akka.actor.deployment {  
  /user/worker {  
    dispatcher = my-dispatcher  
  }  
}
```

```
ActorRef<Worker.Message> worker = this.getContext().spawn(  
    DependencyWorker.create(), "worker");
```

Runtime Architecture

Dispatching

Uncontrolled thread dispatching is the main reason for unresponsive and unstable distributed, actor-based systems!

Advice: Work with custom dispatchers!

- A dedicated `PinnedDispatcher` with **one thread** may be an optimal choice for a central `Master/Leader/Scheduler/...` actor to optimize cache lines and availability for that special role.
- A `Dispatcher` on a `ThreadPool` with `#CPU_Cores - n` threads is a good execution model for `Worker` actors with long running jobs, because they will block at most `#CPU_Cores - n` cores and leave `n` cores for other system activities to keep the system reactive.
- Why use `Dispatchers` for parallelization control and not actor instances? (i.e. we could also limit the number of `Worker` actors to `#CPU_Cores - n`)
 - Answer: **overprovisioning** and **complex systems**
 - If we have more than one type of actor for long running jobs, e.g., `FileReadingWorkers` and `DatabaseReadingWorkers`, there should be in total only `#CPU_Cores - n` workers but we might want to max out either of the two types at times.

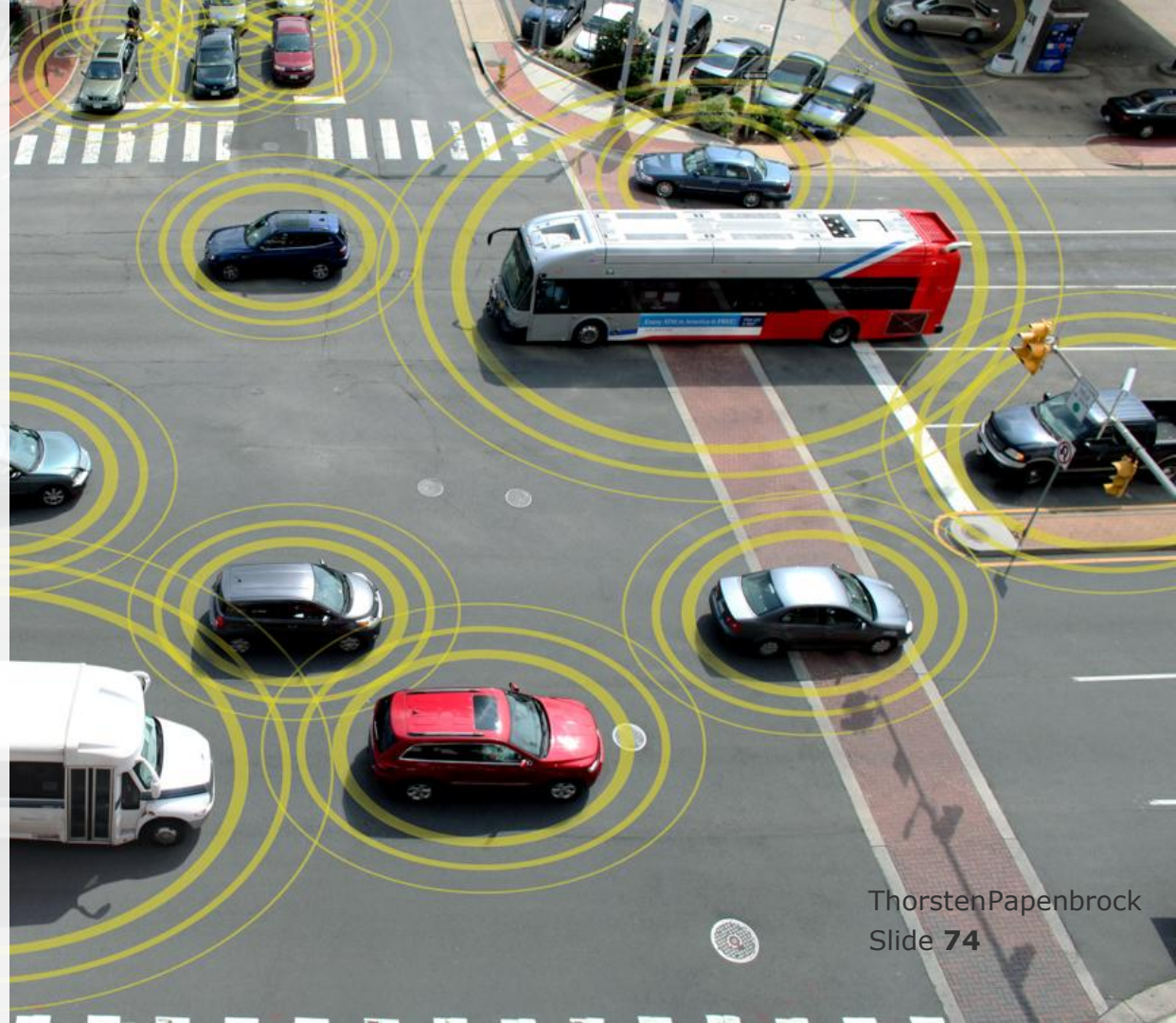
Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **73**

Akka Actor Programming Hands-on

- Actor Model
- Basic Concepts
- Runtime Architecture
- **Demo**
- Messaging
- Parallelization
- Remoting
- Clustering
- Patterns
- Homework



 **ddm-akka** Public

 Edit Pins

 Unwatch 1

 Fork 13

 Star 3

 main

 2 Branches

 0 Tags

 Add file

 Code

 thorsten-papenbrock Update README.md	053ad80 · 3 hours ago	🕒 15 Commits
📁 src	Added a flag for the enterShutdown, increased the logger st...	9 months ago
📄 .gitignore	Updated gitignore to ignore the solution files	8 months ago
📄 LICENSE	Migrated the ddm-akka projekt into this new repository.	last year
📄 README.md	Update README.md	3 hours ago
📄 pom.xml	Update libs and extend readme for token usage of akka	4 days ago

📖 README

 MIT license





ddm-akka

Akka example and homework code for the "Distributed Data Management" lecture.

Requirements

- Java Version >= 21
- Maven Compiler Version >= 3.14.1

Getting started

1. Clone repo

```
git clone https://github.com/UMR-Big-Data-Analytics/ddm-akka.git
```

2. Get the Akka libraries

About



Akka example and homework code for the "Distributed Data Management" lecture.

 Readme

 MIT license

 Activity

 Custom properties

 3 stars

 1 watching

 13 forks

Report repository

Releases

No releases published
[Create a new release](#)

Packages

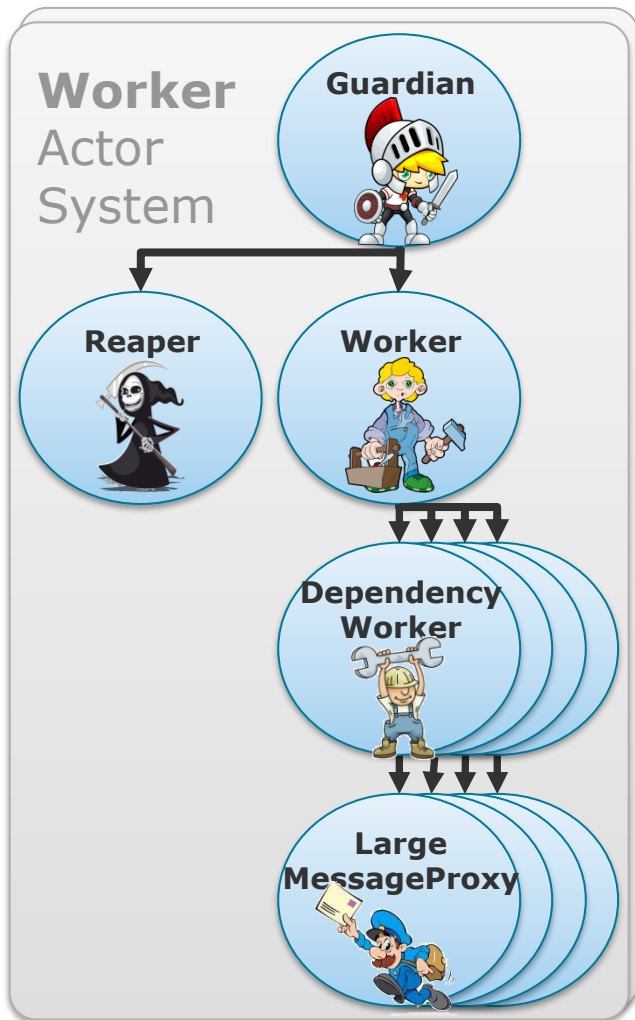
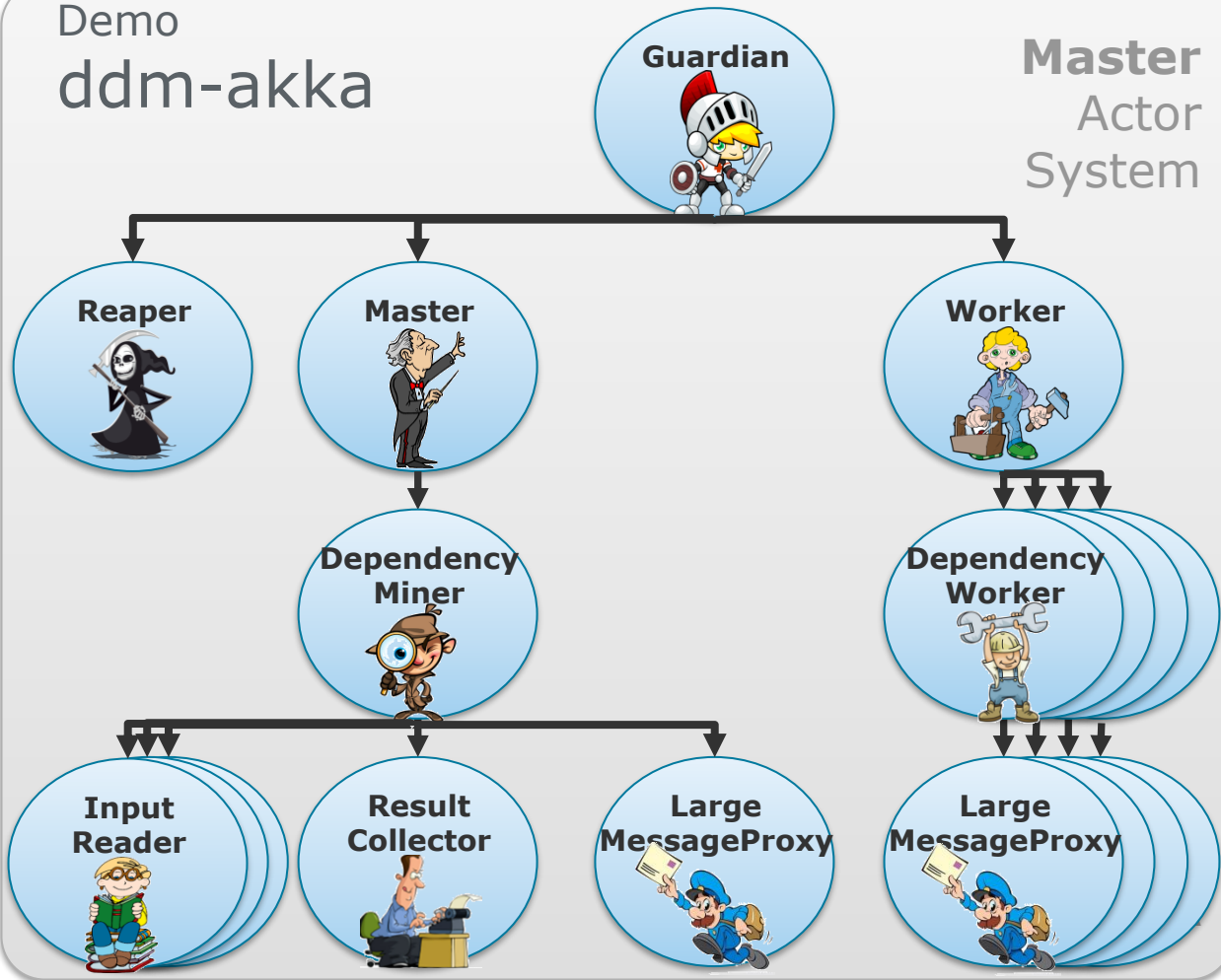
No packages published
[Publish your first package](#)

Contributors 2

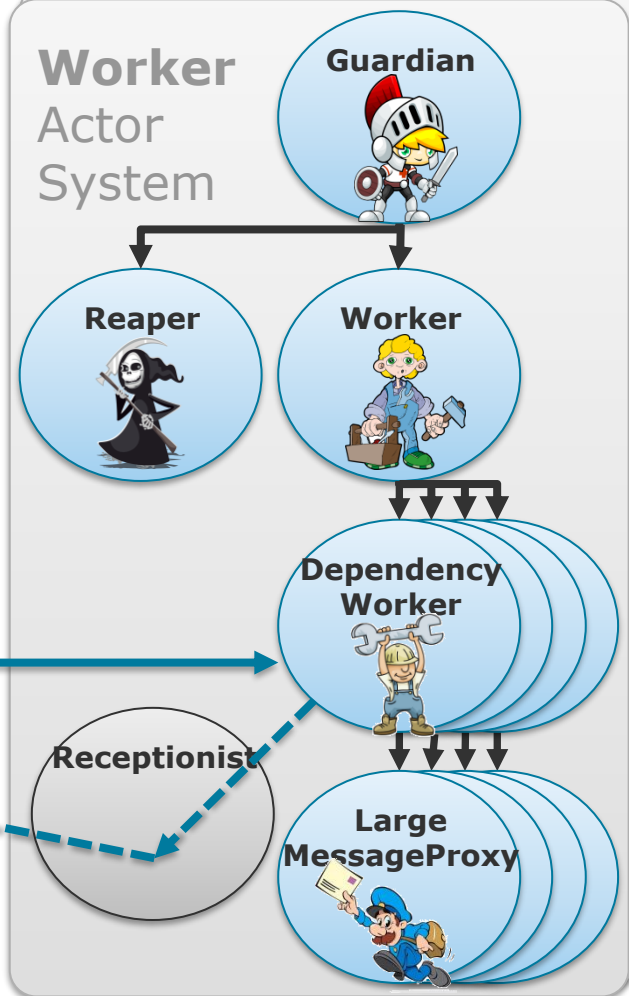
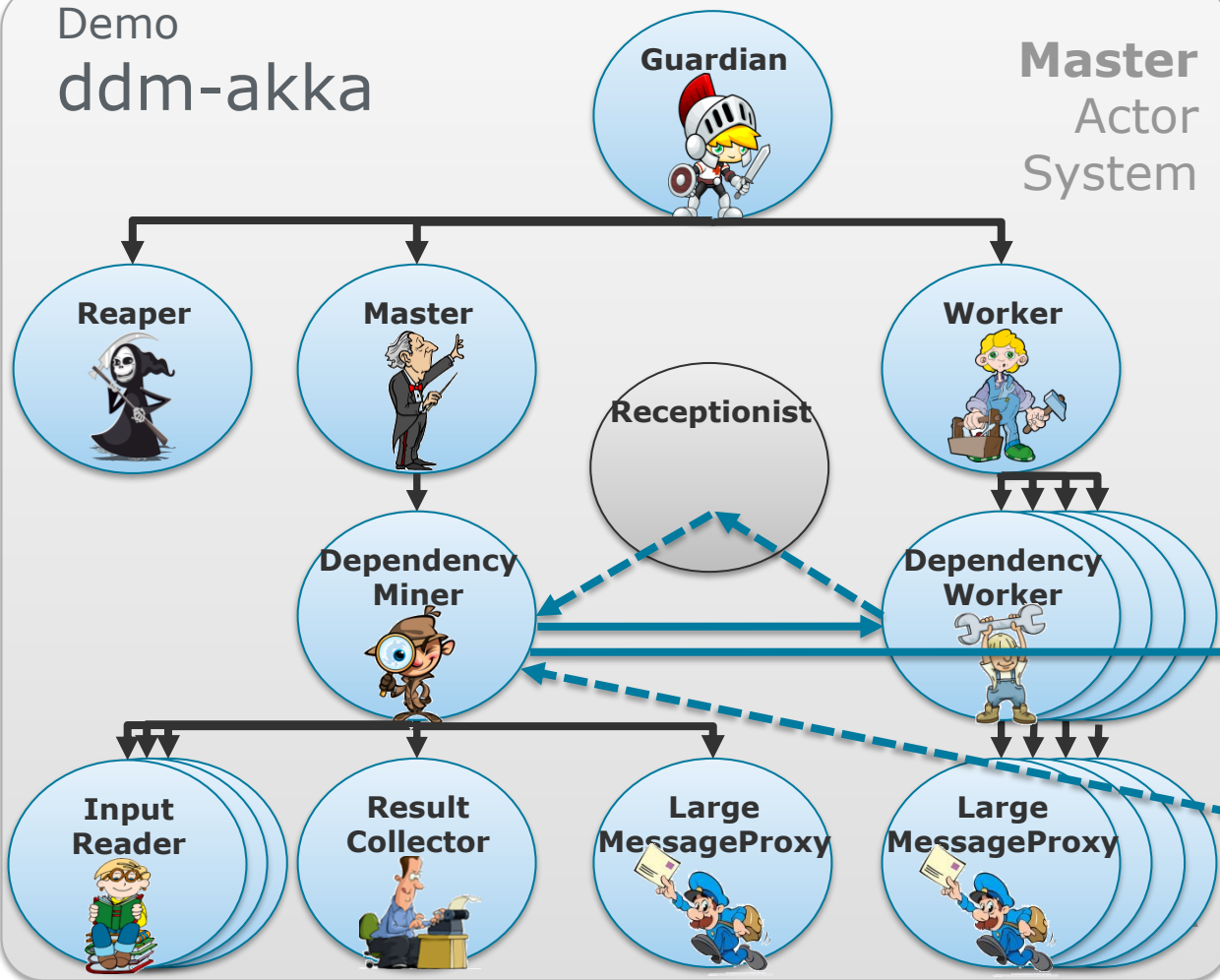
-  **thorsten-papenbrock** Thorsten Pape...
-  **SeegerM** Seeger

<https://github.com/UMR-Big-Data-Analytics/ddm-akka>

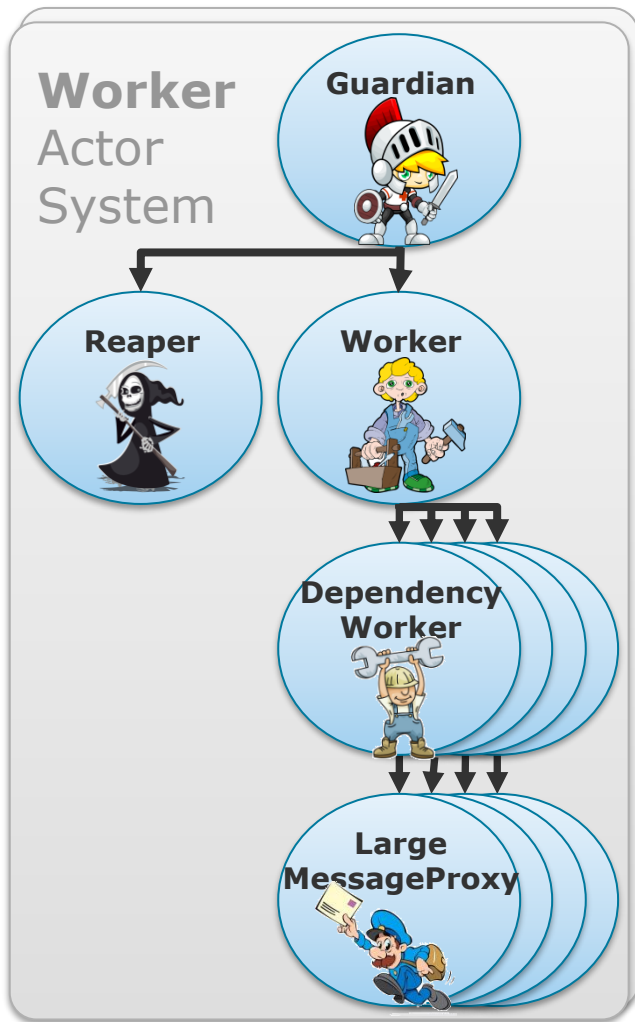
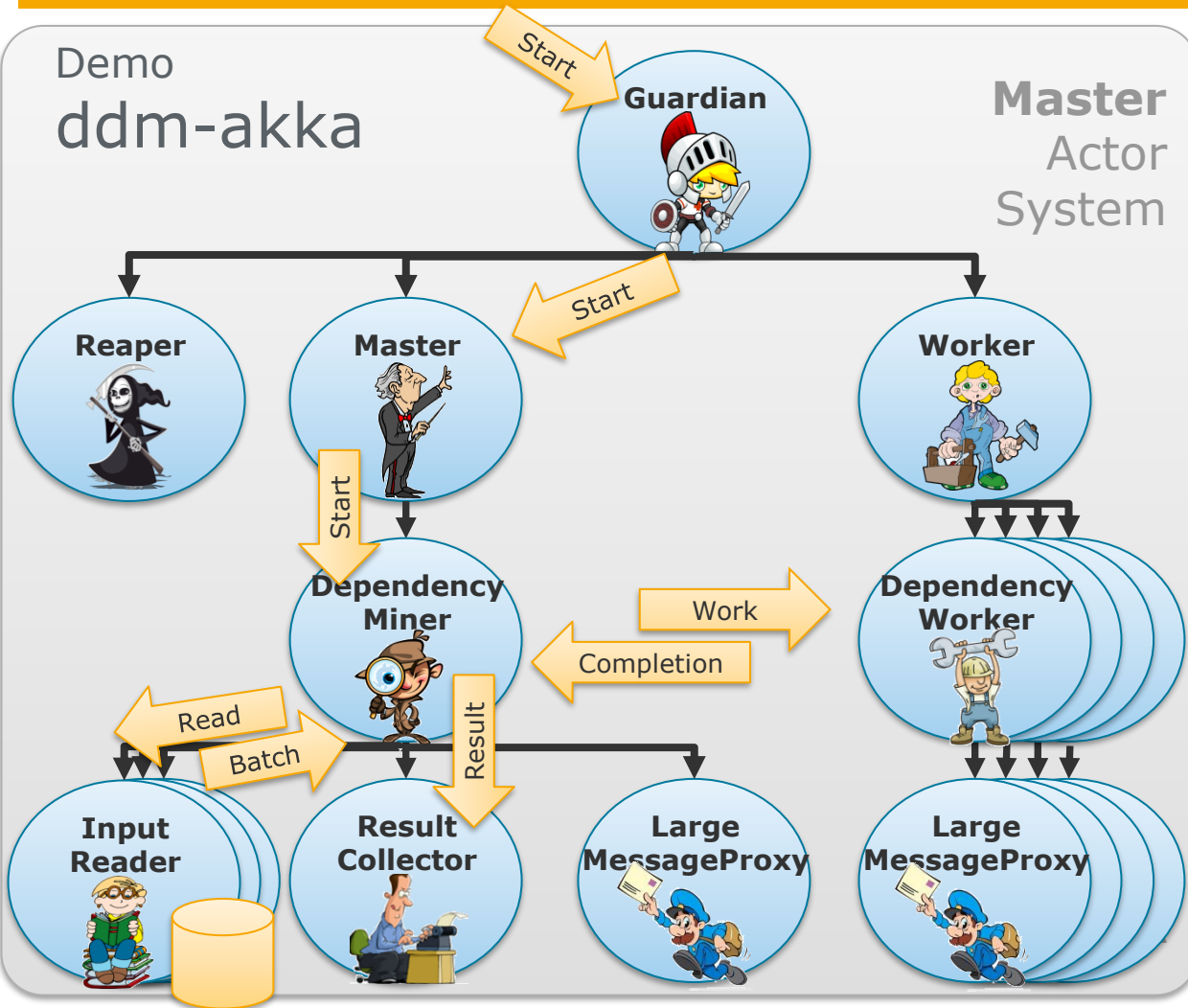
Demo ddm-akka



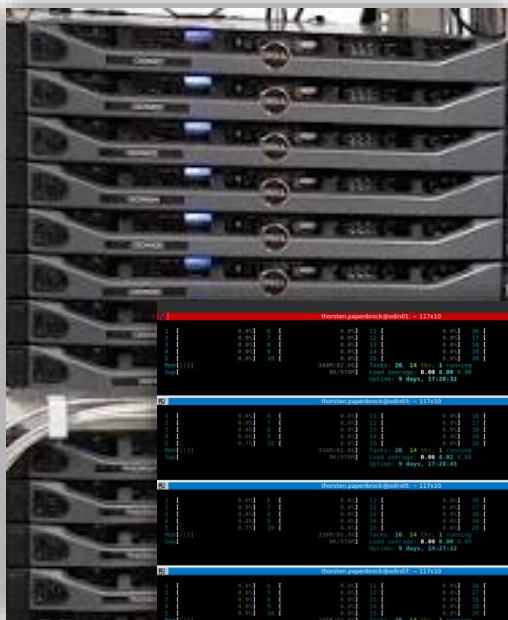
Demo ddm-akka



Demo ddm-akka



Demo ddm-akka

The screenshot shows the IntelliJ IDEA IDE with the 'ddm-akka' project open. The project structure on the left includes 'src/main/java/de/ddm' with sub-packages 'actors', 'patterns', 'profiling', 'Guardian', 'Master', 'Worker', 'configuration', 'serialization', 'AkkaSerializable', 'singletons', 'structures', and 'tutorial'. The 'Main' class is highlighted. The 'Main.java' file is open in the editor, showing the following code:

```
package de.ddm;

import akka.actor.typed.ActorSystem;
import de.ddm.actors.Guardian;
import de.ddm.configuration.Command;
import de.ddm.configuration.SystemConfiguration;
import de.ddm.singletons.SystemConfigurationSingleton;

import java.io.IOException;

public class Main {

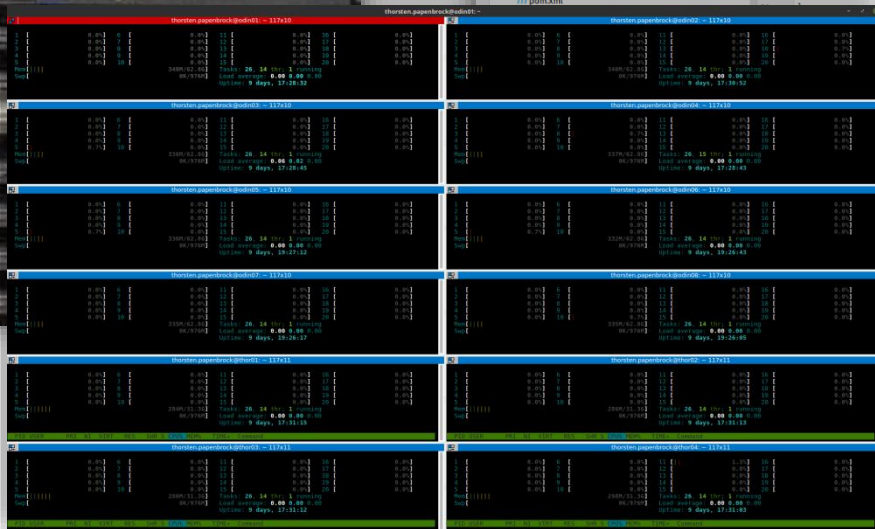
    public static void main(String[] args) {
        Command.applyYml(args);

        SystemConfiguration config = SystemConfigurationSingleton.get();

        final ActorSystem<Guardian.Message> guardian = ActorSystem.create(Guardian.create(), config.getActorSystemName(), config.toAkkaConfig());

        guardian.tell(new Guardian.StartMessage());

        if (config.getRole().equals(SystemConfiguration.MASTER_ROLE)) {
            try {
                System.out.println(">>> Press ENTER to exit <<<");
                System.in.read();
            } catch (IOException ignored) {}
            finally {
                guardian.tell(new Guardian.ShutdownMessage());
            }
        }
    }
}
```



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide 79

Demo

Best Practices

Not everything is an actor!

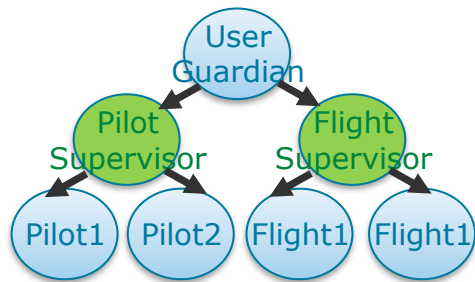
- Sending a message to an actor is more expensive than calling a method.
- Use actors where concurrency is expected.
- You can still use objects to encapsulate functionality and keep actor implementations clean.

Actors should do one thing!

- Although actors can dynamically change their behaviors and react to all kinds of messages, overly complex interfaces and unclear semantics make actors unmaintainable.
- Define a clear role and a semantically meaningful task for every actor.

Actor hierarchies rock!

- Flat hierarchies (e.g. putting everything below the user guardian) prohibit proper failure handling.
- Use supervisor actors for focused failure handling.



Demo

Best Practices

Keep messages organized!

- Polymorphism and inheritance allow us to create, share and define messages for actor behaviors all over the application code.
- Good practice: Define all messages for an actor inside the actor's own class.

Let actors decide about their instantiation!

- We create many actors, so concise instantiation code matters.
- Good practice: Define the creation of actor behaviors inside their own classes.

```
public static Behavior<Message> create(ActorRef<Worker.Message> worker) {  
    if (SystemUtils.hasGPU()) {  
        return Behaviors.setup(context -> new GPUWorker(context, worker, SystemUtils.getGPU()));  
    }  
    return Behaviors.setup(context -> new CPUWorker(context, worker, SystemUtils.getCPU()));  
}
```

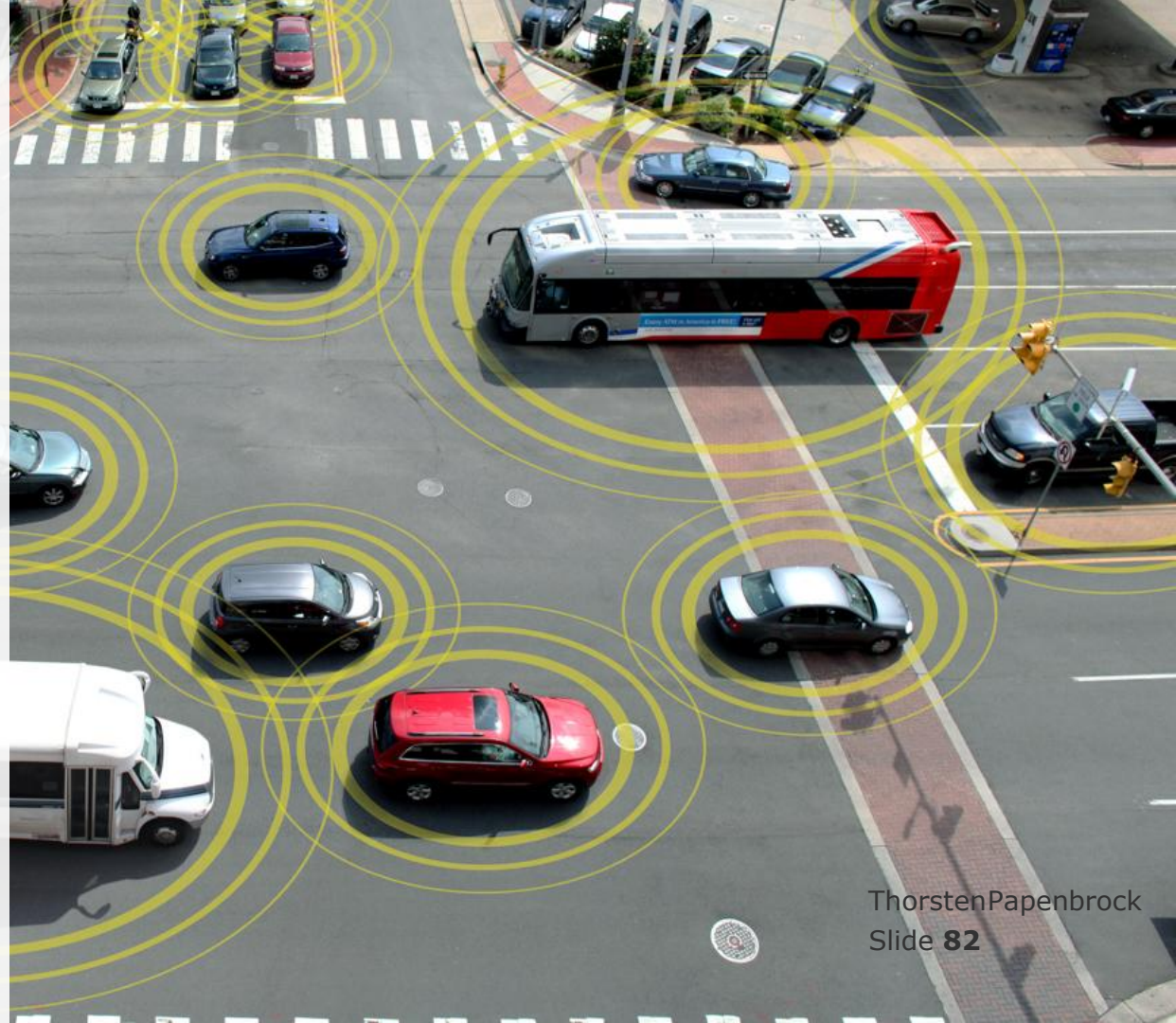
Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **81**

Akka Actor Programming Hands-on

- Actor Model
- Basic Concepts
- Runtime Architecture
- Demo
- **Messaging**
- Parallelization
- Remoting
- Clustering
- Patterns
- Homework



Messaging

Message Delivery Guarantees

Message delivery

- **at-most-once**: each message is delivered zero or one times.

- no guaranteed delivery; no message duplication
- highest performance; no implementation overhead
- fire-and-forget

You can implement
at-least-once and exactly-once,
with at-most-once!

- **at-least-once**: each message is delivered one or more times.

- guaranteed delivery; possibly message duplication
- ok-ish performance; state in sender
- send-and-acknowledge

With TCP, Akka basically
guarantees exactly-once,
but node failures can still
cause message loss!

- **exactly-once**: each message is delivered once.

- guaranteed delivery; no message duplication
- bad performance; state in sender and receiver
- send-and-acknowledge-and-deduplicate

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **83**

Messaging

Message Delivery Guarantees

Message ordering

- **no ordering**: all messages can be arbitrarily out of order
 - no guaranteed ordering
 - highest performance; no implementation overhead
- **sender-receiver ordering**: all messages between specific sender-receiver pairs are ordered (by send order)
 - ordered individual communications
 - good performance; message-oriented middleware simply sustains received order
- **total ordering**: all messages are ordered (by send timestamps)
 - serialized communication (see *total-order-broadcast* later in lecture)
 - bad performance; global ordering

Only with TCP!

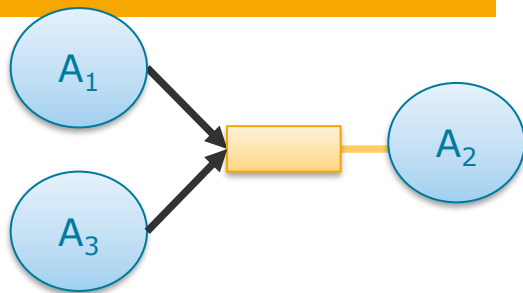
Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **84**

Messaging

Message Delivery Guarantees



Message ordering

- **sender-receiver ordering**: all messages between specific sender-receiver pairs are ordered (by send order)
- Example:
 - Actor A_1 sends messages M_1, M_2, M_3 to A_2
 - Actor A_3 sends messages M_4, M_5, M_6 to A_2

"If X is delivered..."
➤ No guaranteed delivery, i.e., messages may get lost and not arrive at A_2 !

- If M_1 is delivered it must be delivered before M_2 and M_3
- If M_2 is delivered it must be delivered before M_3
- If M_4 is delivered it must be delivered before M_5 and M_6
- If M_5 is delivered it must be delivered before M_6
- A_2 can see messages from A_1 interleaved with messages from A_3

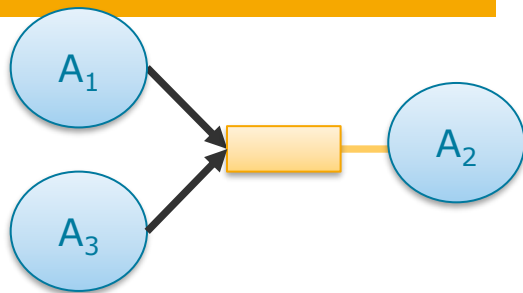
Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **85**

Messaging

Message Delivery Guarantees



Message ordering

- **sender-receiver ordering**: all messages between specific sender-receiver pairs are ordered (by send order)

- Send order is not transitive:

A_1 sends M_1 to A_2

A_1 sends M_2 to A_3

A_2 forwards M_1 to A_3

➤ A_3 may receive M_1 and M_2 in any order!

- Failure communication uses a different channel:

A_1 has child A_2

A_2 sends M_1 to A_1

A_2 fails causing failure message M_2 being send to A_1

➤ A_1 may receive M_1 and M_2 in any order!

Framework does not know that M_1 was forwarded, i.e., that M_1 is "younger" than M_2



In this way, A_1 might receive a message from a dead actor A_2

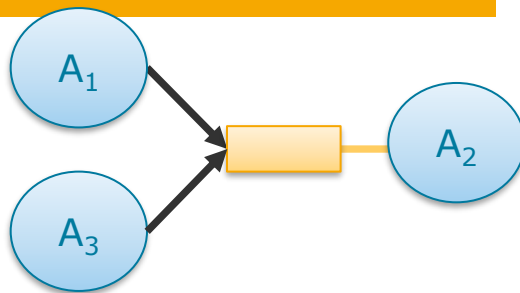
Although A_2 causes M_2 , it is technically not the sender of M_2

Akka Actor
Programming

ThorstenPapenbrock
Slide **86**

Messaging

Message Delivery Guarantees



Message ordering

- **sender-receiver ordering**: all messages between specific sender-receiver pairs are ordered (by send order)
- General notes:
 - The ordering guarantee holds only for TCP-based messaging.
 - The ordering guarantee can be violated by various factors, such as node failures.
 - If ordering is important, add and check custom sequence numbers!

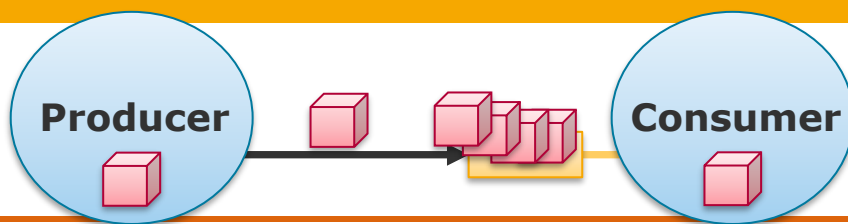
We later discuss how to implement reliable messaging, which achieves both **message ordering** and **exactly-once message delivery**.

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **87**

Messaging Pull vs. Push



Work Propagation

- **Producer** actors generate work for other **consumer** actors.
- **Push propagation**:
 - Producers send work packages to their consumers immediately (in particular, data is copied over the network proactively).
 - Work is queued in the inboxes of the consumers.
 - **Fast work propagation**; risk for message congestion/drops

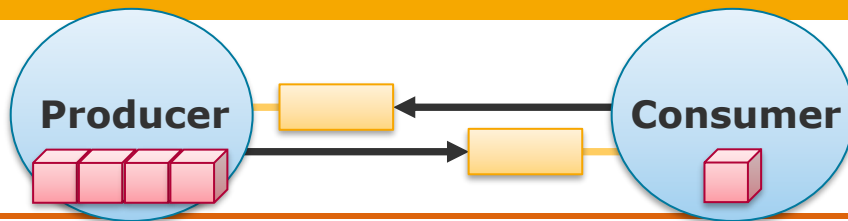
```
public Receive<Message> createReceive() {  
    return newReceiveBuilder()  
        .onMessage(RegistrationMessage.class, message -> {  
            for (Work work : this.work)  
                message.getSender().tell(new Worker.TaskMessage(work));  
            return this; })  
        .build();  
}
```

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **88**

Messaging Pull vs. Push



Work Propagation

- **Producer** actors generate work for other **consumer** actors.
- **Pull propagation**:
 - Consumers ask producers for more work if they are ready.
 - Work is queued in the producers' states.
 - **Slower work propagation**; **no risk for message congestion**

```
public Receive<Message> createReceive() {  
    return newReceiveBuilder()  
        .onMessage(RegistrationMessage.class, message -> {  
            message.getSender().tell(new Worker.TaskMessage(this.work.remove()));  
            return this; })  
        .onMessage(CompletionMessage.class, message -> {  
            message.getSender().tell(new Worker.TaskMessage(this.work.remove()));  
            return this; })  
        .build();  
}
```

Big Data Systems

Akka Actor
Programming

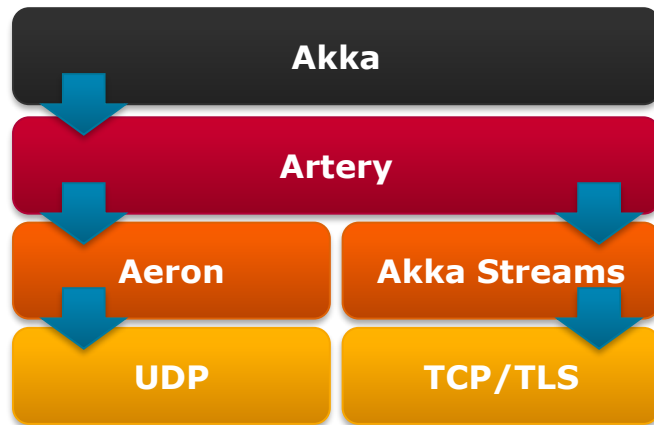
ThorstenPapenbrock
Slide **89**

Messaging

Akka's Messaging System

Artery

- High-performance, streaming-based messaging system
- Part of the Akka toolkit
- Uses compression on actor path strings (in ActorRefs) to reduce the general message overhead
- Based on Aeron for UDP channels and Akka Streams for TCP/TLS channels



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide 90

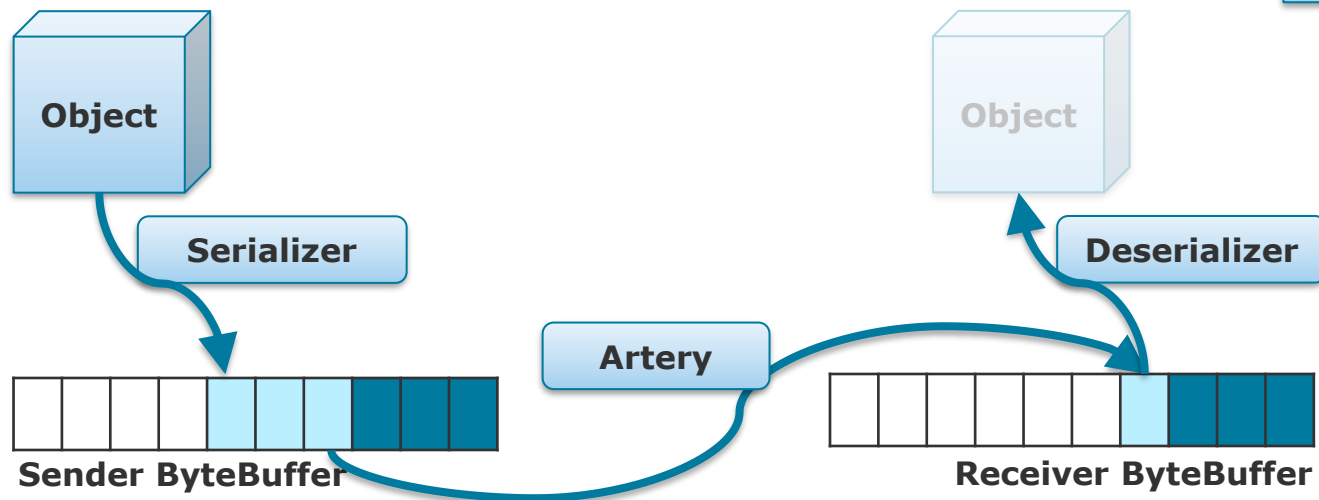
Messaging

Akka's Messaging System

Artery

- Focused on high-throughput, low-latency communication
- Mostly allocation-free operations (serialize data directly into buffers)
- Support for faster serialization/deserialization using ByteBuffers

Streaming!



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide 91

Messaging

Akka's Messaging System

Artery

- Focused on high-throughput, low-latency communication
- Mostly allocation-free operations (serialize data directly into buffers)
- Support for faster serialization/deserialization using ByteBuffers

Big Data Systems

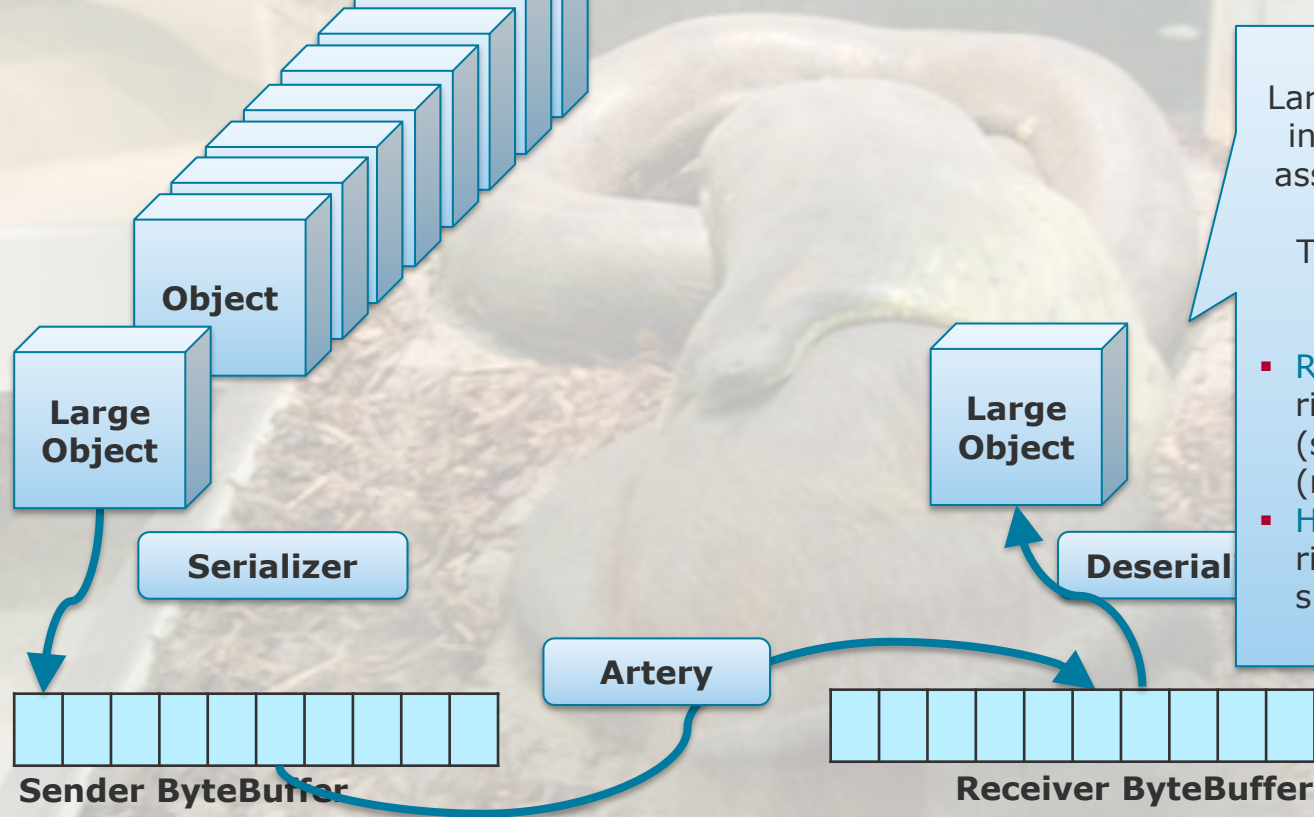
Akka Actor
Programming

- But: What if we need to send large amounts of data **over the network**?

ThorstenPapenbrock
Slide **92**



➤ But: What if we need to send large amounts of data **over the network**?



Large messages are broken down into frames that need to be re-assembled on the receiving side.

This blocks the TCP socket for other messages:

- **Regular messages:**
risk of message congestion (sender) and idle times (receiver)
- **Heartbeat messages:**
risk of cluster partitions and split-brain scenarios

➤ But: What if we need to send large amounts of data **over the network**?

- Use side channels for large data transfer!
 - Different channel that does not block main channel messages
 - Transfer protocol that is optimized for large files (WebSockets, UDP, FTP, ...)
 - Side channel examples:
 - Artery's Large Message Channel
 - Akka's http client-server module
 - Netty, FTP or other file transfer protocols
 - Database or shared file system
1. Send data via side channel to memory/disk of remote host.
 2. Send data reference in an Akka message when data is transferred.

Messaging

Artery's Large Message Channel

application.conf

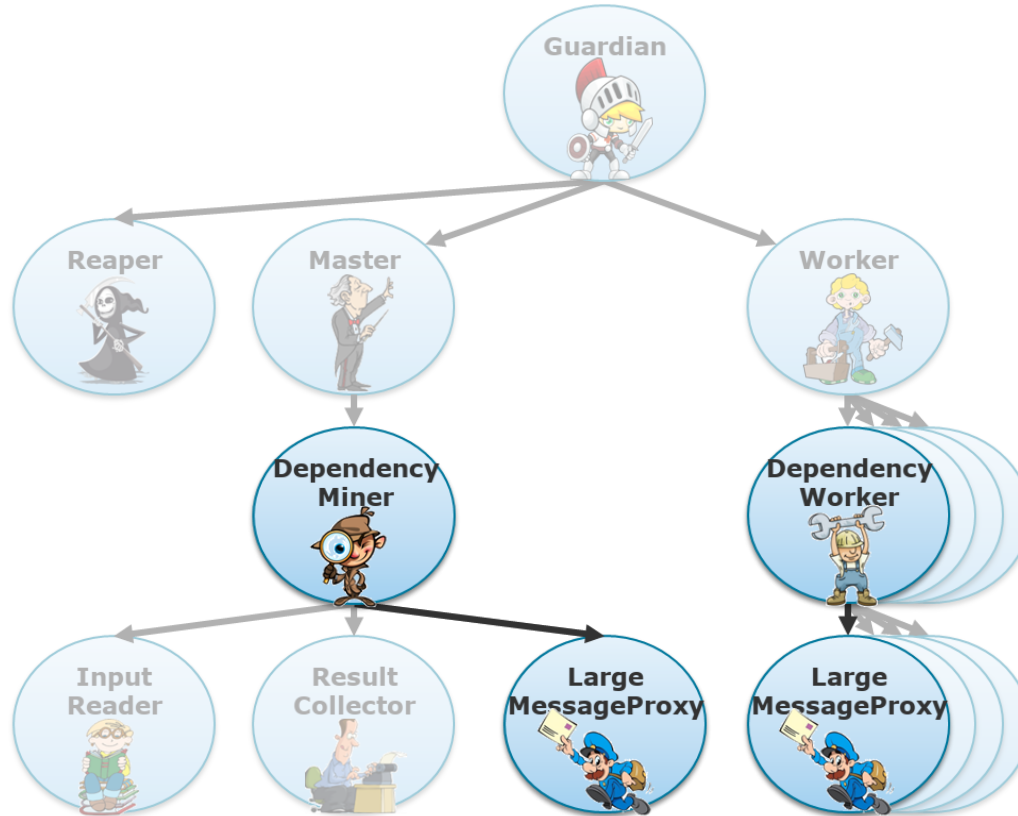
```
akka {  
  remote {  
    artery {  
      enabled = on  
      transport = tcp  
      canonical.hostname = "192.168.0.5"  
      canonical.port = 7787  
      large-message-destinations = [  
        "/user/*/largeMessageProxy",  
        "/user/**/largeMessageProxy"  
      ]  
      advanced {  
        maximum-large-frame-size = 2 MiB  
        large-buffer-pool-size = 32  
        outbound-large-message-queue-size = 256  
      }  
    }  
  }  
}
```

Use side channel for all messages from and to actors named „largeMessageProxy“

Big Data Systems

There is also a maximum message size for large message side channels. If large messages exceed this size, we need to break them down!

Messaging LargeMessageProxy

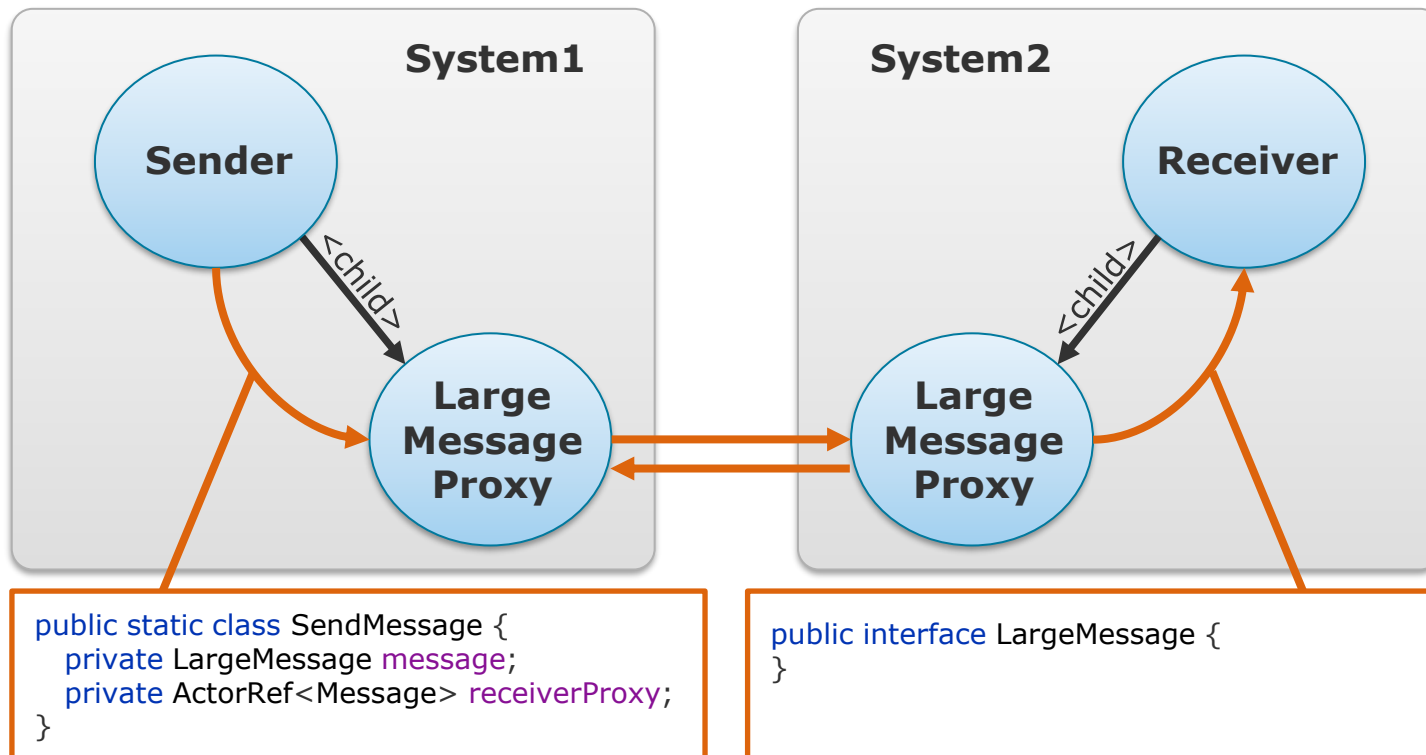


Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **97**

Messaging LargeMessageProxy



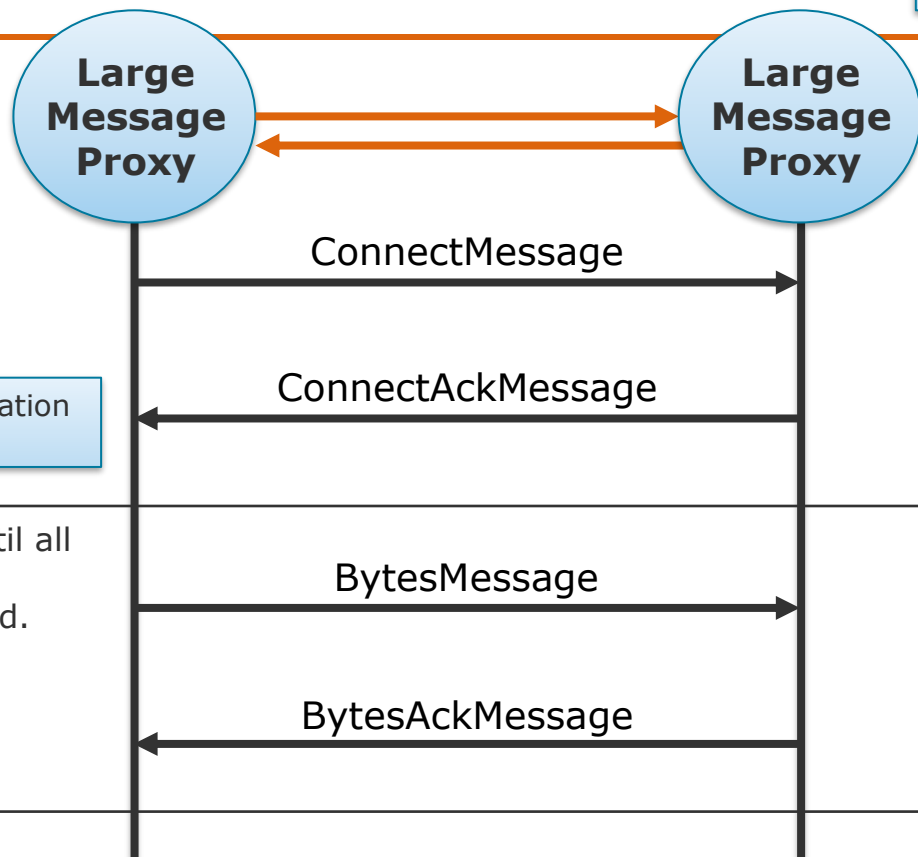
Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **98**

Messaging LargeMessageProxy

- TransmissionKeys for concurrent sends
- LargeMessageSize to find the end
- SerializerId and Manifest for deserialization



```
public static class ConnectMessage {
    private int senderTransmissionKey;
    private ActorRef<Message> senderProxy;
    private int largeMessageSize;
    private int serializerId;
    private String manifest;
}
```

```
public static class ConnectAckMessage {
    private int senderTransmissionKey;
    private int receiverTransmissionKey;
}
```

```
public static class BytesMessage {
    private byte[] bytes;
    private int senderTransmissionKey;
    private int receiverTransmissionKey;
}
```

```
public static class BytesAckMessage {
    private int senderTransmissionKey;
    private int receiverTransmissionKey;
}
```

```
private final Serialization serialization =
    SerializationExtension.get(this.getContext().getSystem());

private Behavior<Message> handle(SendMessage message) {
    LargeMessage largeMessage = message.getMessage();

    byte[] bytes = this.serialization.serialize(largeMessage).get();
    int serializerId = this.serialization.findSerializerFor(largeMessage).identifier();
    String manifest = Serializers.manifestFor(
        this.serialization.findSerializerFor(largeMessage), largeMessage);

    ...
}

private Behavior<Message> handle(BytesMessage message) {
    ...

    LargeMessage largeMessage = (LargeMessage)
        this.serialization.deserialize(bytes, serializerId, manifest).get();

    this.parent.tell(largeMessage);
    return this;
}
```

```
public static class ConnectMessage {
    private int senderTransmissionKey;
    private ActorRef<Message> senderProxy;
    private int largeMessageSize;
    private int serializerId;
    private String manifest;
}
```

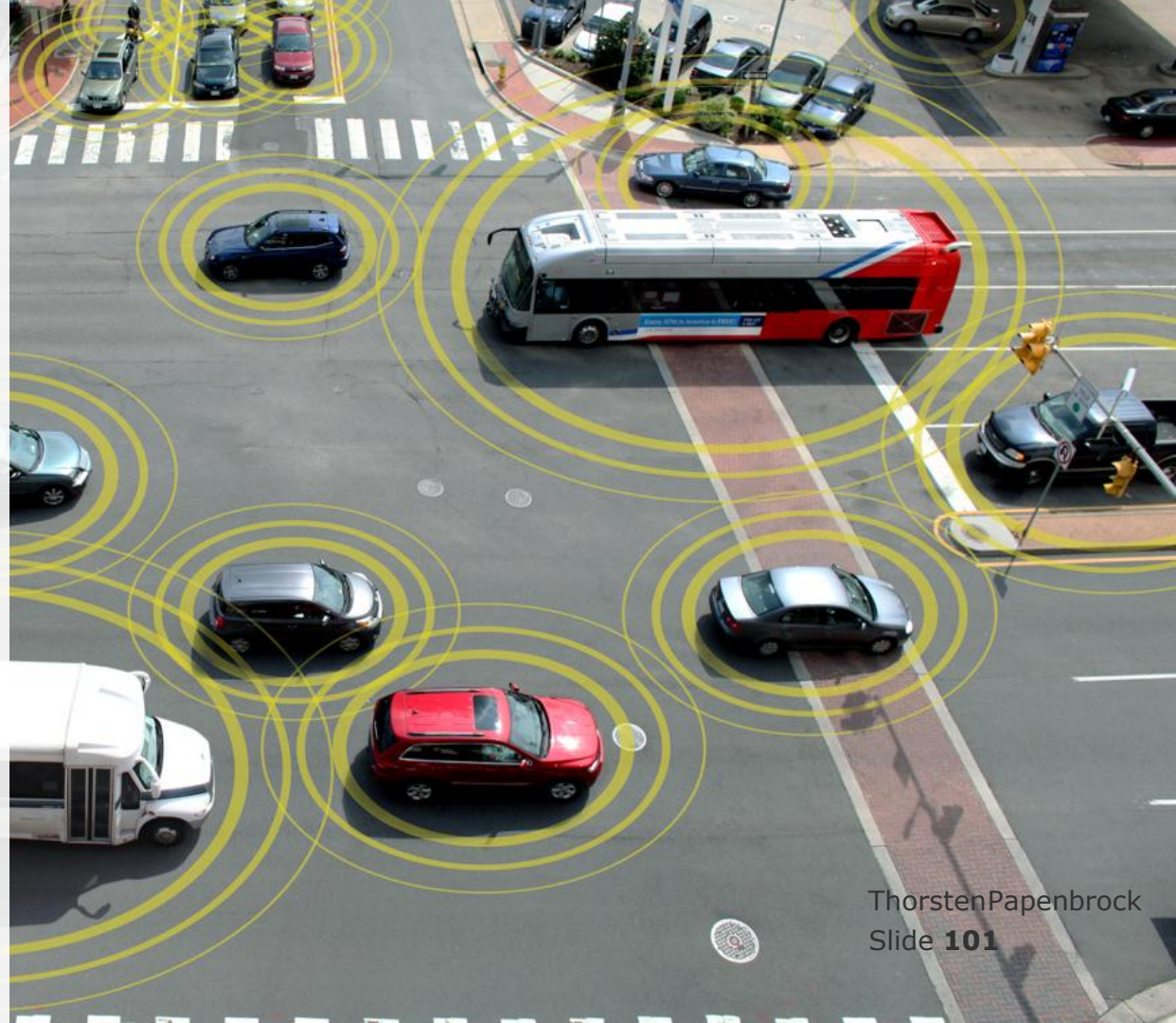
```
public static class ConnectAckMessage {
    private int senderTransmissionKey;
    private int receiverTransmissionKey;
}
```

```
public static class BytesMessage {
    private byte[] bytes;
    private int senderTransmissionKey;
    private int receiverTransmissionKey;
}
```

```
public static class BytesAckMessage {
    private int senderTransmissionKey;
    private int receiverTransmissionKey;
}
```

Akka Actor Programming Hands-on

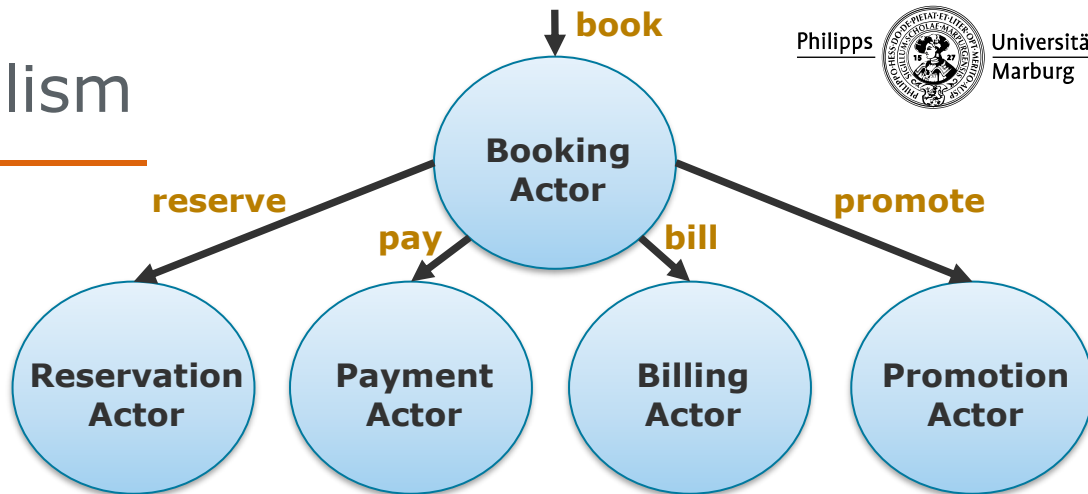
- Actor Model
- Basic Concepts
- Runtime Architecture
- Demo
- Messaging
- **Parallelization**
- Remoting
- Clustering
- Patterns
- Homework



Task- and Data-Parallelism

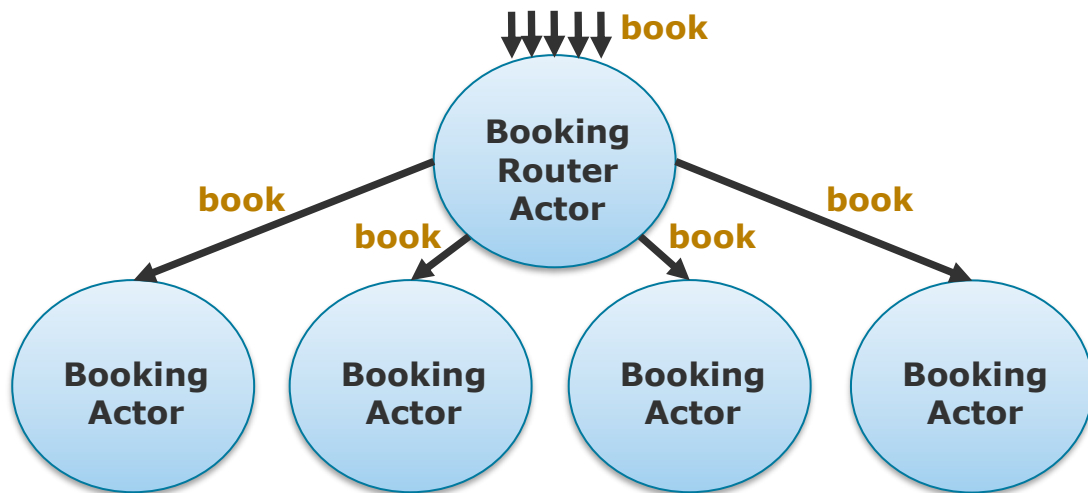
Task-parallelism

- Distribute sub-tasks to different actors



Data-parallelism

- Distribute chunks of data to different actors



Parallelization Router

Push-propagation
;-)

- Actors often delegate work if they are responsible for ...
 - many or compute-intensive tasks (with many subtasks);
 - data-intensive tasks (with independent partitions).
- Work can be delegated to a dynamically managed pool of worker actors:

Task-parallelism

Data-parallelism

```
int poolSize = 32;
```

```
PoolRouter<Worker.Message> pool =  
    Routers.pool(  
        poolSize,  
        Behaviors.supervise(  
            Worker.create()).onFailure(SupervisorStrategy.restart())  
        ).withRoundRobinRouting();
```

```
ActorRef<Worker.Message> router = context.spawn(pool, "worker-pool");
```

```
for (int i = 0; i < 10000; i++)  
    router.tell(new Worker.WorkMessage(42));
```

Router automatically starts this many local routees as children.

Routees should be supervised by their router.

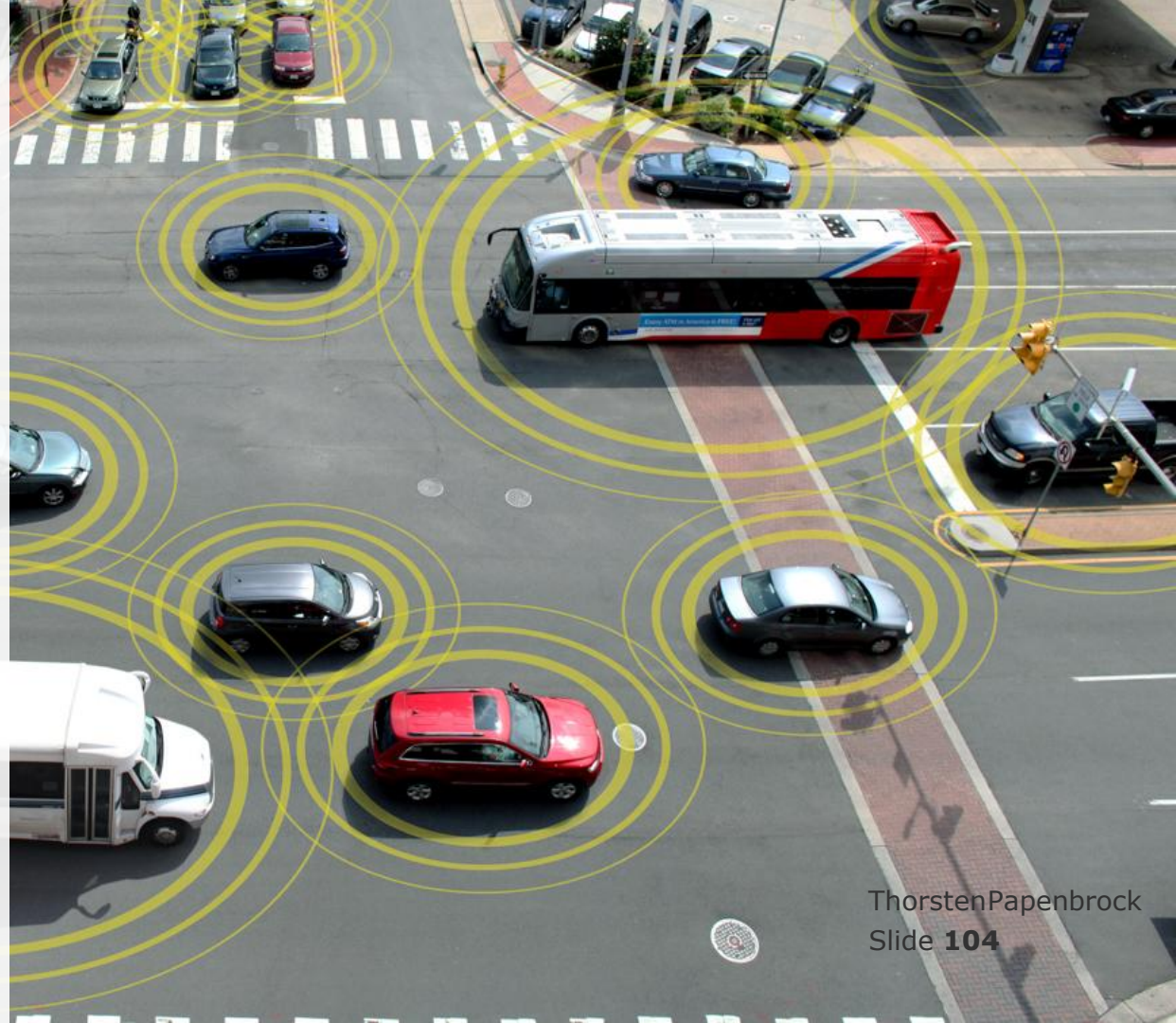
.withConsistentHashingRouting()
.withRoundRobinRouting()
.withBroadcastPredicate()

Routers are actors with the same interface as their routees.

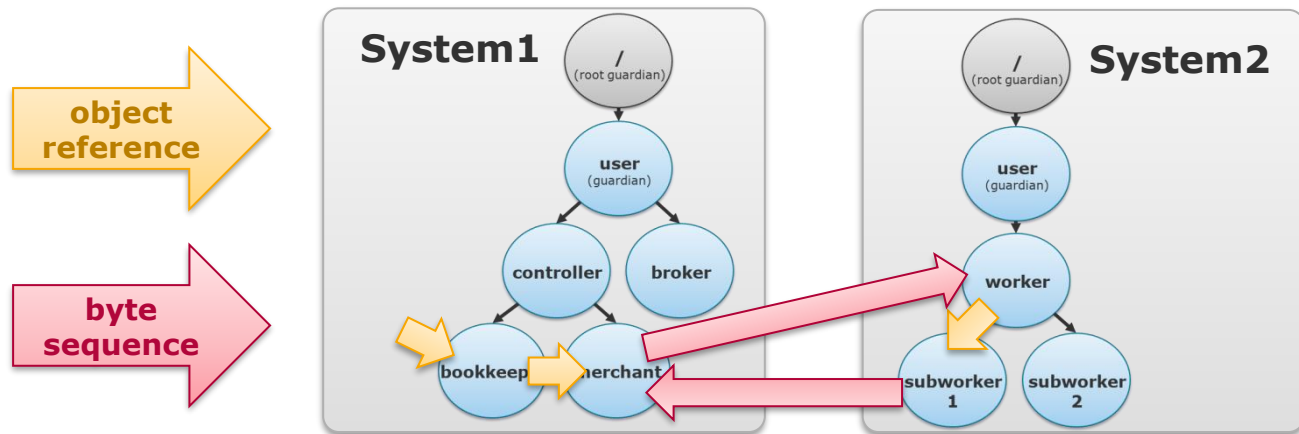
Routers forward messages based on their RoutingStrategy.

Akka Actor Programming Hands-on

- Actor Model
- Basic Concepts
- Runtime Architecture
- Demo
- Messaging
- Parallelization
- **Remoting**
- Clustering
- Patterns
- Homework



- Only messages to **remote actors** are serialized!
 - Communication within one system: **language-specific data types**
 - Pointers and primitive values
 - Communication via process boundaries: **transparent serialization**
 - Java Serializable, JacksonJson, JacksonCbor, Protocol Buffers, ... (configurable)



Remoting

Custom Serialization

application.conf

```
akka {  
  actor {  
    provider = cluster  
    serializers {  
      java = "akka.serialization.JavaSerializer"  
      jackson-json = "akka.serialization.jackson.JacksonJsonSerializer"  
      jackson-cbor = "akka.serialization.jackson.JacksonCborSerializer"  
      kryo = "com.twitter.chill.akka.ConfiguredAkkaSerializer"  
      proto = "akka.remote.serialization.ProtoBufSerializer"  
      my-serializer = "de.ddm.myalgo.serialization.MyMessageSerializer"  
    }  
    serialization-bindings {  
      "de.ddm.serialization.AkkaSerializable" = jackson-cbor  
      "de.ddm.serialization.MyActor.MyMessage" = my-serializer  
    }  
  }  
}
```

Known object serializers

JacksonCborSerializer for all messages that implement the **AkkaSerializable** interface

MyMessageSerializer for all **MyMessage** messages
(more specific bindings bind stronger than general bindings)

Remoting

Custom Serialization - Notes

```
@JsonAutoDetect(fieldVisibility = JsonAutoDetect.Visibility.ANY)
public interface AkkaSerializable extends Serializable {
}
```

```
akka {
  actor {
    provider = cluster
    serializers {
      java = "akka.serialization.JavaSerializer"
      jackson-json = "akka.serialization.jackson.JacksonJsonSerializer"
      jackson-cbor = "akka.serialization.jackson.JacksonCborSerializer"
      kryo = "com.twitter.chill.akka.ConfiguredAkkaSerializer"
      proto = "akka.remote.serialization.ProtobufSerializer"
      my-serializer = "de.ddm.myalgo.serialization.MySerializer"
    }
    serialization-bindings {
      "de.ddm.serialization.AkkaSerializable" = jackson-json
      "de.ddm.serialization.MyActor.MyMessage" = my-serializer
    }
  }
}
```

JavaSerializer

- Does not require no-args constructor
- Serialized messages are large
- Meanwhile categorized as an anti-pattern

JacksonCborSerializer

- Decently efficient and effective
- Can deal with typed ActorRefs
- Requires a specific JsonAnnotation
- Requires a no-args constructor for deserialization
- De-facto standard for custom serialization

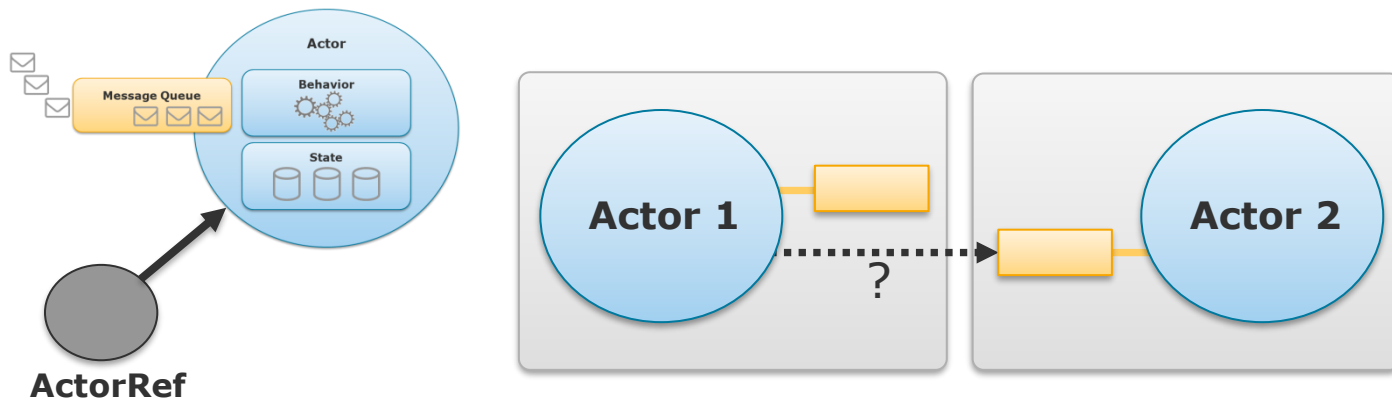
ProtobufSerializer

- Very efficient and effective + schema evolution
- Can deal with typed ActorRefs
- Requires code generation from schema files
- Standard for Akka internal messages

Kryo (any wrapper)

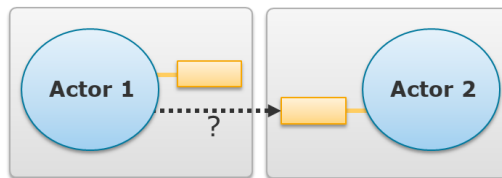
- Very efficient and effective
- Requires a no-args constructor for deserialization
- Cannot deal with typed ActorRefs
- Viable option for specific messages

Remoting Actor Lookup



- An **ActorRef** points to some actor on some node via its address:
"`<protocol>://<ActorSystemName>@<host>:<port>/path/to/actor`"
 - Example: "`akka.tcp://ddm-akka@192.168.0.1:7877/user/worker`"
 - ActorRefs work within and across ActorSystem boundaries!
- But how do we obtain an ActorRef to a remote actor?

Remoting Actor Lookup



1. By **application**:

- If the remote actor is already known to the local ActorSystem, ask for the ActorRef in the constructor or provide a setter.

2. By **message**:

- If the remote actor is already known to some other (remote) actor, ask that actor for the ActorRef.

3. By **actor lookup**:

- If the application is written in Akka Classic (or you mix the interfaces), lookup the ActorRef directly.

```
Address address = new Address("akka.tcp", "MySystem", "localhost", 7877);
ActorSelection selection = this.context().system()
    .actorSelection(String.format("%s/user/%s", address, "worker"));
selection.tell(new HelloMessage(), this.self());
```

4. By **receptionist**:

- I. Register the actor at the local receptionist.
- II. Receptionists will gossip their registered actors in cluster mode!
- III. Ask the local receptionist for the ActorRef.

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **109**

Remoting Actor Lookup

1. By application:

- If the remote actor asks for the ActorRef

2. By message:

- If the remote actor asks that actor for

3. By actor lookup:

- If the application lookup the ActorRef

4. By receptionist:

- Register the actor at the local receptionist.
- Receptionists will gossip their registered actors in cluster mode!
- Ask the local receptionist for the ActorRef.

```
public static final ServiceKey<Master.Message> masterService =  
    ServiceKey.create(Master.Message.class, "masterService");
```

```
private Master(ActorContext<Message> context) {  
    context.getSystem().receptionist().tell(  
        Receptionist.register(masterService, context.getSelf()));  
}
```

```
@Getter @NoArgsConstructor @AllArgsConstructor
```

```
public static class ReceptionistListingMessage implements Message {  
    Receptionist.Listing listing;  
}
```

```
private Worker(ActorContext<Message> context) {  
    final ActorRef<Receptionist.Listing> listingResponseAdapter =  
        context.messageAdapter(Receptionist.Listing.class, ReceptionistListingMessage::new);  
    context.getSystem().receptionist().tell(  
        Receptionist.subscribe(Master.masterService, listingResponseAdapter));  
}
```

```
@Override
```

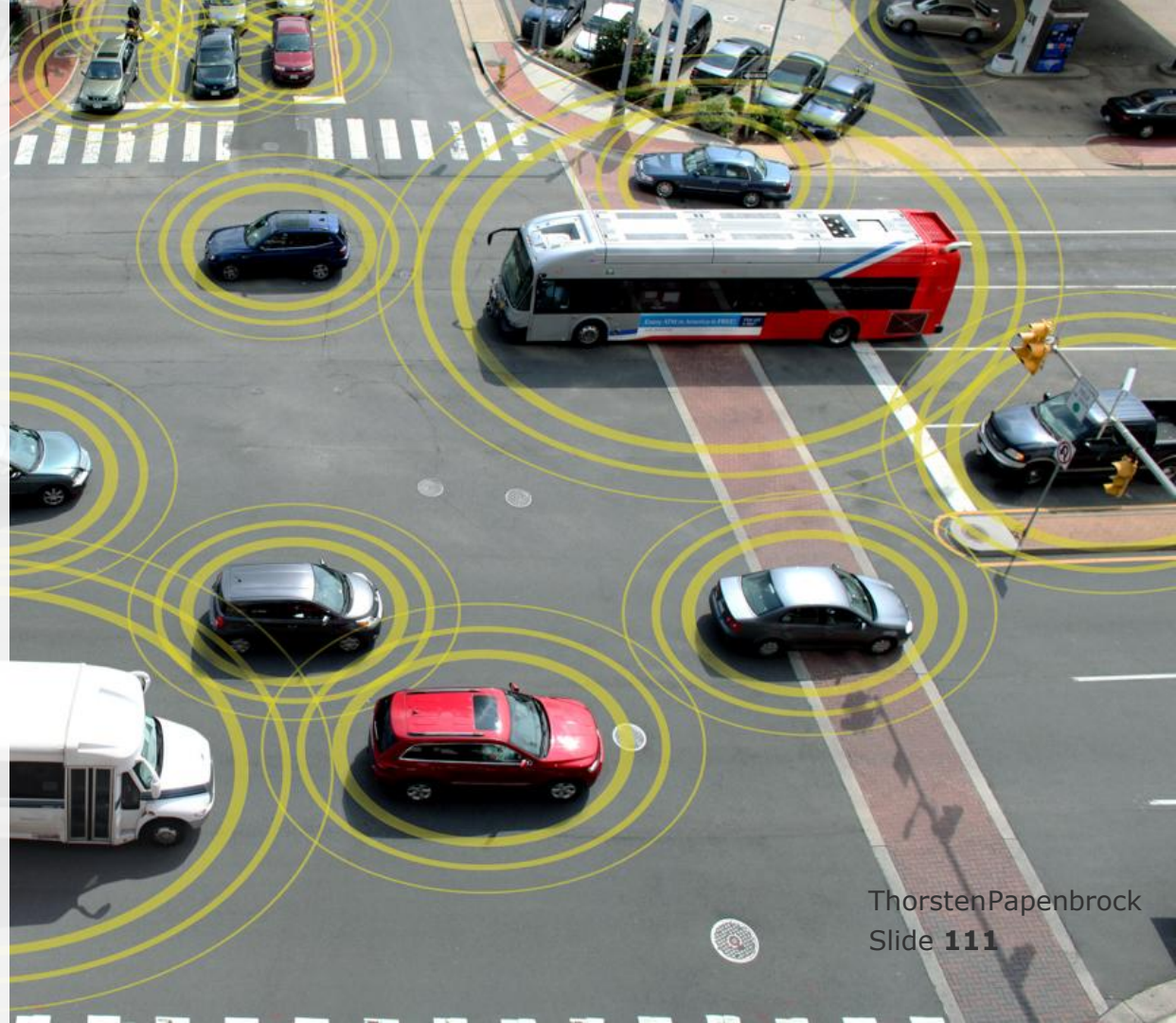
```
public Receive<Message> createReceive() {  
    return newReceiveBuilder().onMessage(ReceptionistListingMessage.class, this::handle).build();  
}
```

```
private Behavior<Message> handle(ReceptionistListingMessage message) {  
    Set<ActorRef<Master.Message>> masters =  
        message.getListing().getServiceInstances(Master.masterService);  
    return this;  
}
```

Recap

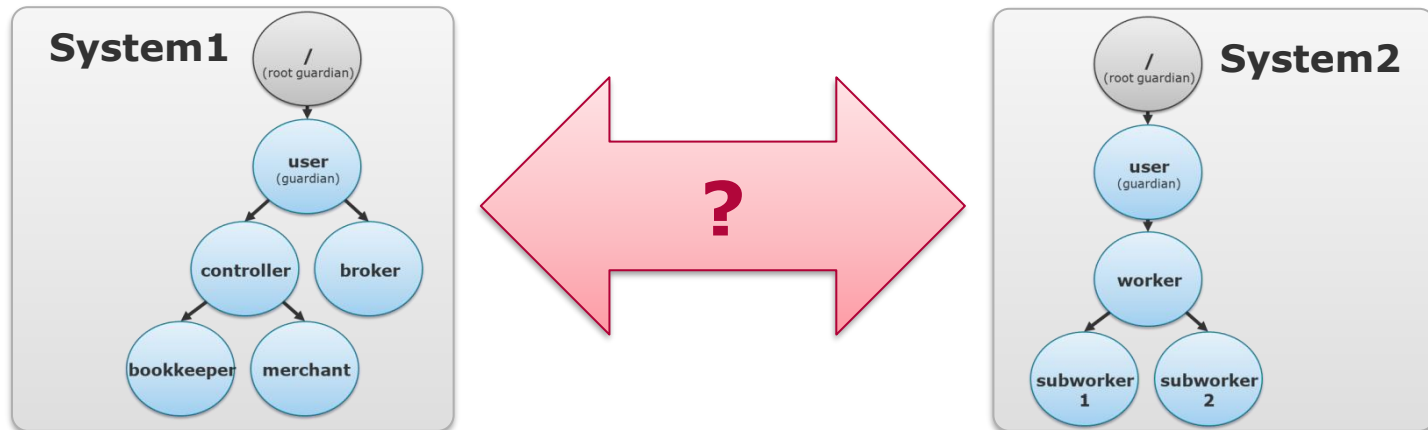
Akka Actor Programming Hands-on

- Actor Model
- Basic Concepts
- Runtime Architecture
- Demo
- Messaging
- Parallelization
- Remoting
- **Clustering**
- Patterns
- Homework



Clustering

Cluster-Awareness



How does System1 know ...

- which other ActorSystems are available?
(the number might also change at runtime!)
- what failures occurred in other ActorSystems?
(single actors but also entire nodes might become unavailable!)
- what roles other ActorSystems take?
(e.g. a master or worker or metrics collector or entirely different application!)

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **112**

Clustering

Cluster-Awareness

Akka Actors

Core Actor
model classes
for concurrency
and distribution

Akka Cluster

Classes for the
resilient and
elastic
distribution over
multiple nodes

Akka Streams

Asynchronous,
non-blocking,
backpressured,
reactive stream
classes

Akka Http

Asynchronous,
streaming-first
HTTP server and
client classes

Cluster Sharding

Classes to
decouple actors
from their
locations
referencing
them by identity

Akka Persistence

Classes to
persist actor
state for fault
tolerance and
state restore
after restarts

Distributed Data

Classes for an
eventually
consistent,
distributed,
replicated key-
value store

Alpakka

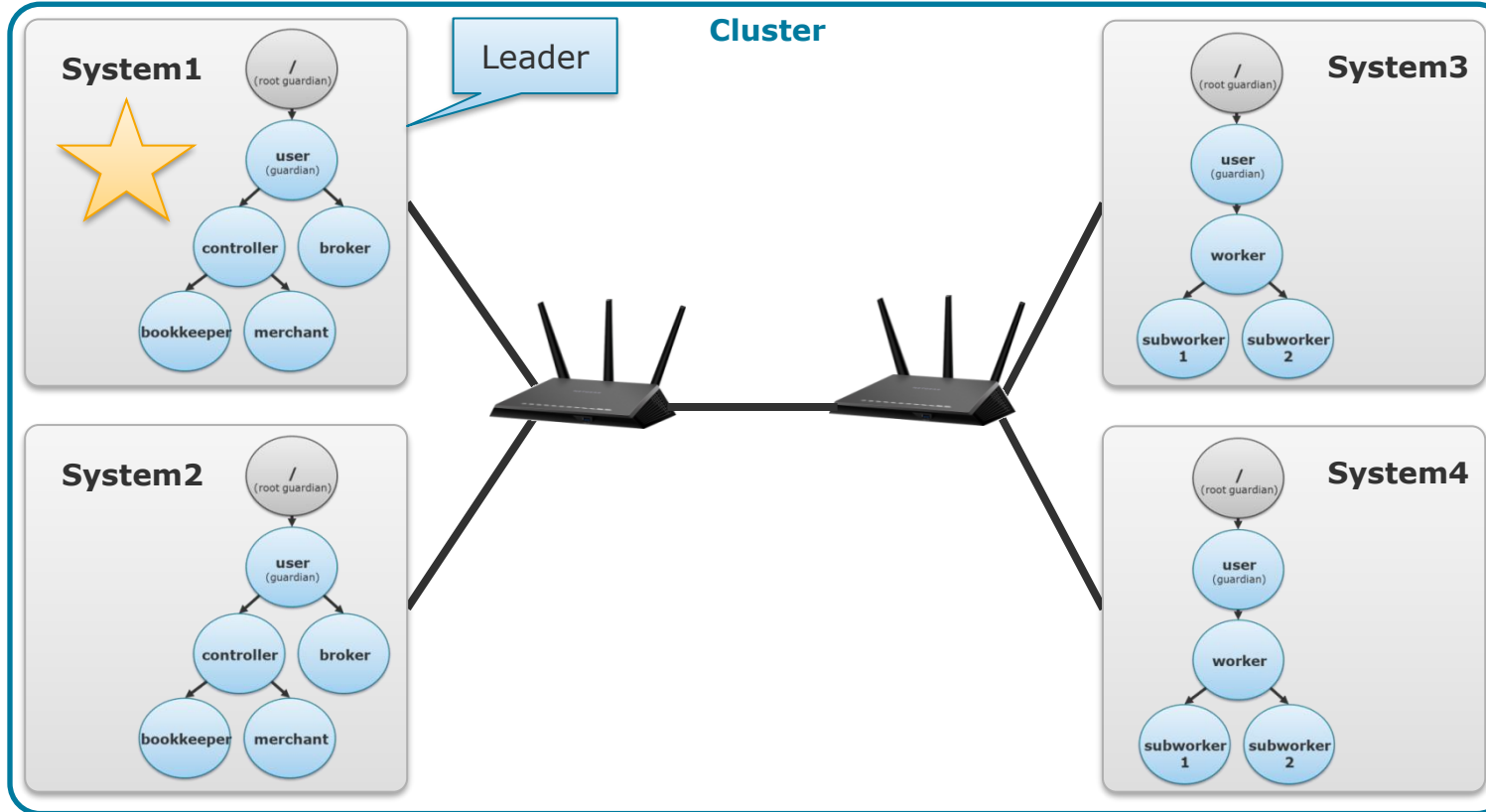
Stream
connector
classes to other
technologies

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **113**

Clustering Cluster



Big Data Systems

Akka Actor
Programming

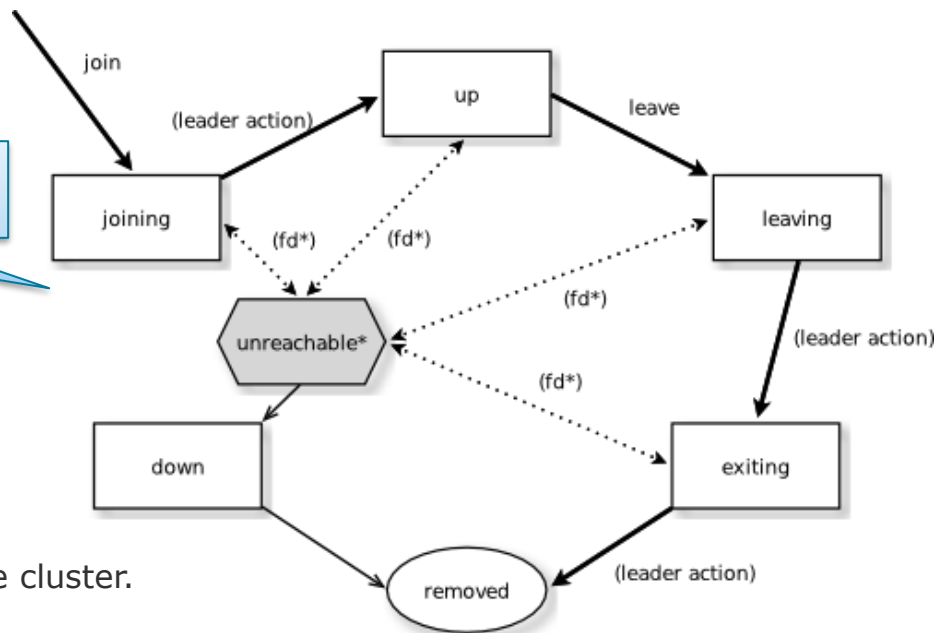
ThorstenPapenbrock
Slide **114**

Clustering

Cluster

fd* = failure detector
(see ϕ accrual failure detector later)

- A distributed membership service.
- Runs in all Akka cluster-managed ActorSystems.
- Stores all membership information in a distributed, fully replicated key-value store (= Riak).
- Uses gossiping¹ to propagate cluster changes.
- Elects the eldest node in a cluster as "leader" of the cluster.
- Leader decisions²:
 - state changes to "up", "exiting", and "removed" for members of the cluster
- Member decisions²:
 - state changes to "joining", "leaving", and (via local failure detector) "unreachable"



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **115**

[1] <https://doc.akka.io/docs/akka/2.5/common/cluster.html#gossip>

[2] <https://doc.akka.io/docs/akka/2.5/common/cluster.html#membership-lifecycle>

Clustering Configuration

application.conf

```
akka {  
  actor {  
    provider = cluster  
    [...]  
  }  
  remote {  
    artery {  
      canonical.hostname = "127.0.0.1"  
      canonical.port = 2551  
      [...]  
    }  
  }  
  cluster {  
    min-nr-of-members = 1  
    downing-provider-class = "akka.cluster.sbr.SplitBrainResolverProvider"  
  }  
  coordinated-shutdown {  
    run-by-actor-system-terminate = on  
    terminate-actor-system = on  
  }  
  [...]  
}
```

instead of `local` or `remote`

The cluster membership waits for this many members before it sets their status to up.

If cluster nodes are abruptly terminated, the downing-provider resolves the nodes' cluster state changes.

Ties the coordinated shutdown to the termination of the ActorSystem.

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **116**

Clustering Configuration

application.conf

```
akka {  
  cluster {  
    gossip-interval = 1s  
    gossip-time-to-live = 2s  
    leader-actions-interval = 1s  
  
    failure-detector {  
      implementation-class = "akka.remote.PhiAccrualFailureDetector"  
      heartbeat-interval = 2s  
      threshold = 12.0  
      max-sample-size = 1000  
      min-std-deviation = 100ms  
      acceptable-heartbeat-pause = 10s  
      monitored-by-nr-of-members = 5  
      expected-response-after = 2s  
    }  
  }  
}
```

Gossip configuration.

Failure detector configuration.

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **117**

Clustering Startup (Recap)

```
public static void main(String[] args) {  
    Config conf = ConfigFactory.parseString("" +  
        "akka.remote.artery.canonical.hostname = \"" + this.getHost(args) + "\"\n" +  
        "akka.remote.artery.canonical.port = " + this.getPort(args) + "\n" +  
        "akka.cluster.roles = [" + this.getRole(args) + "]\n" +  
        "akka.cluster.seed-nodes = [\"akka://" + this.getName(args) + "@" +  
            this.getMasterHost(args) + ":" + this.getMasterPort(args) + "\"]")  
        .withFallback(ConfigFactory.load("application"));
```

Set all cluster-specific
configuration programmatically.

this host, port and role

contact to cluster master

```
final ActorSystem<Guardian.Message> guardian =  
    ActorSystem.create(Guardian.create(), "MyAkkaApp", conf);
```

```
guardian.tell(new Guardian.StartMessage());
```

```
try {  
    System.out.println(">>> Press ENTER to exit <<<");  
    System.in.read();  
} catch (IOException ignored) {  
} finally {  
    guardian.tell(new Guardian.ShutdownMessage());  
}  
}
```

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **118**

Clustering

Programmatic Features

// Member decisions

```
Cluster.get(this.getContext().getSystem()).join(new Address("..."));  
Cluster.get(this.getContext().getSystem()).down(new Address("..."));  
Cluster.get(this.getContext().getSystem()).leave(new Address("..."));
```

// Module access

```
Cluster.get(this.getContext().getSystem()).failureDetector();  
Cluster.get(this.getContext().getSystem()).scheduler();  
Cluster.get(this.getContext().getSystem()).downingProvider();
```

// Cluster event bus

```
Cluster.get(this.getContext().getSystem()).subscribe(<SelfActorRef>, <SomeEvent>);  
Cluster.get(this.getContext().getSystem()).unsubscribe(< SelfActorRef>, <SomeEvent>);
```

// Callback registration

```
Cluster.get(this.getContext().getSystem()).registerOnMemberUp(() -> {  
    // Some action  
});  
Cluster.get(this.getContext().getSystem()).registerOnMemberRemoved(() -> {  
    // Some action  
});
```

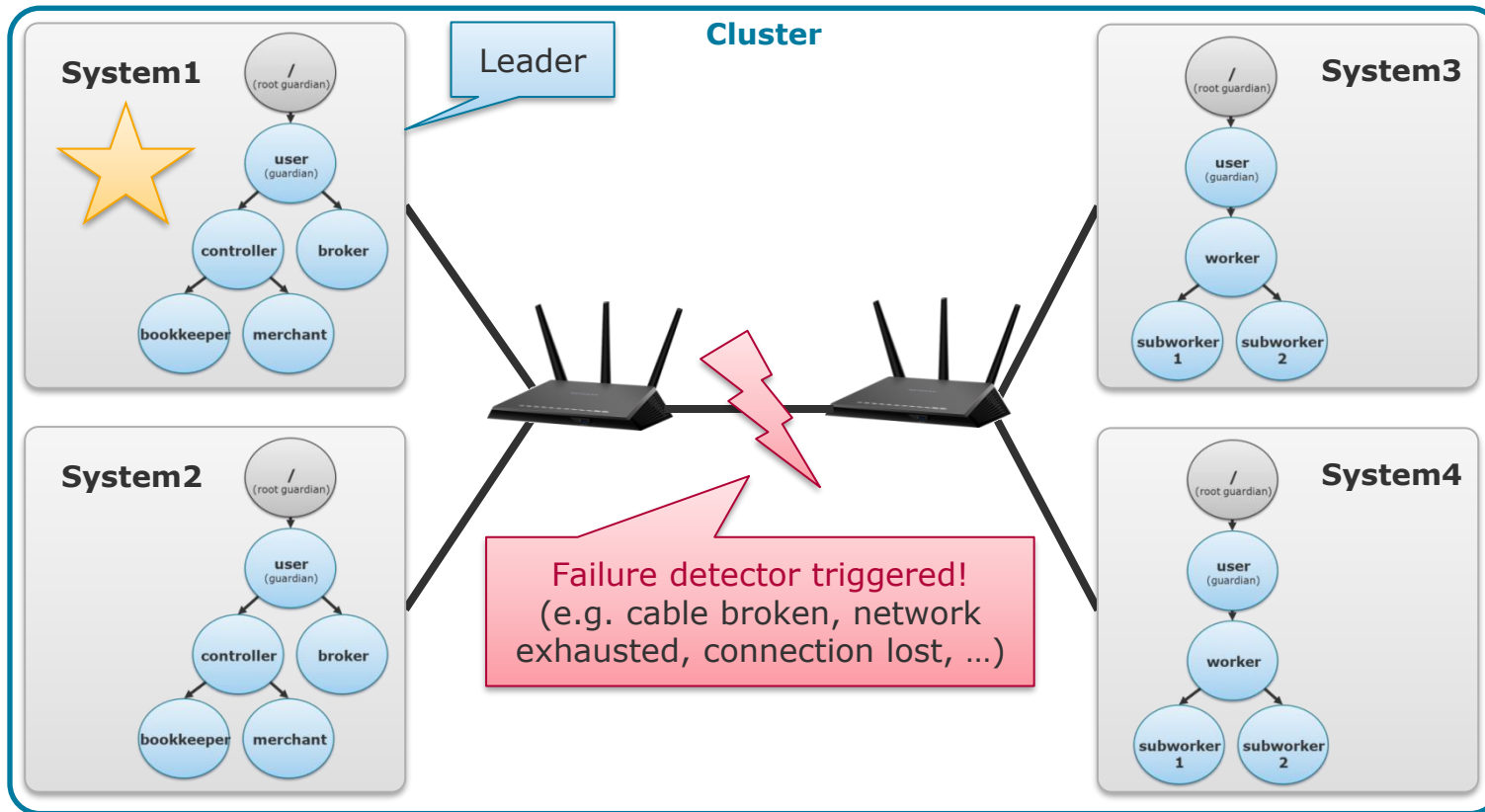
MemberUp.class,
MemberJoined.class,
MemberRemoved.class,
...

For example, find some
role in the cluster and
connect to it.

For example, initiate a
shutdown if this cluster
has some worker role.

Clustering

Cluster Partitioning



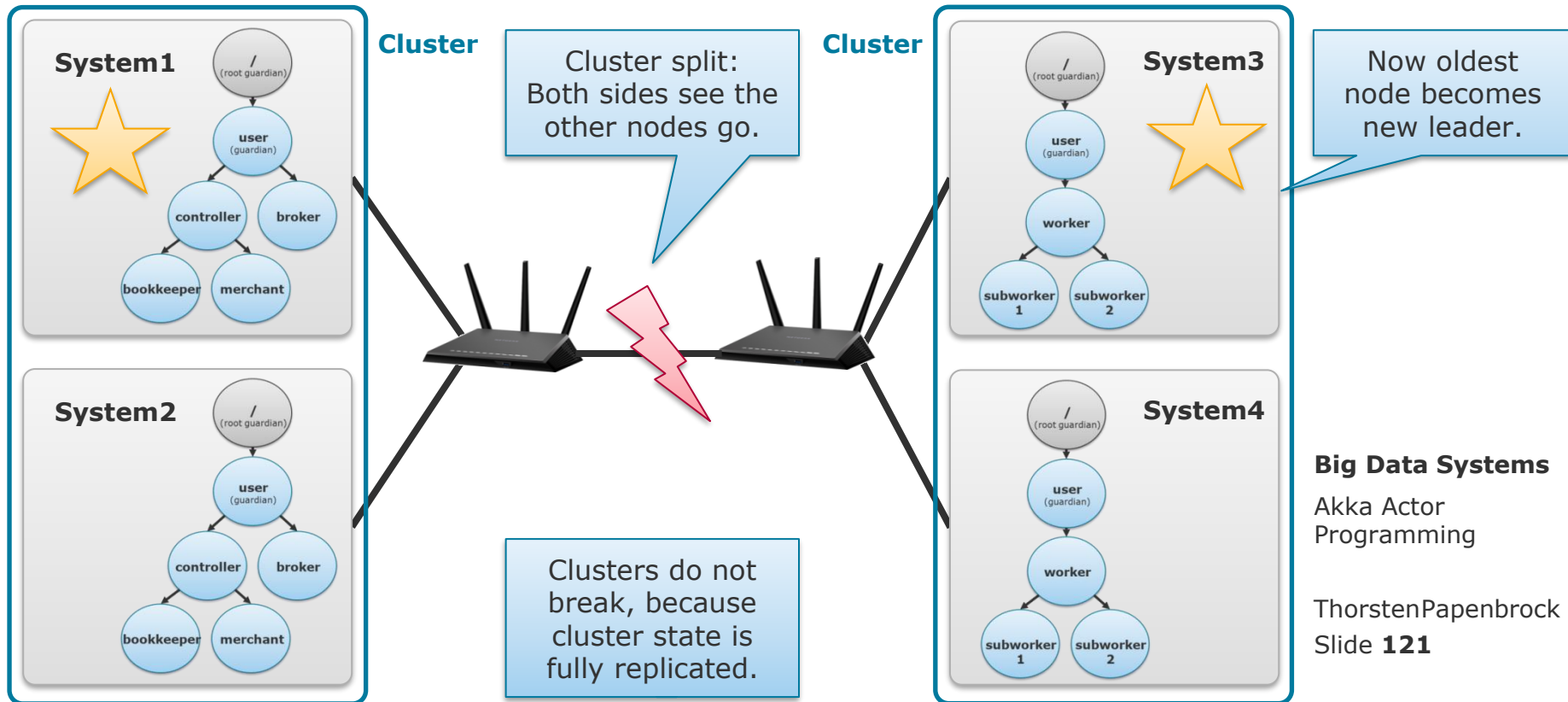
Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **120**

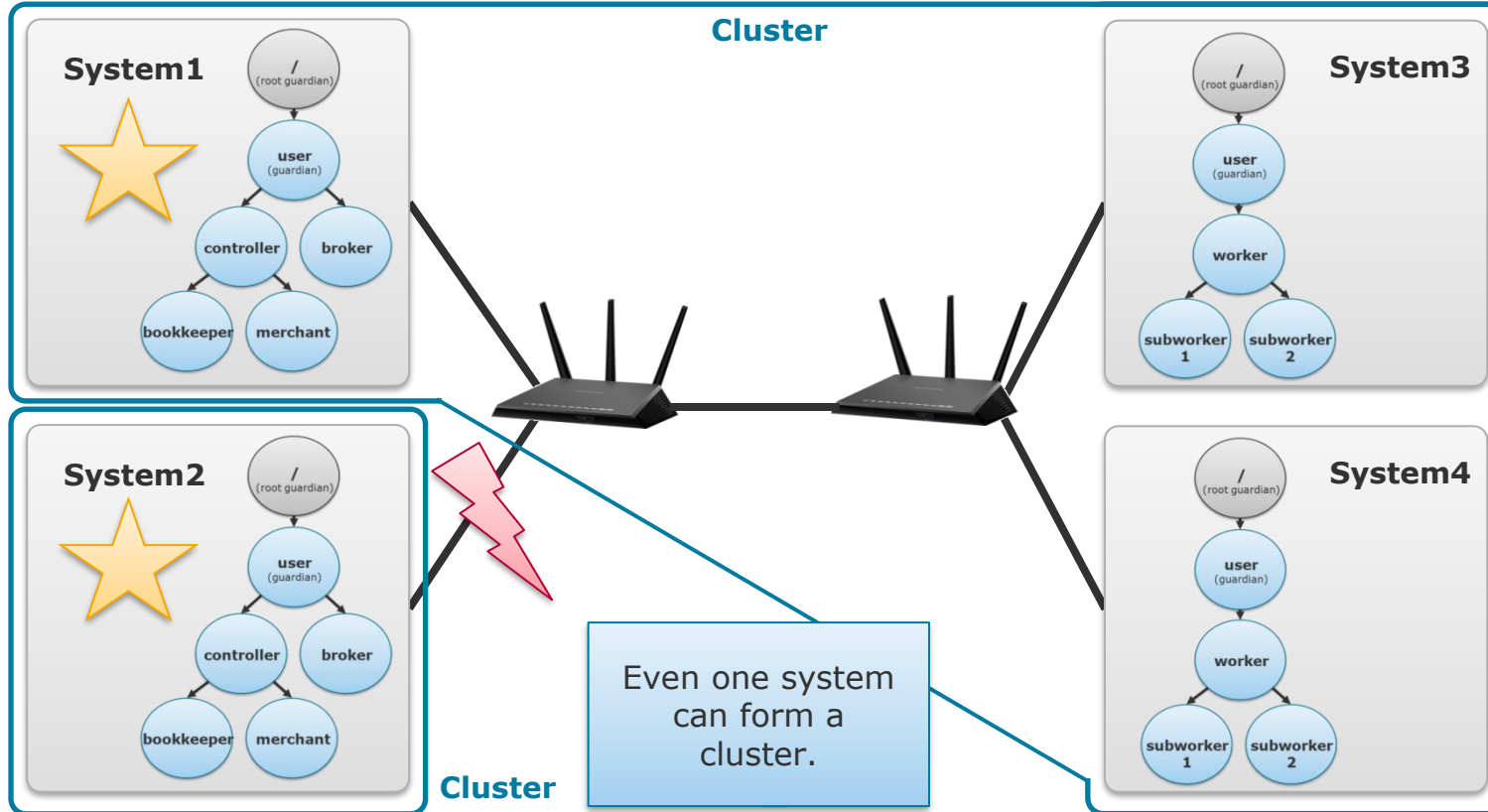
Clustering

Cluster Partitioning



Clustering

Cluster Partitioning



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **122**

Clustering

Cluster Partitioning

If the cluster partitions, ...

- each partition will form its own cluster.
- no *onMemberRemoved()* callback is triggered, because every node stays in some cluster.
- each cluster keeps track of all removed ActorSystems so that "[...] the same actor system can never join a cluster again once it's been removed from that cluster"¹.
(otherwise, the cluster could run into split-brain situations (= two leaders))

To re-unite the nodes, ...

1. identify the "main" cluster
2. terminate all "non-main" clusters
3. and restart ActorSystems on all affected nodes.

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock

Slide **123**

[1] <https://doc.akka.io/docs/akka/2.5/common/cluster.html#cluster-specification>

Clustering Cluster-Aware Scheduler

Akka Classic!

```
int maxWorkersPerNode = 10;
int maxWorkersPerCluster = 1000000;
boolean allowLocalWorkers = true;
Set<String> roles = new HashSet<>(Arrays.asList("worker"));

ActorRef router = system.actorOf(
  new ClusterRouterPool(
    new AdaptiveLoadBalancingPool(SystemLoadAverageMetricsSelector.getInstance(), 0),
    new ClusterRouterPoolSettings(maxWorkersPerCluster, maxWorkersPerNode,
                                  allowLocalWorkers, roles)))
  .props(Props.create(WorkerActor.class), "router");
```

Automatically spawns 10 workers per node
and not more than 1,000,000 per cluster

Scheduling strategy: average load

application.conf

```
akka {
  extensions = ["akka.cluster.metrics.ClusterMetricsExtension"]
  cluster.metrics.native-library-extract-folder=${user.dir}/target/native
}
```

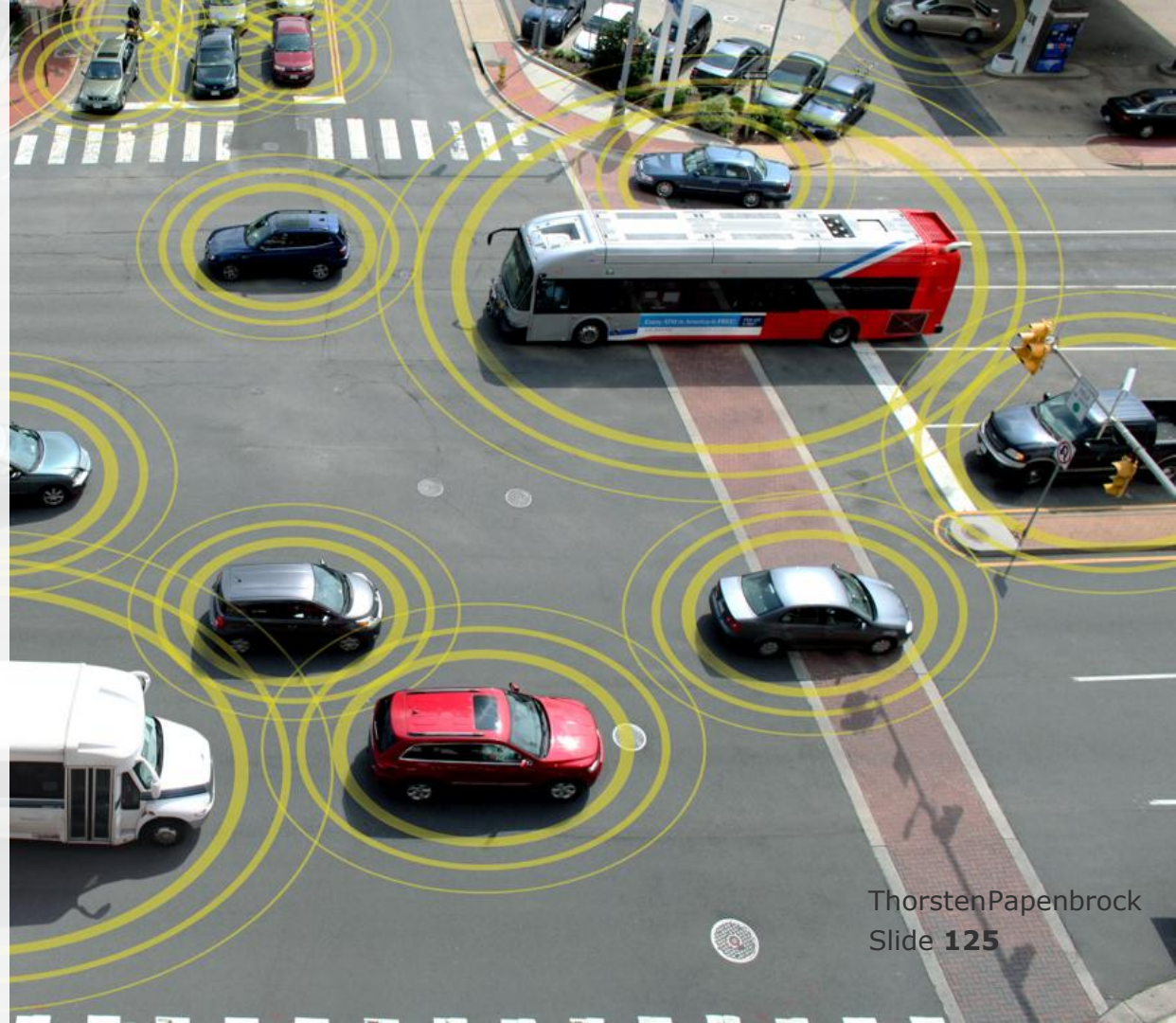
Big Data Systems

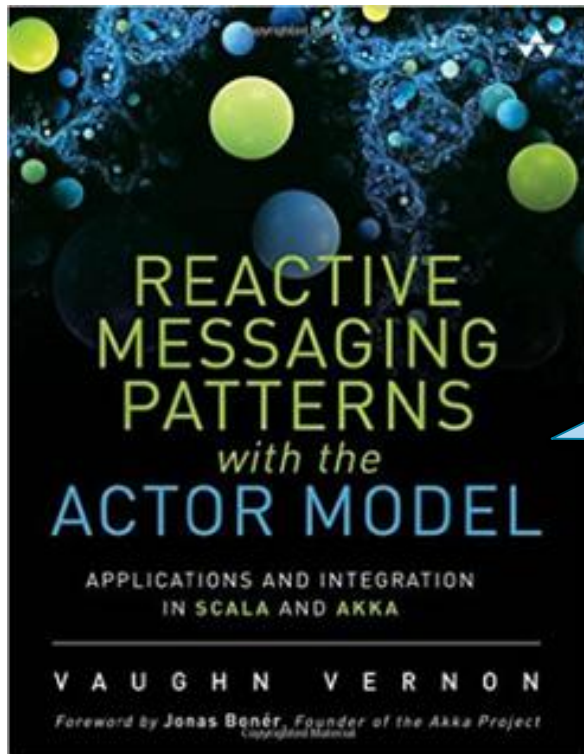
Akka Actor
Programming

ThorstenPapenbrock
Slide **124**

Akka Actor Programming Hands-on

- Actor Model
- Basic Concepts
- Runtime Architecture
- Demo
- Messaging
- Parallelization
- Remoting
- Clustering
- **Patterns**
- Homework





Actor programming is based on a
mathematical model that defines
basic rules for communication
(not a style guide for architecture)

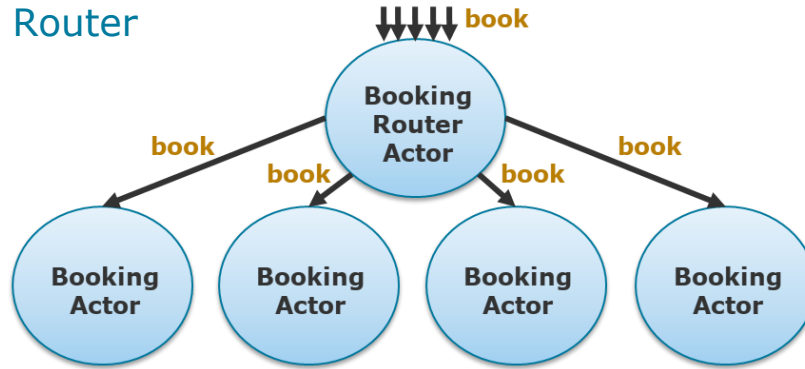
Writing actor-based systems is
based on patterns

Big Data Systems

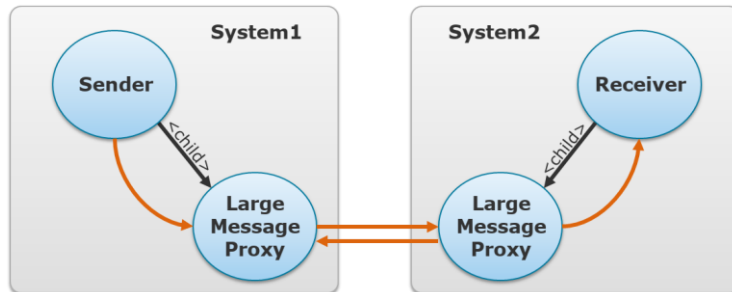
Akka Actor
Programming

ThorstenPapenbrock
Slide **126**

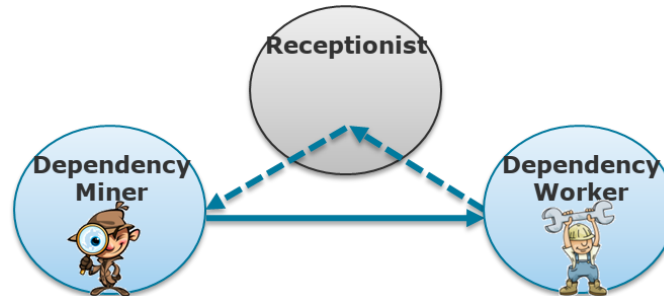
Router



LargeMessageProxy



Receptionist



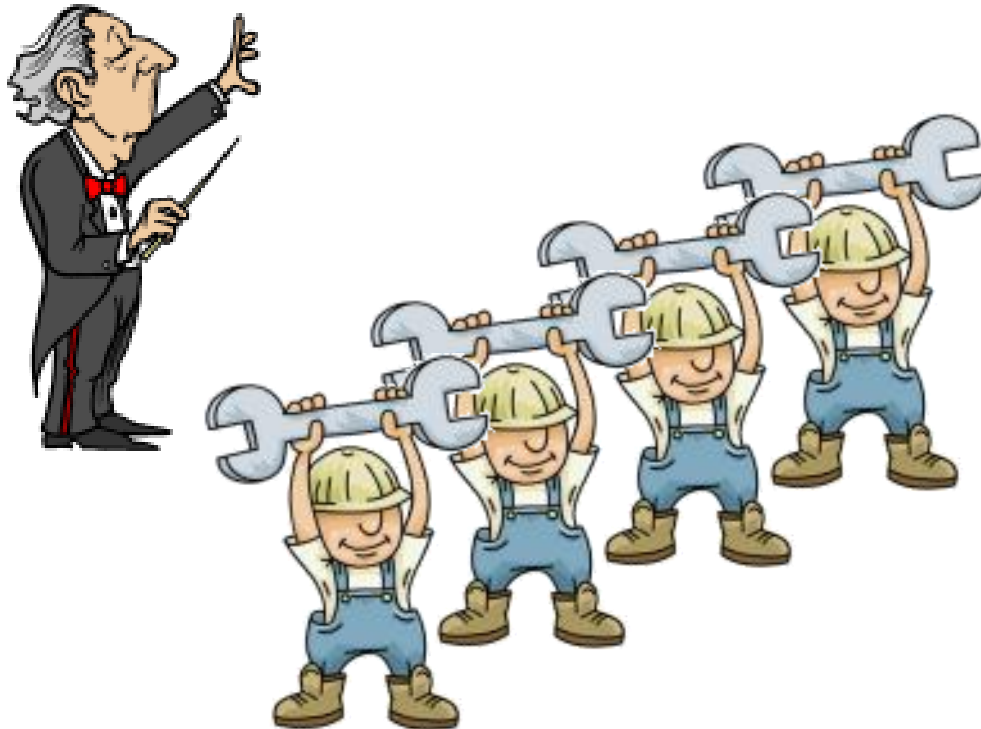
Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **127**

Patterns

Master/Worker



Big Data Systems

Akka Actor
Programming

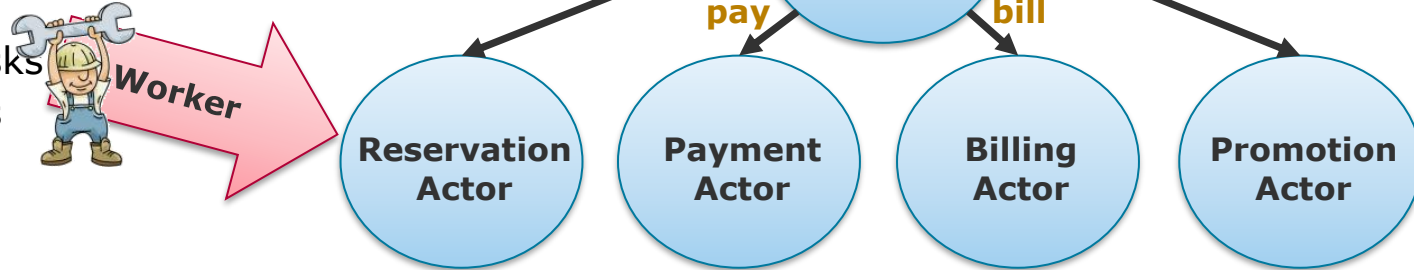
ThorstenPapenbrock
Slide **128**

Patterns

Master/Worker

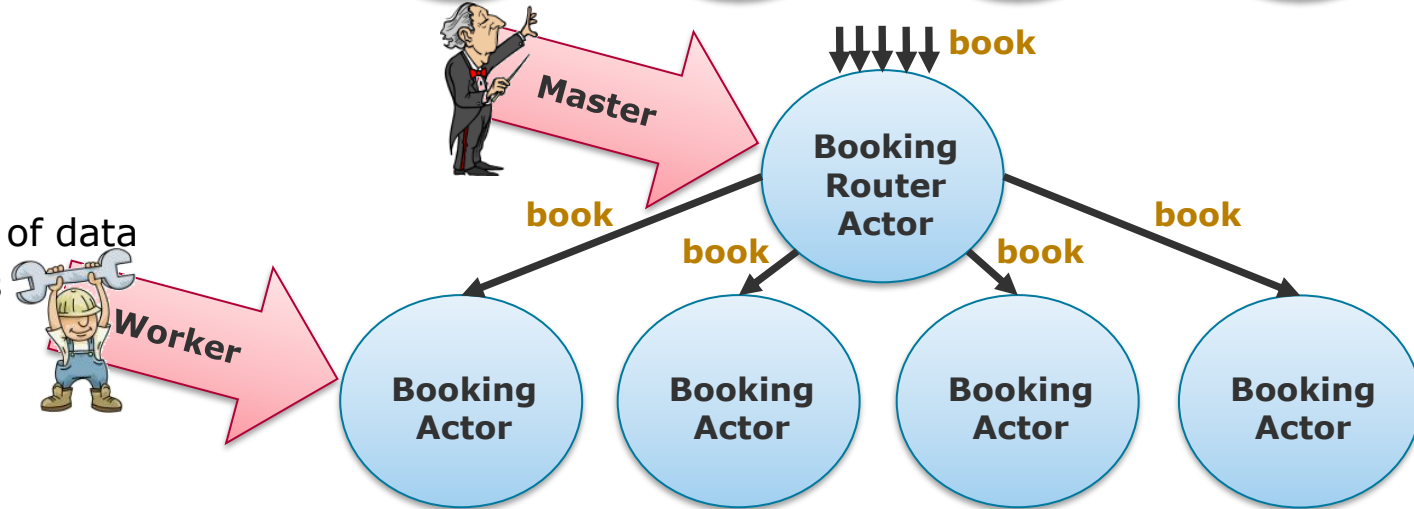
Task-parallelism

- Distribute sub-tasks to different actors



Data-parallelism

- Distribute chunks of data to different actors

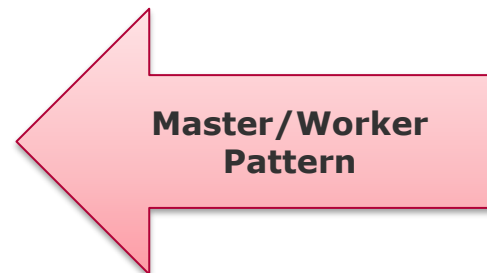
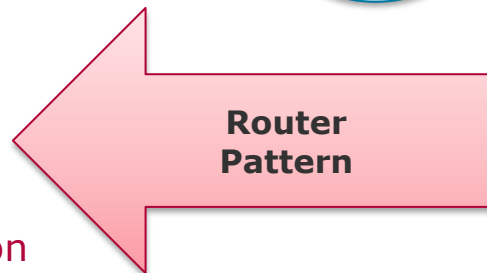
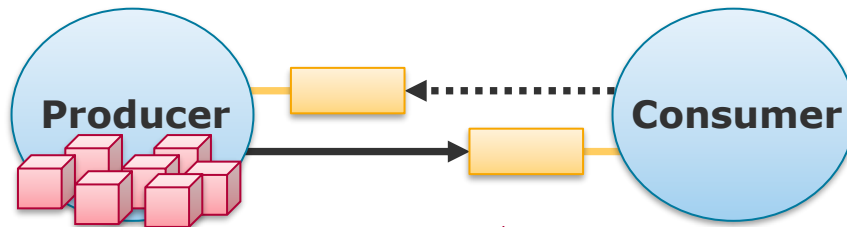


Patterns

Master/Worker

Work Propagation

- **Producer** actors generate work for other **consumer** actors
- **Push propagation:**
 - Producers send work immediately
 - Work queued in inboxes
 - **Fast propagation**; risk for congestion
- **Pull propagation:**
 - Consumers ask for more work
 - Work queued in producer
 - **Slower propagation**; no congestion



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **130**

A concept for ...

1. Parallelization
2. Fault-Tolerance

Master

- Splits a large task into smaller tasks.
- Schedules the work packages to known workers.
- Watches available workers (register new workers; detect and unregister failed workers).
- Monitors task completion (assign pending work packages; re-assign failed work packages).
- Assemble final result (from partial work package results).
 - Does not know how to solve the individual tasks!

Worker

- Register at master.
- Accept and process work packages.
 - Does not know the overall task!

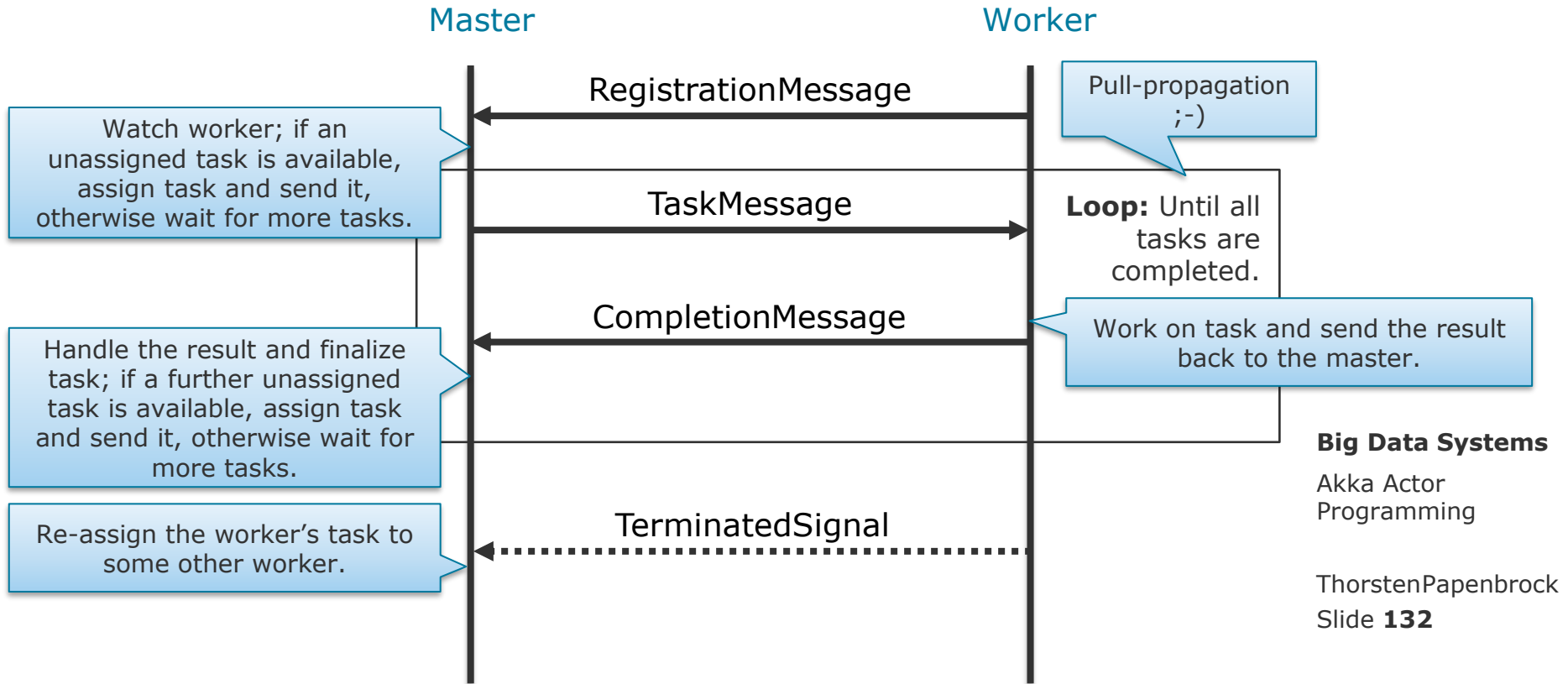
Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **131**

Patterns

Master/Worker



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **132**

Patterns

Master/Worker

Not yet implemented in the ddm-akka project, but highly recommended!

Master

```
public class Master extends AbstractBehavior<Master.Message> {
```

```
    private final Queue<Task> unassignedTasks = new LinkedList<>();
```

```
    private final Queue<ActorRef<Worker.Message>> idleWorkers = new LinkedList<>();
```

```
    private final Map<ActorRef<Worker.Message>, Task> busyWorkers = new HashMap<>();
```

@Override

```
    public Receive<Message> createReceive() {
```

```
        return new ReceiveBuilder()
```

```
            .onMessage(RegistrationMessage.class, this::handle)
```

```
            .onMessage(CompletionMessage.class, this::handle)
```

```
            .onSignal(Terminated.class, this::handle)
```

```
            .build();
```

```
    }
```

```
    ...
```

```
}
```

Note: If we have different types of workers and tasks (task parallelism), we need this for each type!

tasks

worker

worker + tasks

Watch and try to assign work

Collect and send new work

Un-watch and re-assign work

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **133**

Patterns

Master/Worker

Not yet implemented in the ddm-akka project, but highly recommended!

Master

```
private Behavior<Message> handle(RegistrationMessage message) {  
    ActorRef<Worker.Message> worker = message.getWorker();  
  
    if (this.busyWorkers.containsKey(worker) || this.idleWorkers.containsKey(worker))  
        return this;  
  
    this.getContext().watch(worker);  
  
    if (this.unassignedTasks.isEmpty()) {  
        this.idleWorkers.add(worker);  
        return this;  
    }  
  
    Task task = this.unassignedTasks.remove();  
    this.busyWorkers.put(worker, task);  
    worker.tell(new Worker.TaskMessage(work));  
    return this;  
}
```

Ignore if already known

Watch for termination

Idle if no task available

Assign task

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **134**

Patterns

Master/Worker

Not yet implemented in the ddm-akka project, but highly recommended!

Master

```
private Behavior<Message> handle(CompletionMessage message) {  
    ActorRef<Worker.Message> worker = message.getWorker();  
  
    this.resultCollector.tell(new ResultCollector.ResultMessage(message.getResult()));  
  
    if (this.unassignedTasks.isEmpty()) {  
        this.idleWorkers.add(worker);  
        return this;  
    }  
  
    Task task = this.unassignedTasks.remove();  
    this.busyWorkers.put(worker, task);  
    worker.tell(new DependencyWorker.TaskMessage(work));  
    return this;  
}
```

Handle result;
this often generates new tasks!

Idle if no task available

Assign task

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **135**

Patterns

Master/Worker

Not yet implemented in the ddm-akka project, but highly recommended!

Master

```
private Behavior<Message> handle(Terminated signal) {  
    ActorRef<Worker.Message> terminatedWorker = signal.getRef().unsafeUpcast();
```

```
    if (this.idleWorkers.remove(terminatedWorker))  
        return this;
```

Simply remove worker, if it was idle

```
    Task task = this.busyWorkers.remove(terminatedWorker);
```

Get task of terminated actor

```
    if (this.idleWorkers.isEmpty()) {  
        this.unassignedTasks.add(task);  
        return this;  
    }
```

Store task if not assignable

```
    ActorRef<Worker.Message> worker = this.idleWorkers.remove();  
    this.busyWorkers.put(worker, task);  
    worker.tell(new DependencyWorker.TaskMessage(work));  
    return this;  
}
```

Re-assign task

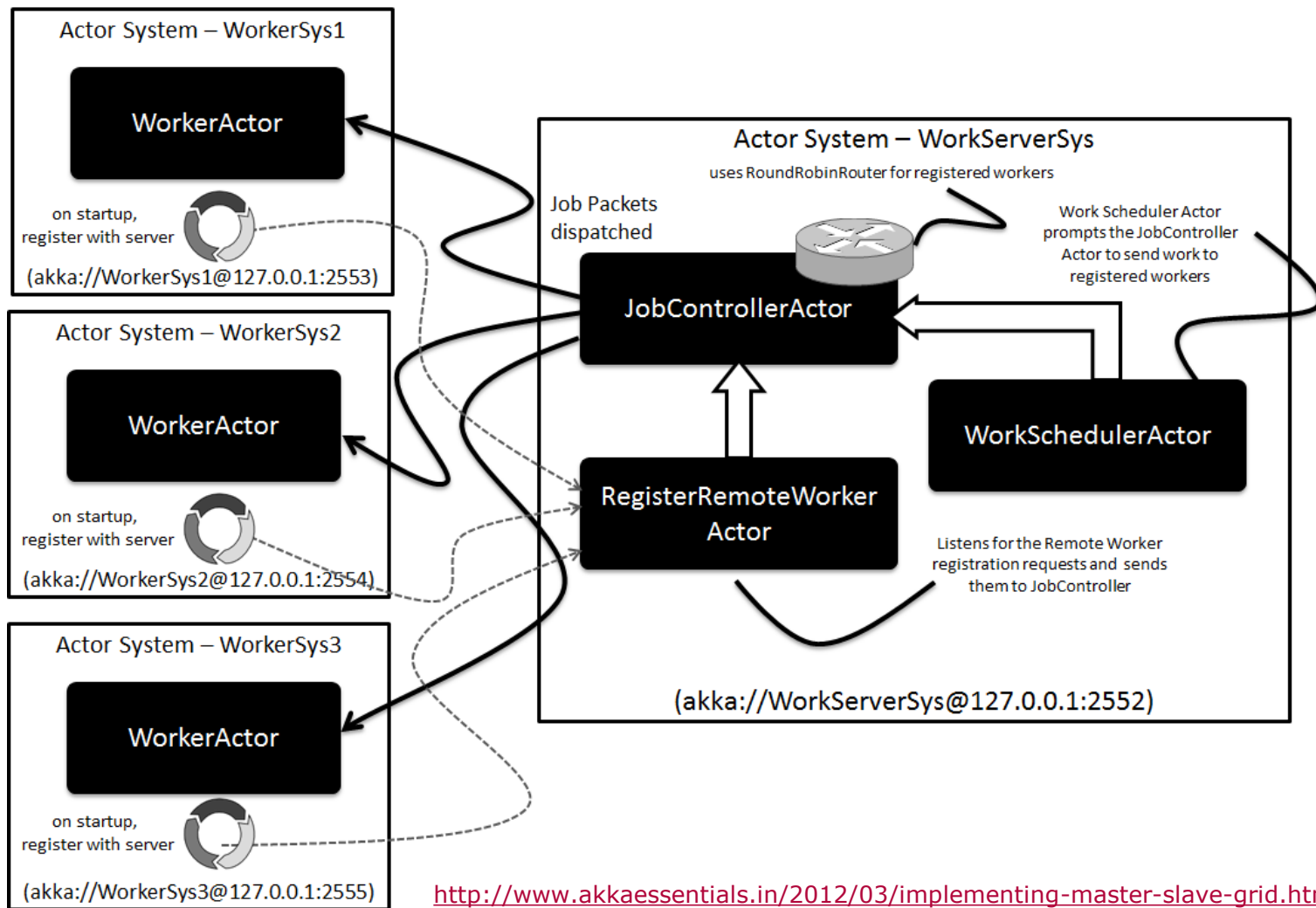
Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **136**

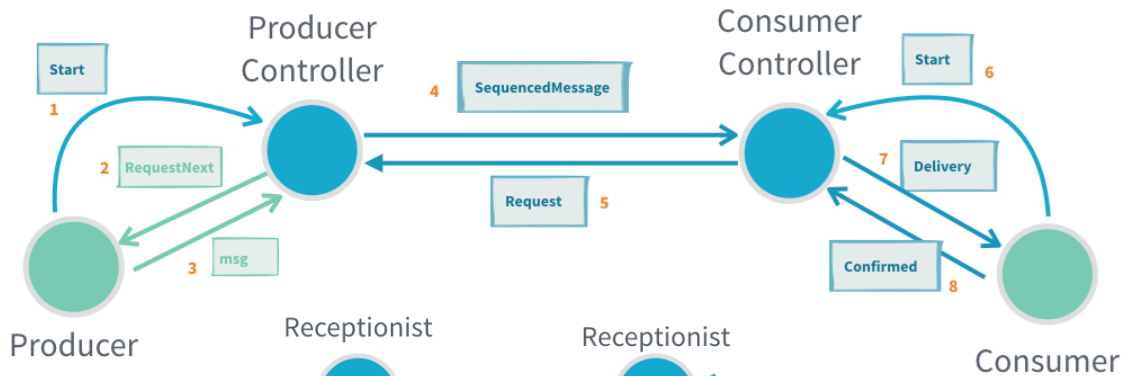
Patterns Master/ Worker

Example



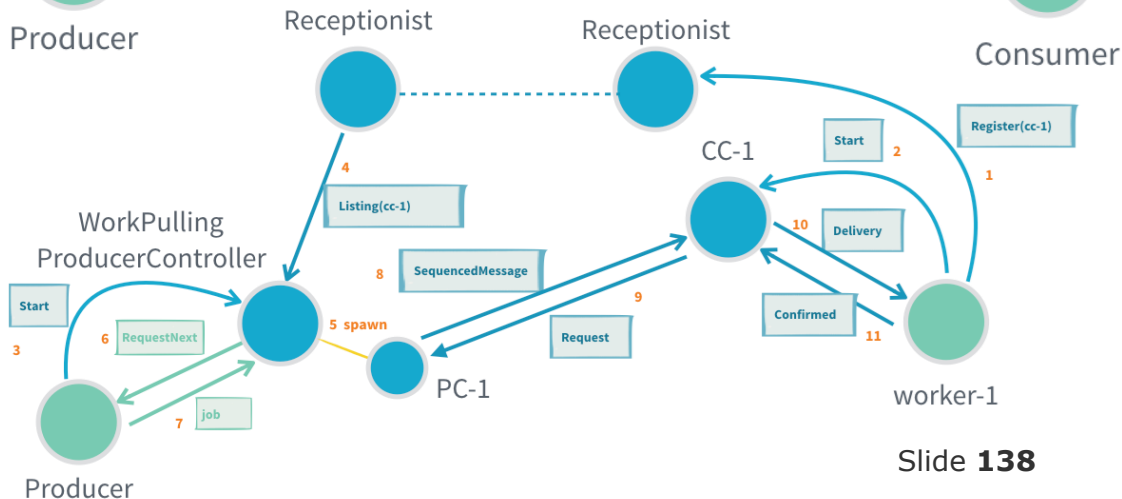
Point-to-Point

- One producer to one consumer
 - Pull-propagation
 - Reliable exactly-once delivery
 - Splitting up of large messages (LargeMessageProxy)



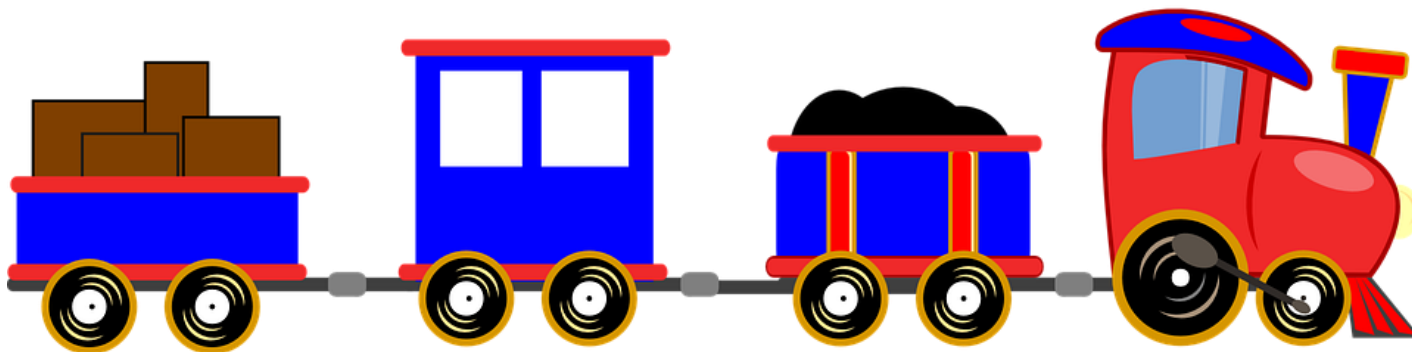
Work Pulling

- One producer to many consumers
 - Pull-propagation
 - Reliable exactly-once delivery
 - Dynamic worker registration (Receptionist)



Patterns

Adapter



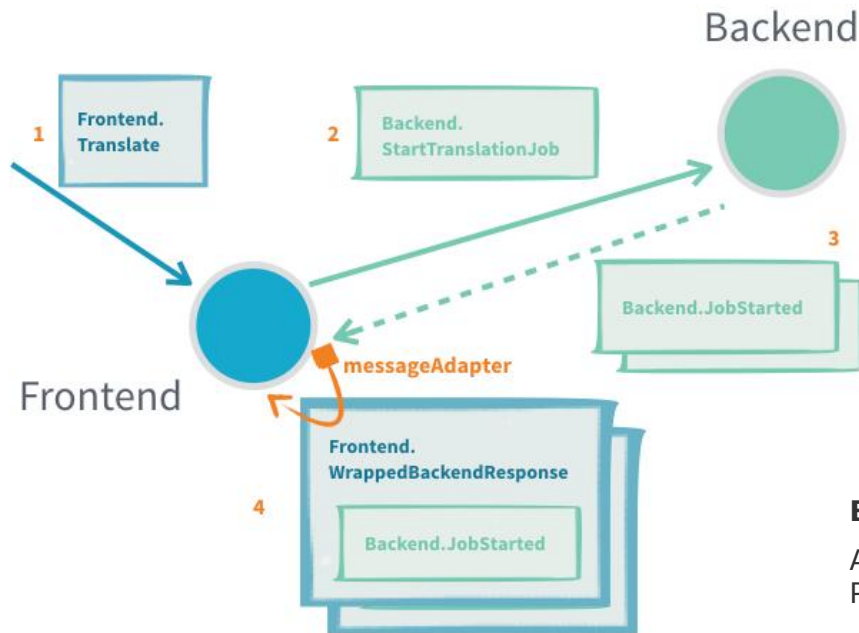
Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **139**

Adapted Response

- Sometimes an actor X needs to respond to another actor Y that was not known at implementation time of X (e.g. when working with legacy actors, actor libraries, or mixed protocols)
- Adapter actors are translators that wrap some response of X into a message type that the requester Y understands.



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **140**

Patterns

Adapter

Answer message of type
`Listing`
(from the `Receptionist` interface)

wrapped in a message
of this actor of type
`ReceptionistListingMessage`
(from the `Worker` interface)

Message adapter actor
definition and registration;
wraps `Listing` objects in
`ReceptionistListingMessage`
objects.

```
public class Worker extends AbstractBehavior<Worker.Message> {  
  
    @Getter @NoArgsConstructor @AllArgsConstructor  
    public static class ReceptionistListingMessage implements Message {  
        Receptionist.Listing listing;  
    }  
  
    private Worker(ActorContext<Message> context) {  
        super(context);  
        final ActorRef<Receptionist.Listing> listingResponseAdapter =  
            context.messageAdapter(Receptionist.Listing.class, ReceptionistListingMessage::new);  
        context.getSystem().receptionist().tell(  
            Receptionist.subscribe(Master.masterService, listingResponseAdapter));  
    }  
  
    @Override  
    public Receive<Message> createReceive() {  
        return newReceiveBuilder()  
            .onMessage(ReceptionistListingMessage.class, this::handle)  
            .build();  
    }  
  
    private Behavior<Message> handle(ReceptionistListingMessage message) {  
        Set<ActorRef<Master.Message>> masters =  
            message.getListing().getServiceInstances(Master.masterService);  
        return this;  
    }  
}
```

Characteristic:
The target actor responds
with a message from its
own interface.



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **142**



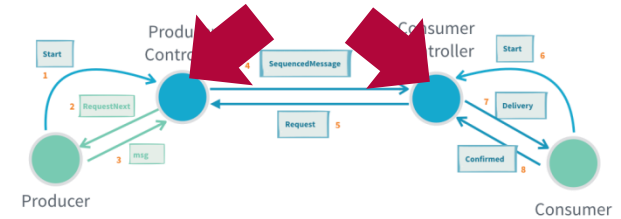
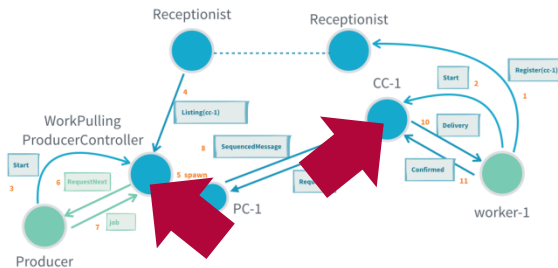
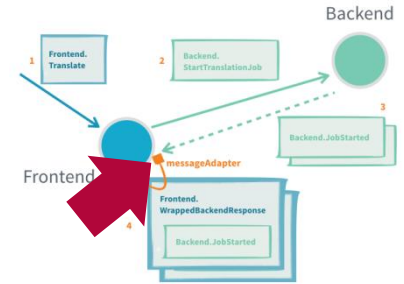
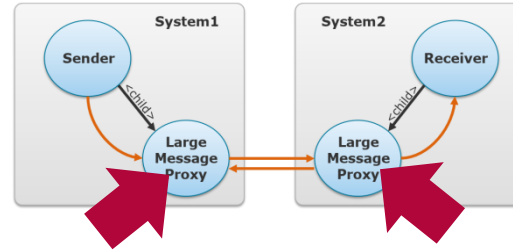
Proxy Actor

- Acts as an agent or surrogate for some other actor.
 - Handles a certain (standard) task.
 - Serves to ...
 - externalize behavior/state.
(e.g., prevent cluttering code in real actor)
 - hide the real actor.
(e.g., protect against DOS attacks)
 - handle short-lived concepts.
(e.g., communications)
 - handle resource/time intensive actions.
(e.g., data transfer)
- Other actors “think” they were talking to the real actor!

Patterns

Proxy

Patterns with proxy actors – so far:



Example: Simple Proxy

- Delegate a new communication to a proxy.
- If the communication returns a result, the proxy reports it to the real actor.

@Override

```
public Receive<Message> createReceive() {  
    return new ReceiveBuilder()  
        .onMessage(HelloMessage.class, message -> {  
            ActorRef<MyProxy.Message> proxy = this.getContext().spawn(  
                MyProxy.create(this.getContext().getSelf()), "proxy");  
            message.getSender().tell(new MyProxy.HelloBackMessage(proxy));  
        })  
        .match(ProxyResultMessage.class, this::handle)  
        .build();  
}
```

Create a proxy for
this communication.

Final result
from the proxy.

The response to the sender
provides the proxy actor as
further replyTo-address.

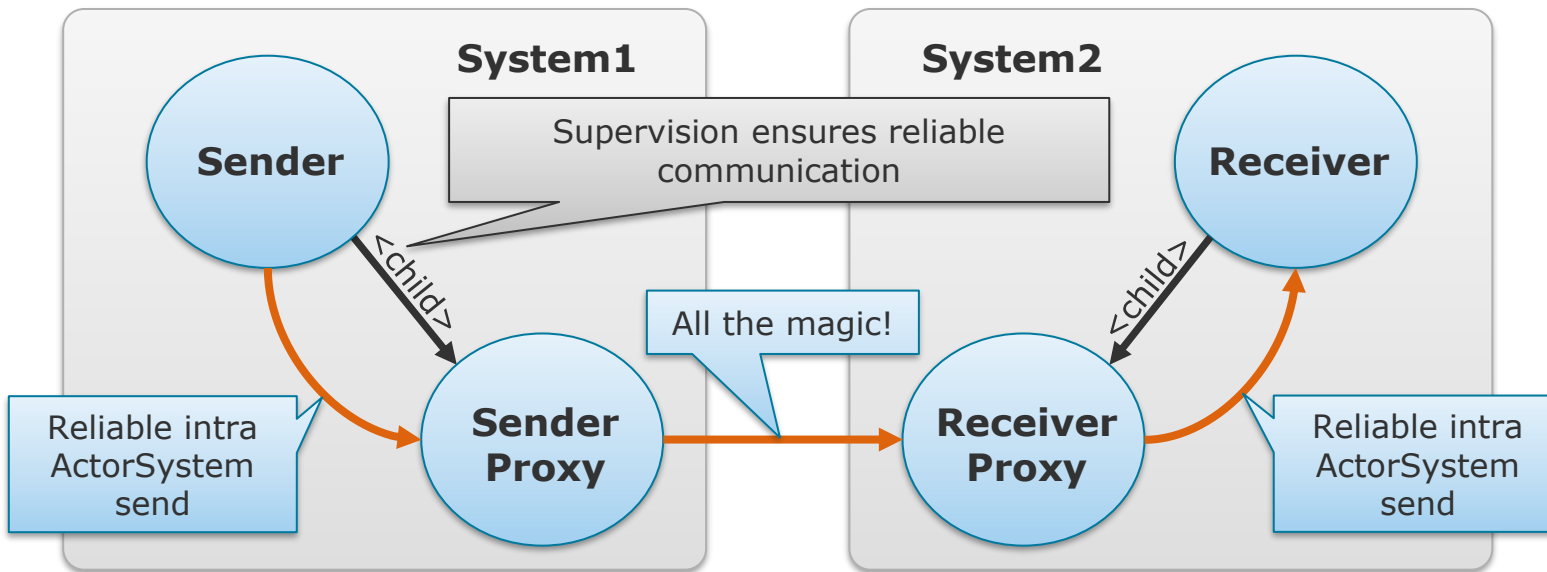
Big Data Systems

Akka Actor
Programming

Thorsten Papenbrock
Slide 145

Example: Reliable Communication Proxy

- Provides **exactly-once messaging** on top of at-most-once messaging
- Implements an ACK-RETRY protocol



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **146**

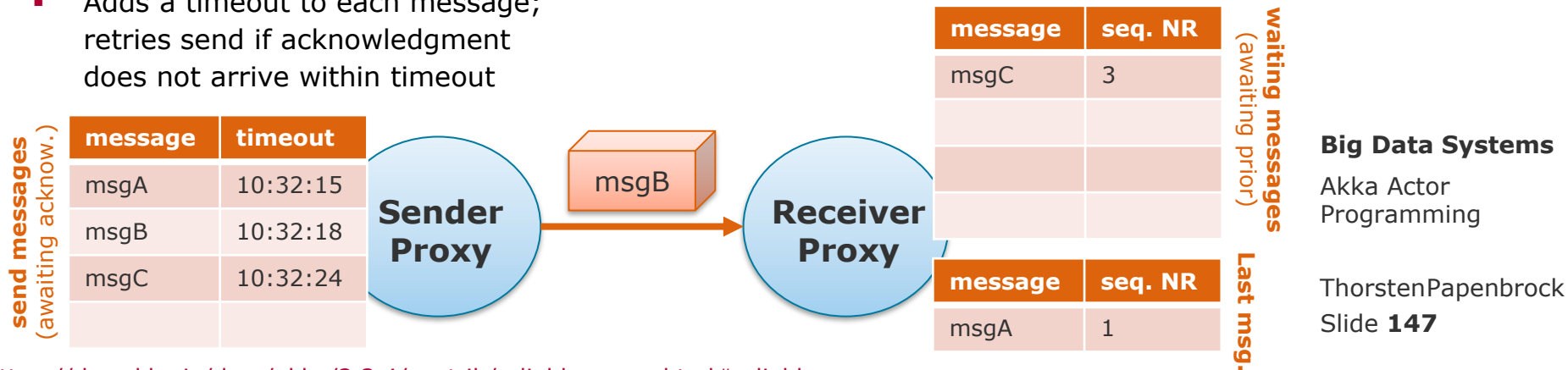
Example: Reliable Communication Proxy

Sender Proxy

- Adds sequence numbers to messages
- Forwards messages to Receiver Proxy
- Stores messages until successfully acknowledged by Receiver Proxy
- Adds a timeout to each message; retries send if acknowledgment does not arrive within timeout

Receiver Proxy

- Acknowledges received messages to Sender Proxy
- Forwards acknowledged messages to Receiver
- Detects missing/duplicate messages by checking sequence number of last forwarded message



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **147**

Sender Proxy

- ## Receiver Proxy

- Acknowledges received messages to Sender Proxy
- Forwards acknowledged messages to Receiver
- Detects missing/duplicate messages by checking sequence number of last forwarded message



Akka Actor Programming

Thorsten Papenbrock
Slide **148**

```
private final TimerScheduler<Message> timers;
```

TimerScheduler allow us to schedule future and periodic message sends!

```
public static Behavior<Message> create() {  
    return Behaviors.setup(  
        context -> Behaviors.withTimers(timers -> new ReliableProxy(context, timers));  
    }  
}
```

Wrapping this behavior in a timer behavior gives us access to the TimerScheduler.

```
private ReliableProxy(ActorContext<Message> context, TimerScheduler<Message> timers) {  
    super(context);  
    this.timers = timers;  
}
```

@Override

```
public Receive<Message> createReceive() {  
    return newReceiveBuilder()  
        .onMessage(StartTimerMessage.class, message -> {  
            this.timers.startTimerAtFixedRate("PingMessage", new PingMessage(),  
                Duration.ofSeconds(10), Duration.ofSeconds(5));  
            return this;  
        })  
        .onMessage(StopTimerMessage.class, message -> {  
            this.timers.cancel("PingMessage");  
            return this;  
        })  
        .onMessage(PingMessage.class, this::handle)  
        .build();  
}
```

Start a timer that sends a PingMessage to this actor after 10 seconds every 5 seconds.

Stop the timer "PingMessage".

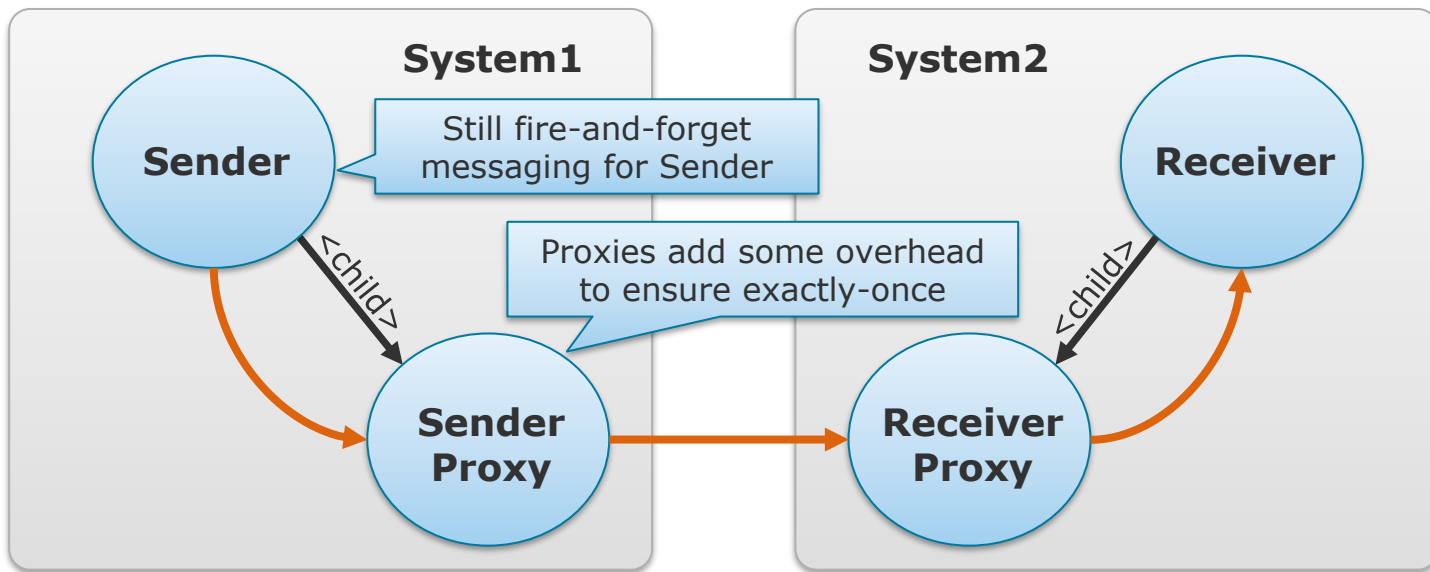
Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **149**

Example: Reliable Communication Proxy

- Provides **exactly-once messaging** on top of at-most-once messaging
- Implements an ACK-RETRY protocol

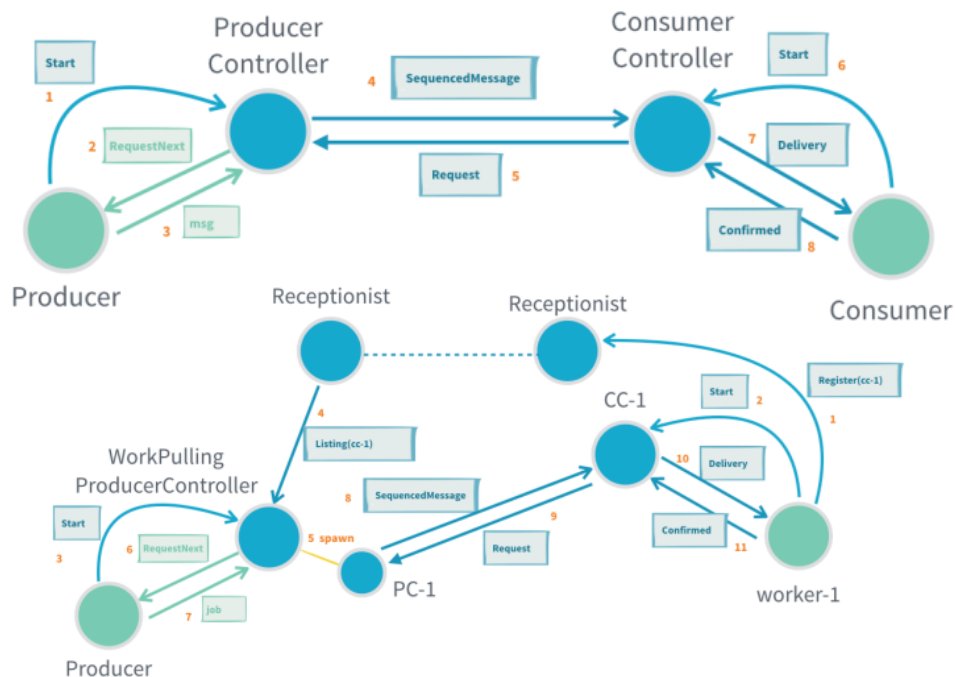


Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **150**

Example: Reliable Communication Proxy



Akka's reliable delivery patterns do exactly what we just learned.

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **151**

Patterns
Ask



Patterns

Ask

Tell messaging

- non-blocking, asynchronous, fire-and-forget
 - Java: `someActor.tell(message)`
 - Scala: `someActor ! Message`

Ask pattern

- blocking, synchronous
 - Java: `someActor.ask(message)`
 - Scala: `someActor ? message`
- Returns a `Future` that the calling entity can wait for



Patterns

Ask

Tell messaging

- non-blocking
 - Java:
 - Scala:

Ask pattern

- blocking
 - Java:
 - Scala:
- Return

Ask pattern

- blocking, synchronous

- Java:

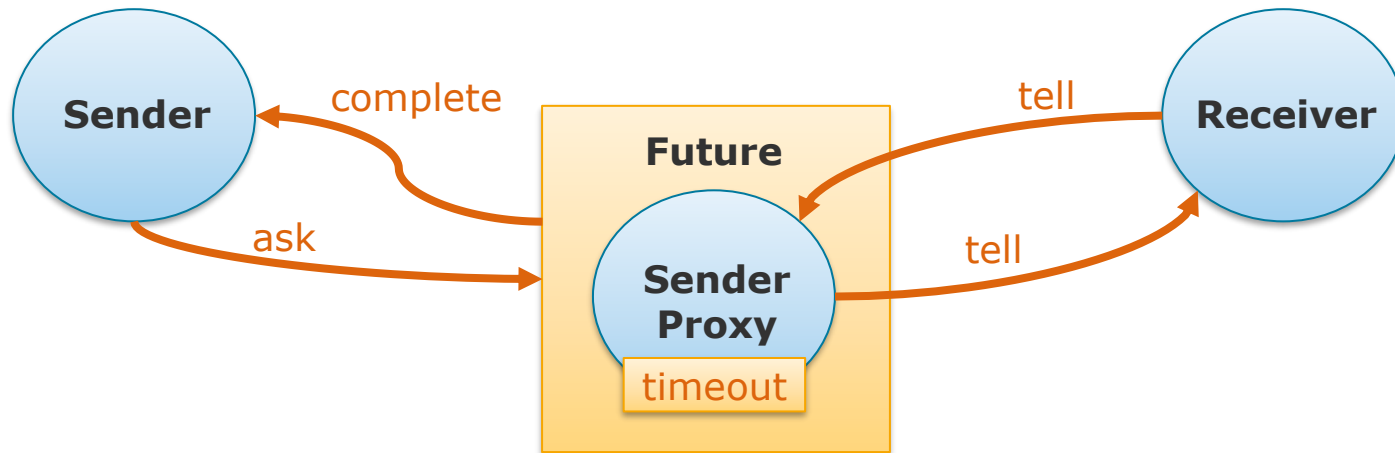
```
CompletionStage<Message> result = AskPattern.ask(
    someActor,
    replyTo -> new SomeActor.QuestionMessage(replyTo),
    timeout,
    system.scheduler());
```

```
result.whenComplete(
    (reply, failure) -> {
        if (reply instanceof AnswerMessage)
            System.out.println("Success!");
        else System.out.println("Failed " + failure);
    });
```

- Essentially the same but separated from the ActorRef class

Ask pattern

- Via ask, the **Sender** creates a `Future` that wraps a **Proxy** actor that tells the message to a **Receiver** with a `timeout` for that message.
- The **Proxy** completes the `Future` either when it receives a response or the `timeout` elapses.



Ask pattern

- Useful if ...
 - the outside, non-actor world needs to communicate with an actor.
- Not a good solution to ...
 - make the communication reliable, i.e., enable exactly-once messaging (use reliable proxy pattern).
 - implement timeouts for message sends (use scheduled tasks).
 - let an actor wait for a response (use reactive waiting – see later).

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **156**

Avoid “ask” if possible!

(the need to ask is usually the result of a bad architecture)

Why to avoid ask:

- **Paradigm violation**
 - Synchronous calls break the strict decoupling of actors and introduce alternative parallelization paradigms (under the hood).
- **Resource blocking**
 - Actively waiting for other actors locks resources (in particular threads) and the Future-construct does require additional threads.
- **Inefficient messaging**
 - Asking requires more effort than telling messages (i.a. for additional threads and timeouts).
- **Error susceptibility**
 - Asking is highly prone to deadlocks (A asks B while B asks A; both cannot see that they are waiting for each other).
- **Timeout sensitivity**
 - How do find a good timeout and what do we do if the timeout elapses? Ask again?

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **157**

Patterns Ask

Wait!

We can use **Futures**
with the **Pipe-Pattern**
in a non-blocking way!

- Example from the
official Akka Docu:

[https://doc.akka.io/docs/
akka/2.5/actors.html](https://doc.akka.io/docs/akka/2.5/actors.html)

```
import static akka.pattern.PatternsCS.ask;
import static akka.pattern.PatternsCS.pipe;

import java.util.concurrent.CompletableFuture;
Timeout t = Timeout.create(Duration.ofSeconds(5));

// using 1000ms timeout
CompletableFuture<Object> future1 =
    ask(actorA, "request", 1000).toCompletableFuture();

// using timeout from above
CompletableFuture<Object> future2 =
    ask(actorB, "another request", t).toCompletableFuture();

CompletableFuture<Result> transformed =
    CompletableFuture.allOf(future1, future2)
        .thenApply(v -> {
            String x = (String) future1.join();
            String s = (String) future2.join();
            return new Result(x, s);
        });

pipe(transformed, system.dispatcher()).to(actorC);
```

- Futures are an alternative model for parallelization.
- There are many discussions on “Actors vs. Futures” as means for parallelization control.
 - The question here is more which pattern for parallelization you prefer!
- Actors + Futures is a bad decision, because ...
 - the mix both models makes your **code harder to understand and maintain**.
 - Futures should never mutate actor state to avoid **concurrency issues**.
 - Futures cannot be passed between actors and, hence, **violate location transparency**.
 - Futures make **assumptions about the timing of messages**.
 - **callbacks** are needed to avoid blocking – why this is problematic:
 - Callbacks are executed by non-actor threads on the side (i.e., the callback thread might be completing a Future while the actor that created the Future might process a different message at the same time).
 - Complicates **debugging**, **parallelization control** and **resource management**.
 - Prone to introduce **shared mutable state** and, hence, to **break encapsulation**.
 - Interferes with **failure handling**, because the asked actor needs to reply with certain, ask-specific error messages that completes the Future.

For instance, Chris Stuccio’s blog:

https://www.chrisstucchio.com/blog/2013/actors_vs_futures.html

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock

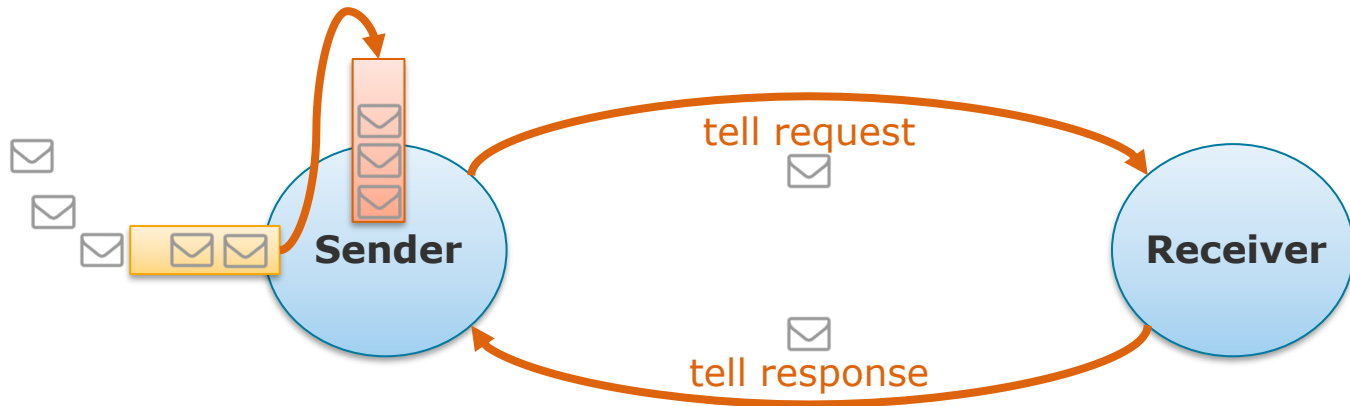
Slide **159**

Patterns

Reactive Waiting

Ok, no ask. So what else can we do to **wait** for a particular response?

1. Create a simple queue in the actor's private state.
2. Append all messages, which are not the awaited message, to this queue.
3. Once the awaited message arrives ...
 1. process the awaited message;
 2. process all messages from the queue in FIFO order;
 3. continue processing messages from the actor's mailbox.

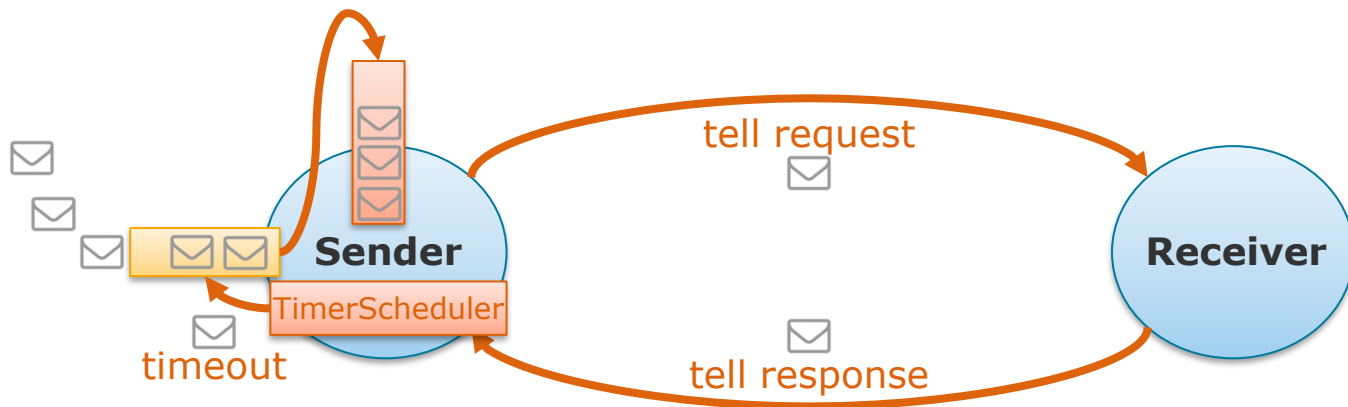


Patterns

Reactive Waiting

And how to **ensure** that either the result or a “message-failed” status is recorded?

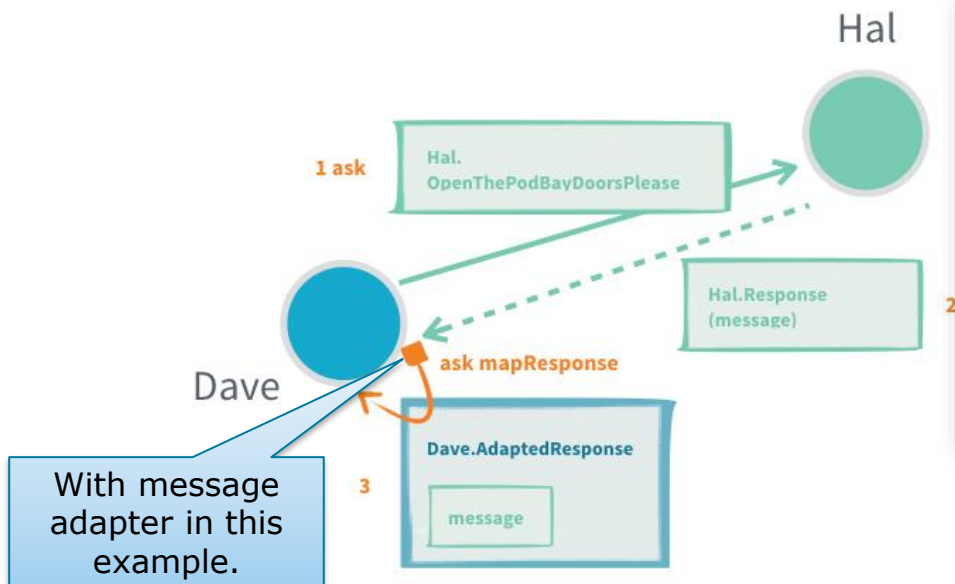
- Create timeout with a TimerScheduler:
 - If it sends a timeout message, assume the call to have failed.
 - If the Receiver sends a response, cancel the TimerScheduler.



Patterns

Reactive Waiting

- Akka's **Request-Response with ask** pattern basically implements ask between actors in this way:



```
context.ask(  
    Hal.HalResponse.class,  
    hal,  
    timeout,  
    // construct the outgoing message  
    (ActorRef<Hal.HalResponse> ref) -> new Hal.OpenThePodBayDoorsPlease(ref),  
    // adapt the response (or failure to respond)  
    (response, throwable) -> {  
        if (response != null) {  
            return new AdaptedResponse(response.message);  
        } else {  
            return new AdaptedResponse("Request failed");  
        }  
    }  
));
```

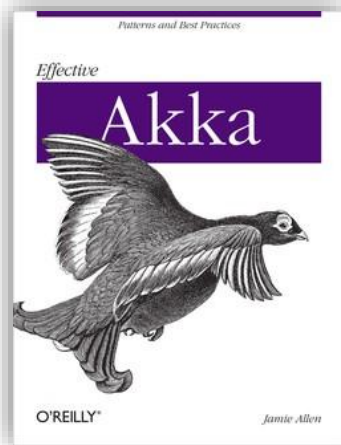
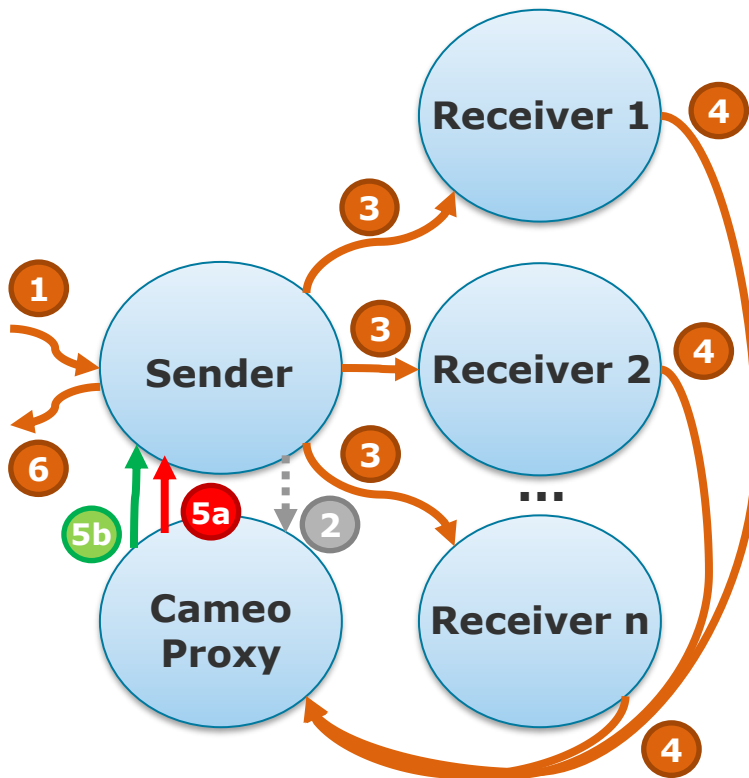
Dave will receive one of these AdaptedResponses next.

Patterns

The Cameo Pattern

How to **recognize** that a complex task has succeeded or failed?

- The Sender issues a set of coherent subtasks.
- The CameoProxy ...
 - collects multiple responses that belong to the same context;
 - encapsulates waiting and timeout functionality;
 - Ensures that the sender receives the **entire result** or a **failure message**.



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **163**

Patterns

Singleton



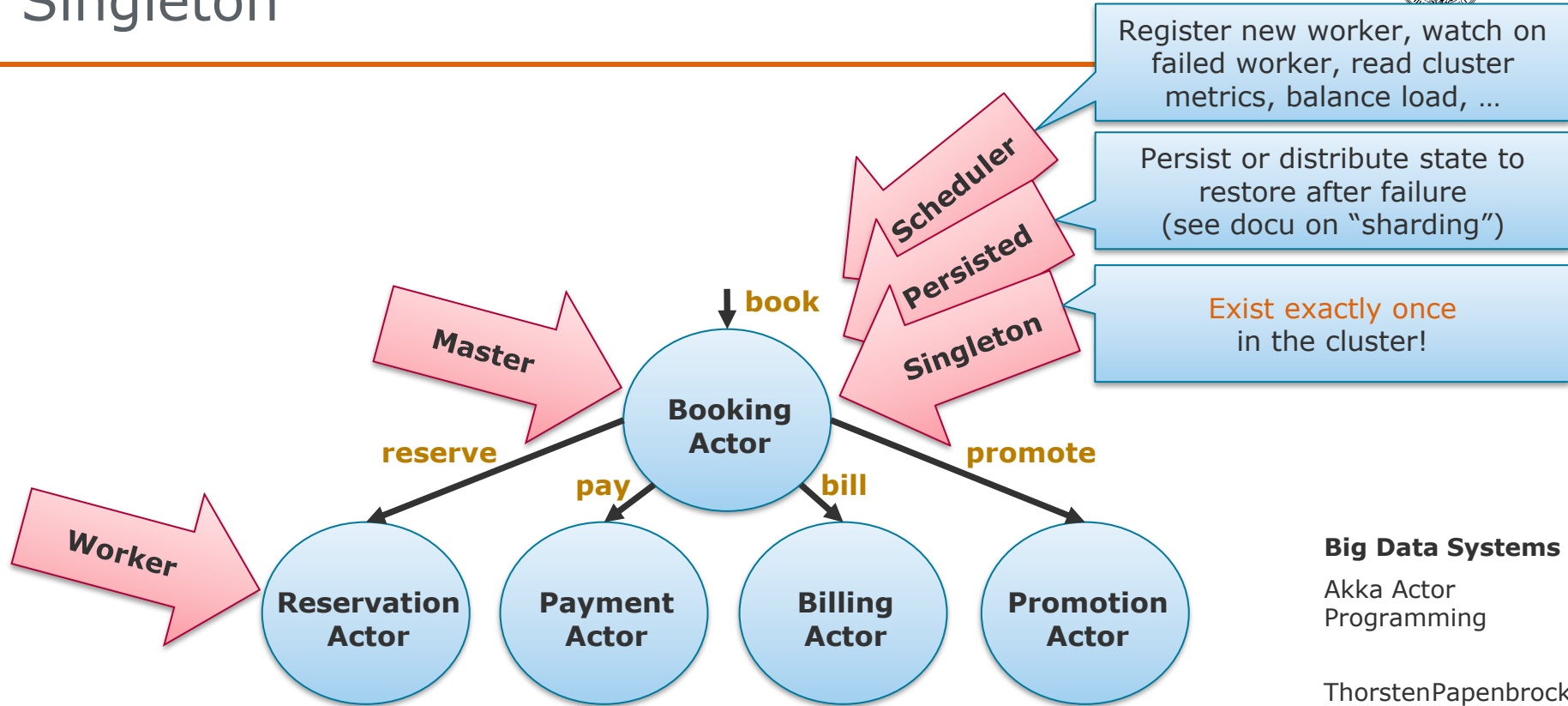
Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **164**

Patterns

Singleton



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **165**

Motivation

- Sometimes, there needs to be exactly one actor of some type, e.g.,
 - one Endpoint actor for external communication;
 - one Leader actor for consensus enforcement;
 - one Resource actor of some type.

First idea:

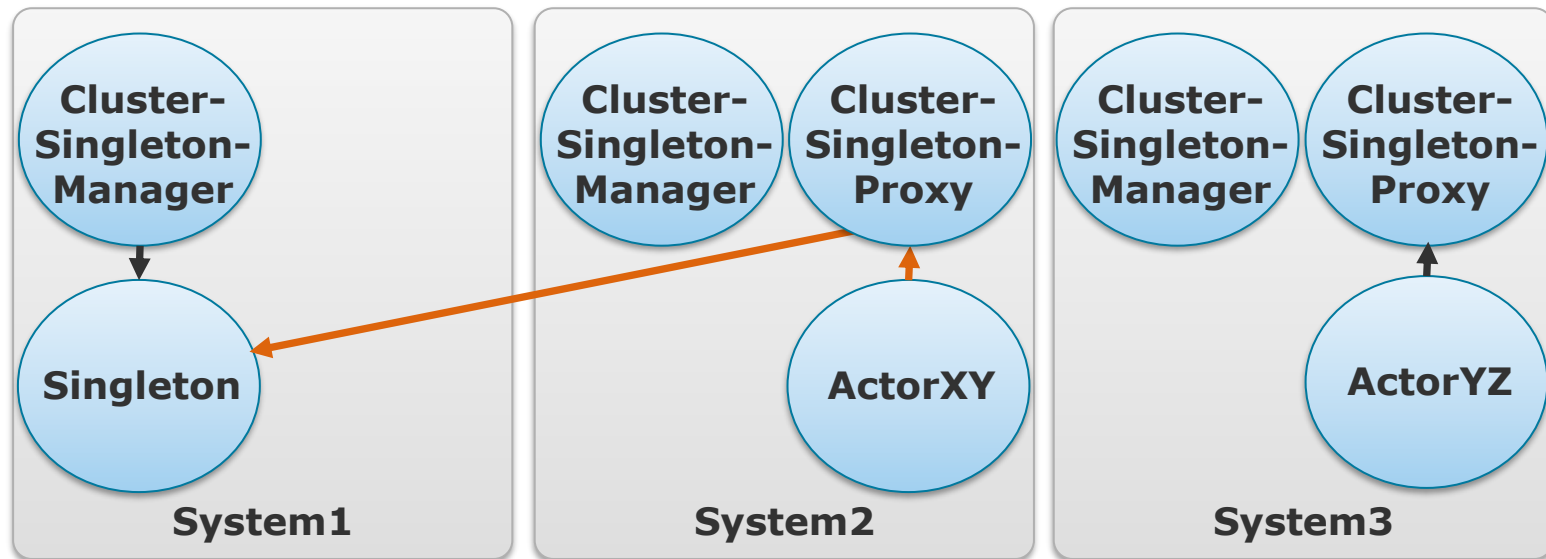
- Simply create that actor once in the cluster.
- Problems:
 1. Requires a **dedicated ActorSystem** that is responsible for creating the singleton.
 2. If that ActorSystem dies, the **singleton is unavailable** until the ActorSystem is back.
 3. Starting the same dedicated ActorSystem twice might cause **split brain**.
 4. Every ActorSystem needs to **know the address** of the singleton.

Patterns

Singleton

Cluster Singleton

- `akka.cluster.singleton.ClusterSingletonManager`
 - Runs in every ActorSystem
 - Creates exactly one Singleton actor in the cluster (on the oldest node; singleton moves if node goes down)
- `akka.cluster.singleton.ClusterSingletonProxy`
 - Created to communicate with the singleton
 - Redirects messages to the current Singleton actor (buffering messages if singleton is temporarily unavailable)



Patterns

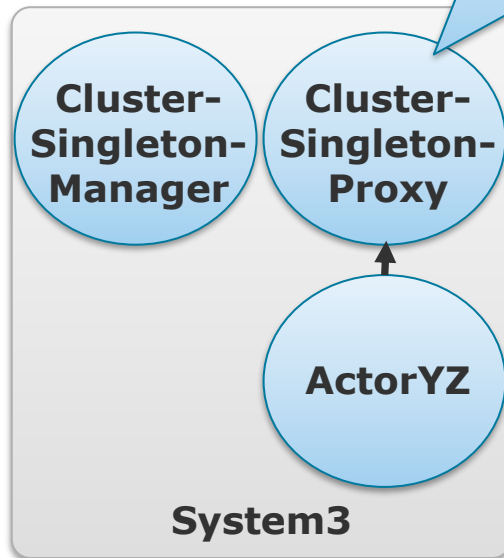
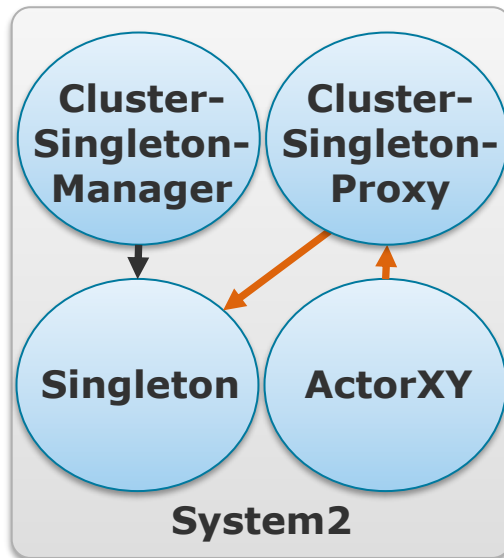
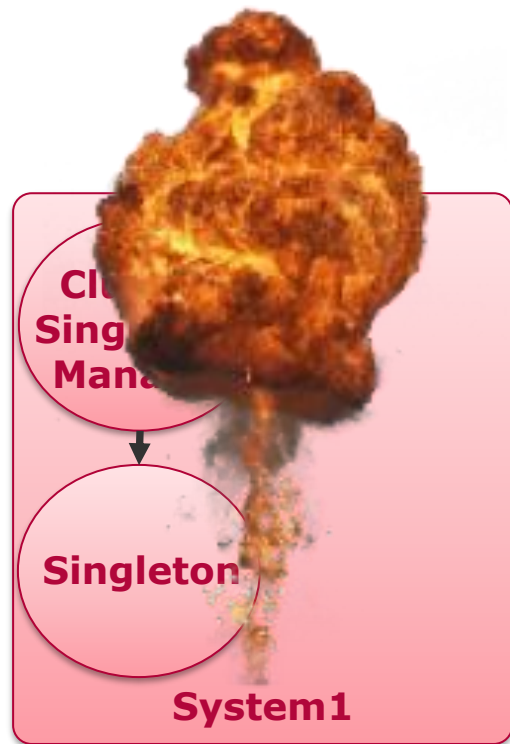
Singleton

Cluster Singleton

- `akka.cluster.singleton.ClusterSingletonManager`
 - If an ActorSystem hosting the singleton dies, the singleton is re-created on the then oldest node.

- `akka.cluster.singleton.ClusterSingletonProxy`
 - Knows where the current singleton lives and tracks singleton movements.

Note:
This is no "reliable proxy"
so messages can get lost!



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **168**

Be aware that cluster singletons

- easily become a **bottleneck**;
- are not guaranteed 100% **available**;
- are **duplicated** when clusters split.

Cluster Singleton

```
ClusterSingleton singleton = ClusterSingleton.get(this.getContext().getSystem());
```

```
ActorRef<Master.Message> masterProxy = singleton.init(SingletonActor  
    .of(Master.create(), "Master")  
    .withStopMessage(Master.StopMessage.INSTANCE));
```

```
masterProxy.tell(new Master.RegistrationMessage());
```

A call to `init()` returns an `ActorRef` of the `SingletonProxy` (same interface) that forwards all messages to the singleton (wherever that instance is).

A call to `init()` spawns

- the local **SingletonManager** (if it is not already started locally)
- the **singleton actor** (if it is not already started globally)
- a local **SingletonProxy** (to the specified singleton)

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **169**

Patterns Reaper



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **170**

Application Shutdown?

Task vs. Actor shutdown

- Tasks finish and vanish.
- Actors finish and wait for more work.
 - Actors need to be **notified** to stop working.

How to detect that an application has finished?

- **All mailboxes empty?**
 - No: Actors might still be working on messages (and produce new ones).
- All mailboxes empty **and all actors idle?**
 - No: Messages can still be in transmission, i.e., on the network.
- All mailboxes empty and all actors idle **for “a longer time”?**
 - No: Actors might be idle for “longer times” if they wait for resources.
- **Only the application knows when it is done** (e.g. a final result was produced).



Application Shutdown?

Problem

- ActorSystems stay alive when the main application thread ends.

Forced shutdown

- Kill the JVM process.
- Problems:
 - Risk of resource corruption (e.g. corrupted file if actor was writing to it)
 - Many, distributed JVM processes that need to be killed individually

ActorSystem terminate

```
this.getContext().getSystem().terminate();
```

- Calling terminate() on an ActorSystem will stop() all its actors.
- Problem:
 - Communication protocols will not end in a clean way.
 - ActorSystems on remote nodes are still alive!

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **172**

Application Shutdown?

Problem

- ActorSystems stay alive when the main application thread ends.

Akka coordinated shutdown

- Akka lets you define callback functions for a coordinated shutdown.
- These can be used to cleanly terminate protocols and other ActorSystems.
- Problem:
 - Not a general solution for actor-based applications.

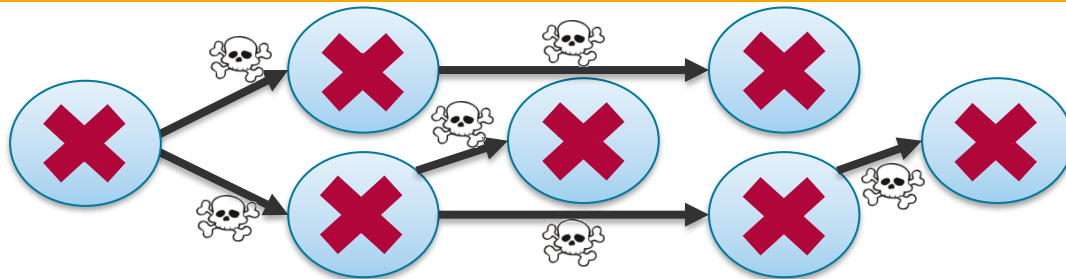
Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **173**

Patterns

Reaper



Reaper Pattern

- A pattern that allows an actor system to shutdown in a coordinated way.
- Idea (1/2):
 - **Start:** Once an actor recognizes that the system needs to shut down (final solution calculated, external shutdown request received, ...) it starts to spread some custom ShutdownMessage.
 - **Propagate:** An **application-specific shutdown protocol** must ensure that the custom ShutdownMessage spreads to all (relevant) actors.
 - In cluster setups, also spread the message into other systems!
 - **Stop:** Upon receiving a ShutdownMessage, an actor
 - stops itself (and children) in a coordinated way (rejecting further tasks, stopping the production of new tasks).
 - propagates the ShutdownMessage to other actors according to the shutdown protocol.



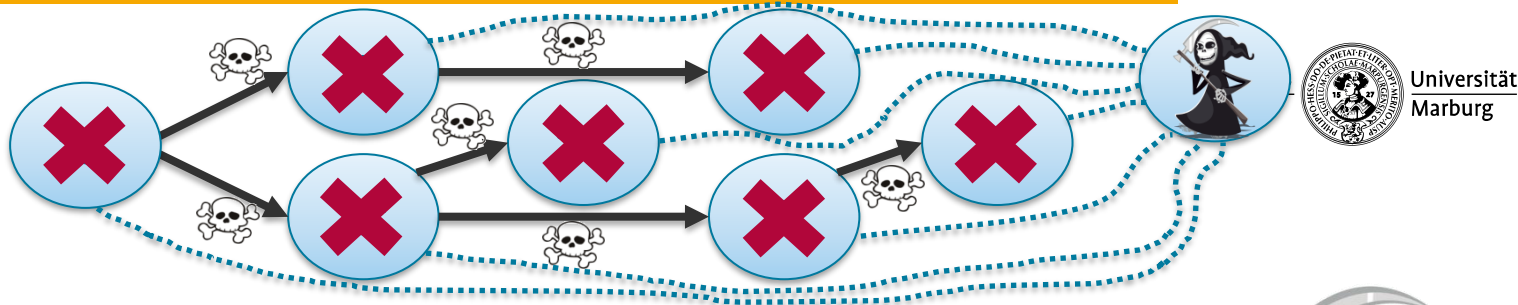
Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **174**

Patterns

Reaper



Reaper Patter

- A pattern that allows an actor system to shutdown in a coordinated way.
- Idea (2/2):

Reaper

- A dedicated actor that
 - “**knows**” all (relevant) actors in the actor system.

```
➤ if (this.watchees.add(actor))  
    this.getContext().watch(actor);
```

- “**reaps**” actor souls when they die.

```
➤ this.watchees.remove(terminatedSignal.getRef());
```

- “**ends the world**” if everyone is dead.

```
➤ if (this.watchees.isEmpty())  
    this.getContext().getSystem().terminate();
```



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **175**

```
public class Reaper extends AbstractBehavior<Reaper.Message> {
```

```
    public static <T> void watchWithDefaultReaper(ActorRef<T> actor) {  
        ReaperSingleton.get().tell(new WatchMeMessage(actor.unsafeUpcast()));  
    }
```

```
    public interface Message extends AkkaSerializable {  
    }
```

```
    @Getter @NoArgsConstructor @AllArgsConstructor  
    public static class WatchMeMessage implements Message {  
        private ActorRef<Void> actor;  
    }
```

```
    public static Behavior<Message> create() {  
        return Behaviors.setup(Reaper::new);  
    }
```

```
    private Reaper(ActorContext<Message> context) {  
        super(context);  
        ReaperSingleton.set(this.getContext().getSelf());  
    }
```

```
    private final Set<ActorRef<Void>> watchees =  
        new HashSet<>();
```



Can simply be called by any actor that needs to be watched!

@Override

```
    public Receive<Message> createReceive() {  
        return new ReceiveBuilder()  
            .onMessage(WatchMeMessage.class, this::handle)  
            .onSignal(Terminated.class, this::handle)  
            .build();  
    }
```

```
    private Behavior<Message> handle(WatchMeMessage m) {  
        if (this.watchees.add(m.getActor()))  
            this.getContext().watch(message.getActor());  
        return this;  
    }
```

```
    private Behavior<Message> handle(Terminated s) {  
        this.watchees.remove(s.getRef());  
        if (!this.watchees.isEmpty())  
            return this;  
        this.getContext().getSystem().terminate();  
        return Behaviors.stopped();  
    }
```


1. At startup:

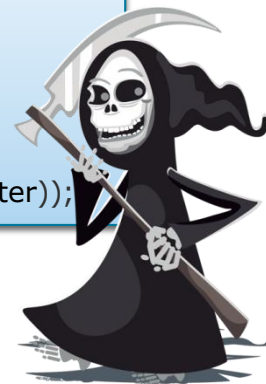
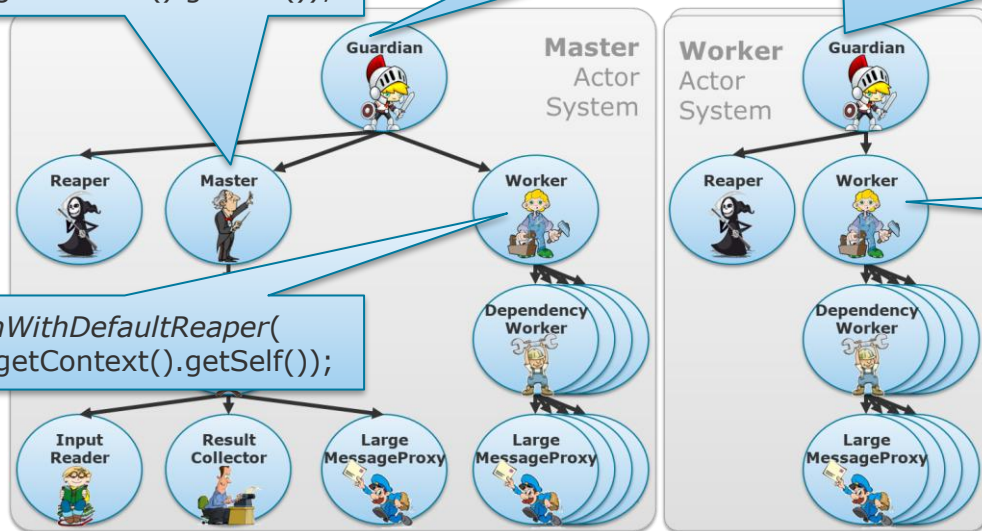
- All relevant actor register at their local reaper.
- All guardians register to each other.

```
Reaper.watchWithDefaultReaper(
    this.getContext().getSelf());
```

```
Reaper.watchWithDefaultReaper(
    this.getContext().getSelf());
```

```
context.getSystem().receptionist().tell(
    Receptionist.register(
        guardianService, context.getSelf()));

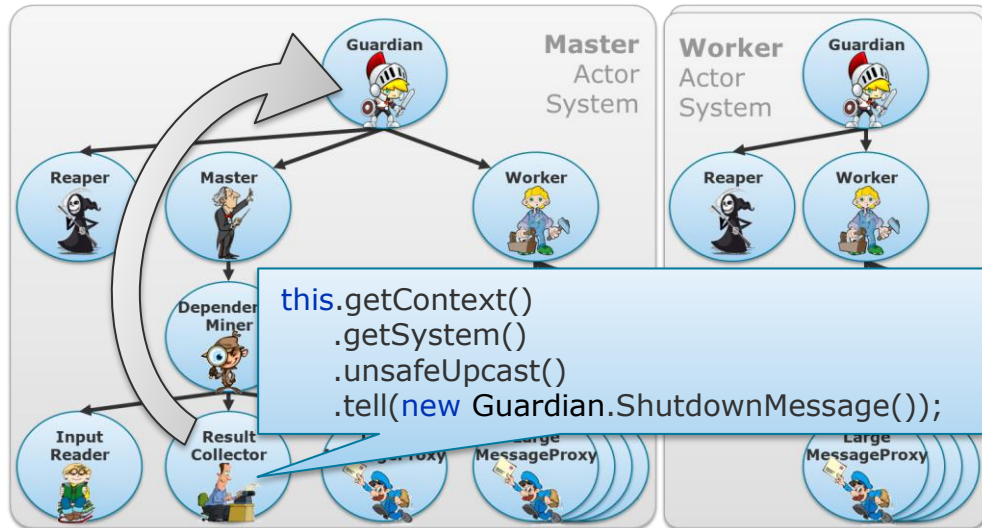
context.getSystem().receptionist().tell(
    Receptionist.subscribe(
        guardianService, listingResponseAdapter));
```



```
Reaper.watchWithDefaultReaper(
    this.getContext().getSelf());
```

Application-specific shutdown protocol – an example

2. Once some actor recognizes that the application is done (final solution calculated, external shutdown request received, ...) it sends a custom ShutdownMessage to the local UserGuardian actor.



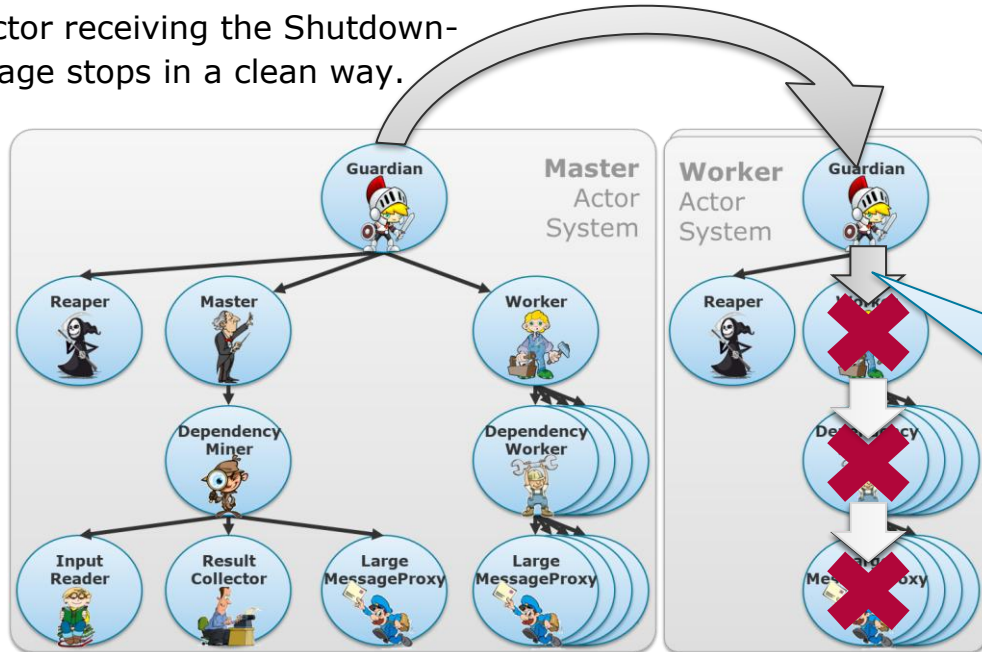
Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **178**

Application-specific shutdown protocol – an example

3. The guardian propagates the ShutdownMessage to all other guardians in the cluster.
4. If a guardian receives such an externally initiated Shutdown, it forwards the message.
5. An actor receiving the Shutdown-Message stops in a clean way.

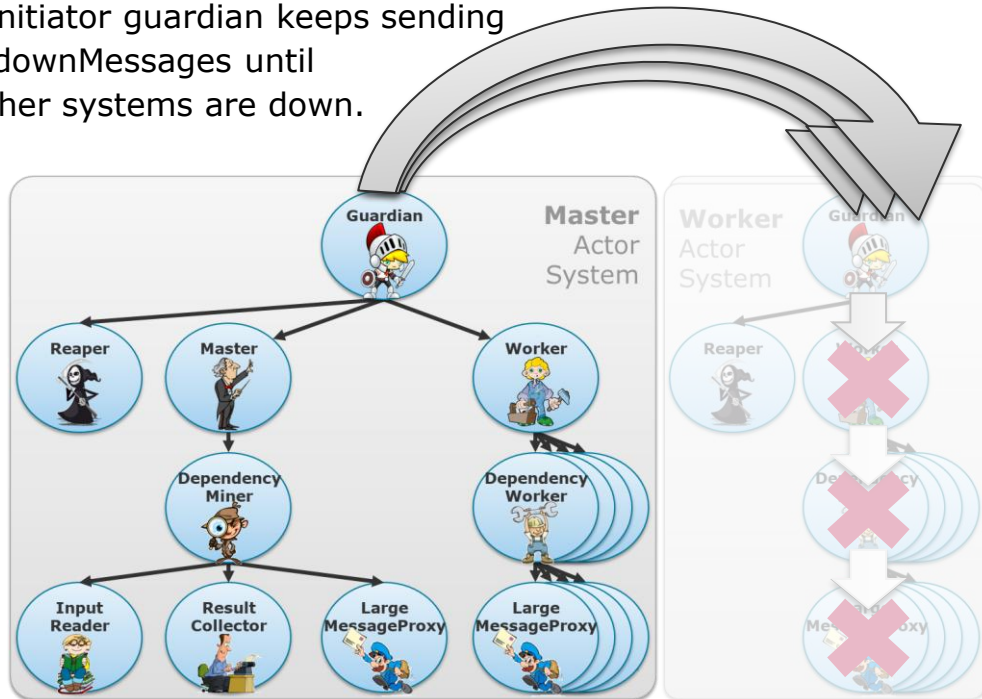


In our simple ddm-akka project, the workers stop directly, which also downs their children.

You might want a more sophisticated downing in your homework!

Application-specific shutdown protocol – an example

6. Once all local actors have been reaped, the reaper terminates the actor system.
7. The initiator guardian keeps sending ShutdownMessages until all other systems are down.



Big Data Systems

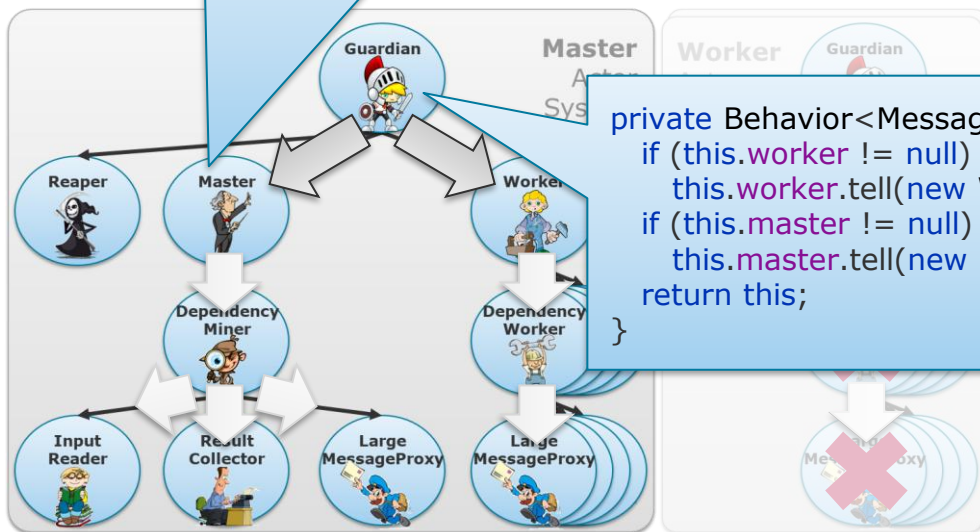
Akka Actor
Programming

ThorstenPapenbrock
Slide **180**

Application-specific shutdown protocol – an example

8. Now the initiator guardian propagates the ShutdownMessage locally.

Again: In our simple ddm-akka project, the worker and master stop directly, which also downs their children.

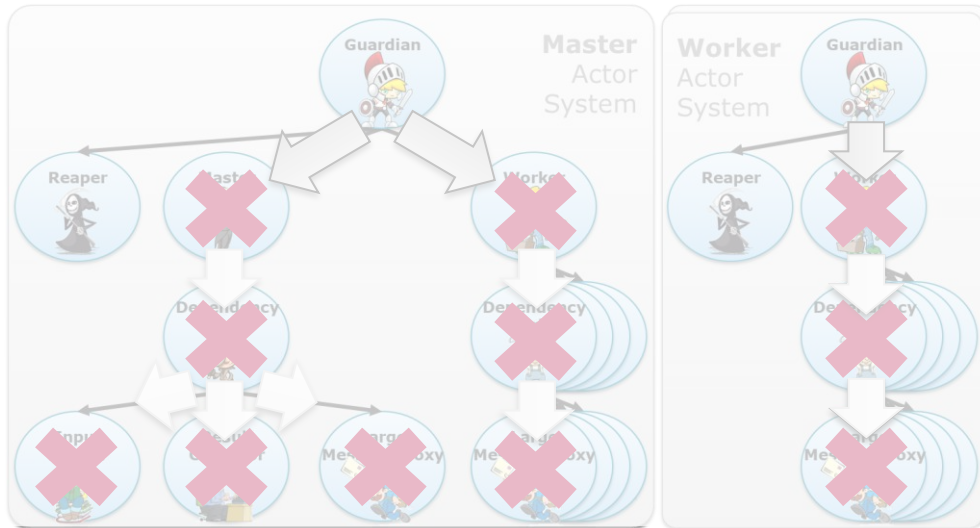


```
private Behavior<Message> handle(ShutdownMessage msg) {  
    if (this.worker != null)  
        this.worker.tell(new Worker.ShutdownMessage());  
    if (this.master != null)  
        this.master.tell(new Master.ShutdownMessage());  
    return this;  
}
```



Application-specific shutdown protocol – an example

9. An actor receiving the ShutdownMessage stops in a clean way.
10. Once all local actors have been reaped, the reaper terminates the actor system.



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **182**


```
public class Guardian extends AbstractBehavior<Guardian.Message> {
```

```
    public static class StartMessage implements Message ...
    public static class ShutdownMessage implements Message ...
    public static class ReceptionistListingMessage implements Message ...
```

Get timer functionality.

```
    public static Behavior<Message> create() {
        return Behaviors.setup(context -> Behaviors.withTimers(timers -> new Guardian(context, timers)));
    }
```

```
    private Guardian(ActorContext<Message> context, TimerScheduler<Message> timers) {
        super(context);
        this.timers = timers;
    }
```

Expose this guardian as a service.

...

```
    context.getSystem().receptionist().tell(Receptionist.register(guardianService, context.getSelf()));
```

```
    ActorRef<Receptionist.Listing> listingAdapter = context.messageAdapter(Receptionist.Listing.class, ReceptionistListingMessage::new);
    context.getSystem().receptionist().tell(Receptionist.subscribe(guardianService, listingAdapter));
}
```

```
private final TimerScheduler<Message> timers;
private Set<ActorRef<Message>> userGuardians = new HashSet<>();
```

Find other guardian services.

```
@Override
public Receive<Message> createReceive() {
    return newReceiveBuilder()
        .onMessage(StartMessage.class, this::handle)
        .onMessage(ShutdownMessage.class, this::handle)
        .onMessage(ReceptionistListingMessage.class, this::handle)
        .build();
}
```

```
private Behavior<Message> handle(StartMessage message) {
    if (this.master != null)
        this.master.tell(new Master.StartMessage());
    return this;
}
```



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **183**

```
private Behavior<Message> handle(ShutdownMessage message) {
    ActorRef<Message> self = this.getContext().getSelf();
```

```
    if ((message.getInitiator() != null) || this.isClusterDown()) {
        this.shutdown();
    } else {
```

```
        for (ActorRef<Message> userGuardian : this.userGuardians)
            if (!userGuardian.equals(self))
                userGuardian.tell(new ShutdownMessage(self));
```

```
        if (!this.timers.isTimerActive("ShutdownReattempt"))
            this.timers.startTimerAtFixedRate("ShutdownReattempt", message, Duration.ofSeconds(5), Duration.ofSeconds(5));
    }
    return this;
}
```

```
private boolean isClusterDown() {
    return this.userGuardians.isEmpty() || (this.userGuardians.contains(this.getContext().getSelf()) && this.userGuardians.size() == 1);
}
```

```
private void shutdown() {
    if (this.worker != null) {
        this.worker.tell(new Worker.ShutdownMessage());
        this.worker = null;
    }
    if (this.master != null) {
        this.master.tell(new Master.ShutdownMessage());
        this.master = null;
    }
}
```

```
private Behavior<Message> handle(ReceptionistListingMessage message) {
    this.userGuardians = message.getListing().getServiceInstances(Guardian.guardianService);
```

```
    if (this.timers.isTimerActive("ShutdownReattempt") && this.isClusterDown())
        this.shutdown();

    return this;
}
```

Propagation protocol essentials

If shutdown was initiated somewhere else or no other ActorSystems exist, shutdown this system immediately.

Otherwise, propagate the ShutdownMessage into other systems.

Reminder message to resend Shutdown if in 5 sec still other guardians exist.

Propagate shutdown to children; reaper will terminate the system once they all stopped.

Propagate shutdown locally if all other guardians, i.e., ActorSystems are gone.



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **184**

The diagram illustrates a family tree structure for a user guardian. At the top is a purple box labeled "/user Guardian". Below it, a horizontal line separates the "Parent / Child" section from the "DeathWatch" section. The "Parent / Child" section shows three children: A (blue circle), D (green circle), and G (blue circle). Each child has two children of their own: A has B (green circle) and C (green circle); D has E (blue circle) and F (blue circle); G has H (green circle). The "DeathWatch" section shows a red circle labeled "The Reaper". Dashed lines connect the children to "The Reaper": A to B, C, and D; D to E, F, and G; and G to H. The diagram is part of a presentation slide from Philipps-Universität Marburg, featuring the university's logo and name in the top right corner.



Akka Actor Programming

See: <http://letitcrash.com/post/30165507578/shutdown-patterns-in-akka-2>

Patterns

Further Reading

Akka documentation

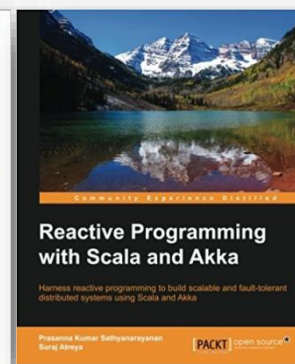
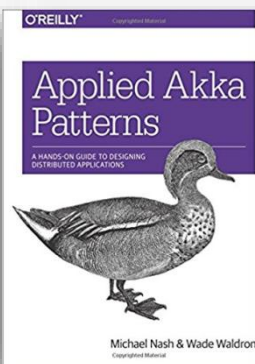
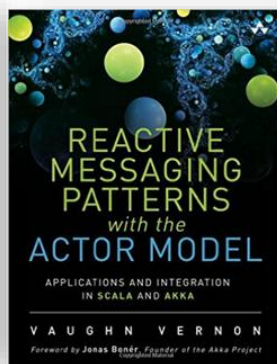
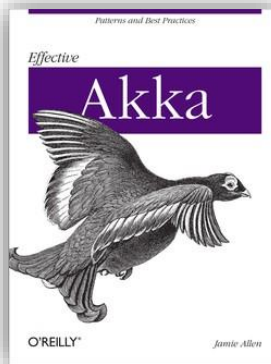
- <http://doc.akka.io/docs/akka/current/java/index.html>
- <http://doc.akka.io/docs/akka/current/scala/index.html>

Example code @ GitHub:
<https://github.com/akka/akka-samples>

Experiences, best practices, and patterns

- <http://letitcrash.com>
- <http://akka.io/blog>
- <https://github.com/sksamuel/akka-patterns>

- Akka actor programming literature:



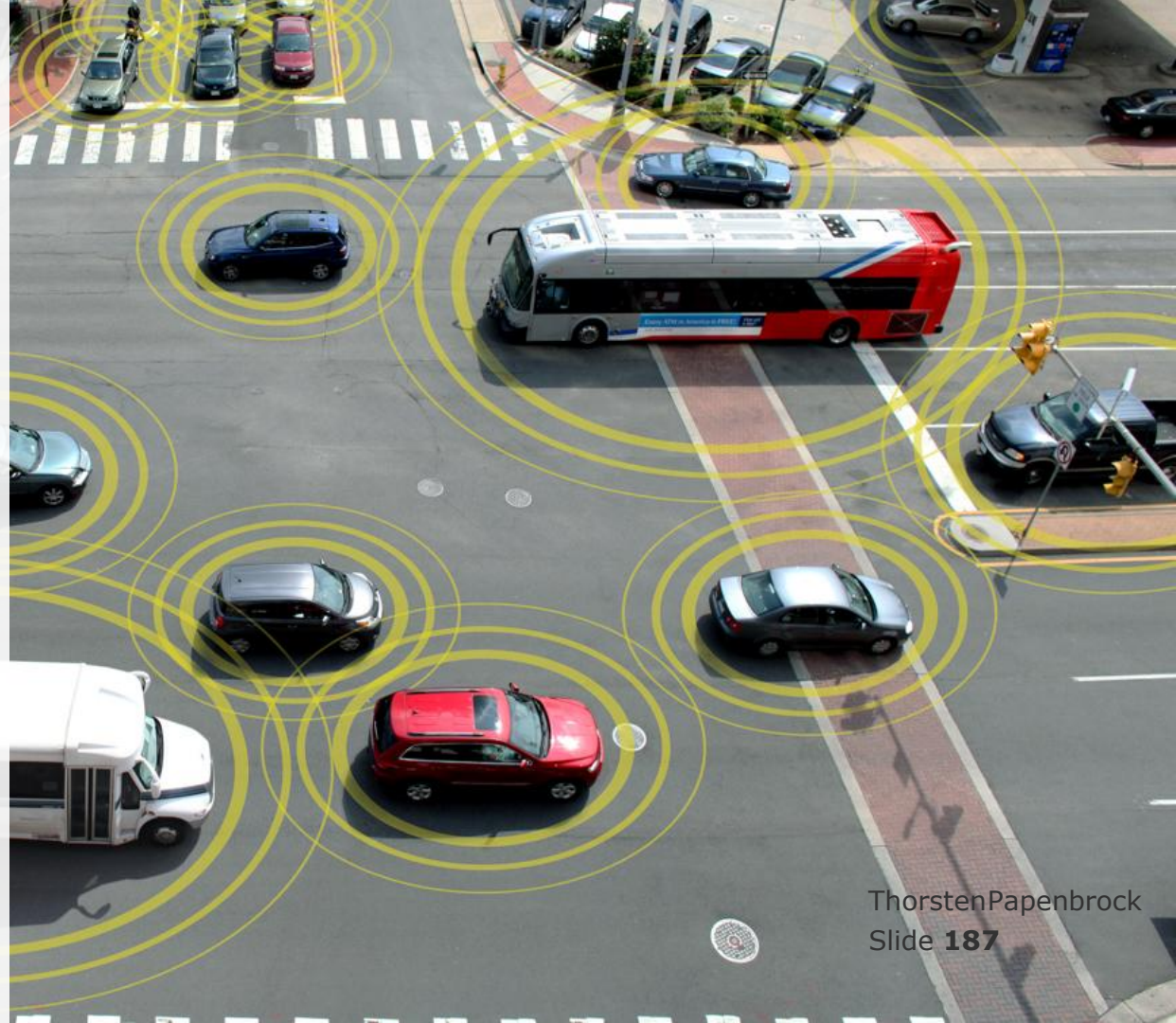
Big Data Systems

Akka Actor Programming

ThorstenPapenbrock
Slide **186**

Akka Actor Programming Hands-on

- Actor Model
- Basic Concepts
- Runtime Architecture
- Demo
- Messaging
- Parallelization
- Remoting
- Clustering
- Patterns
- **Homework**



 ddm-akka Public

✎ Edit Pins

👁 Unwatch 1

🍴 Fork 13

☆ Star 3

👤 main

🌿 2 Branches

🏷 0 Tags

🔍 Go to file

t

➕ Add file

<> Code

 thorsten-papenbrock Update README.md	053ad80 · 3 hours ago	🕒 15 Commits
📁 src	Added a flag for the enterShutdown, increased the logger st...	9 months ago
📄 .gitignore	Updated gitignore to ignore the solution files	8 months ago
📄 LICENSE	Migrated the ddm-akka projekt into this new repository.	last year
📄 README.md	Update README.md	3 hours ago
📄 pom.xml	Update libs and extend readme for token usage of akka	4 days ago

📖 README

📄 MIT license

✎

☰

ddm-akka

Akka example and homework code for the "Distributed Data Management" lecture.

Requirements

- Java Version >= 21
- Maven Compiler Version >= 3.14.1

Getting started

1. Clone repo

```
git clone https://github.com/UMR-Big-Data-Analytics/ddm-akka.git
```

2. Get the Akka libraries

About

Akka example and homework code for the "Distributed Data Management" lecture.

📖 README

📄 MIT license

🔄 Activity

📖 Custom properties

☆ 3 stars

👁 1 watching

🍴 13 forks

Report repository

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

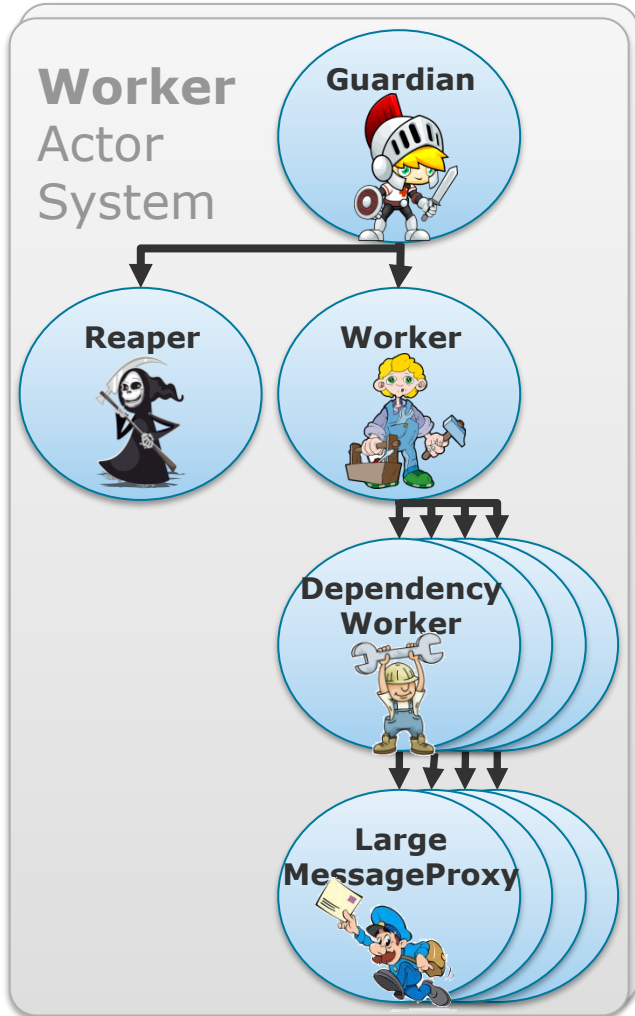
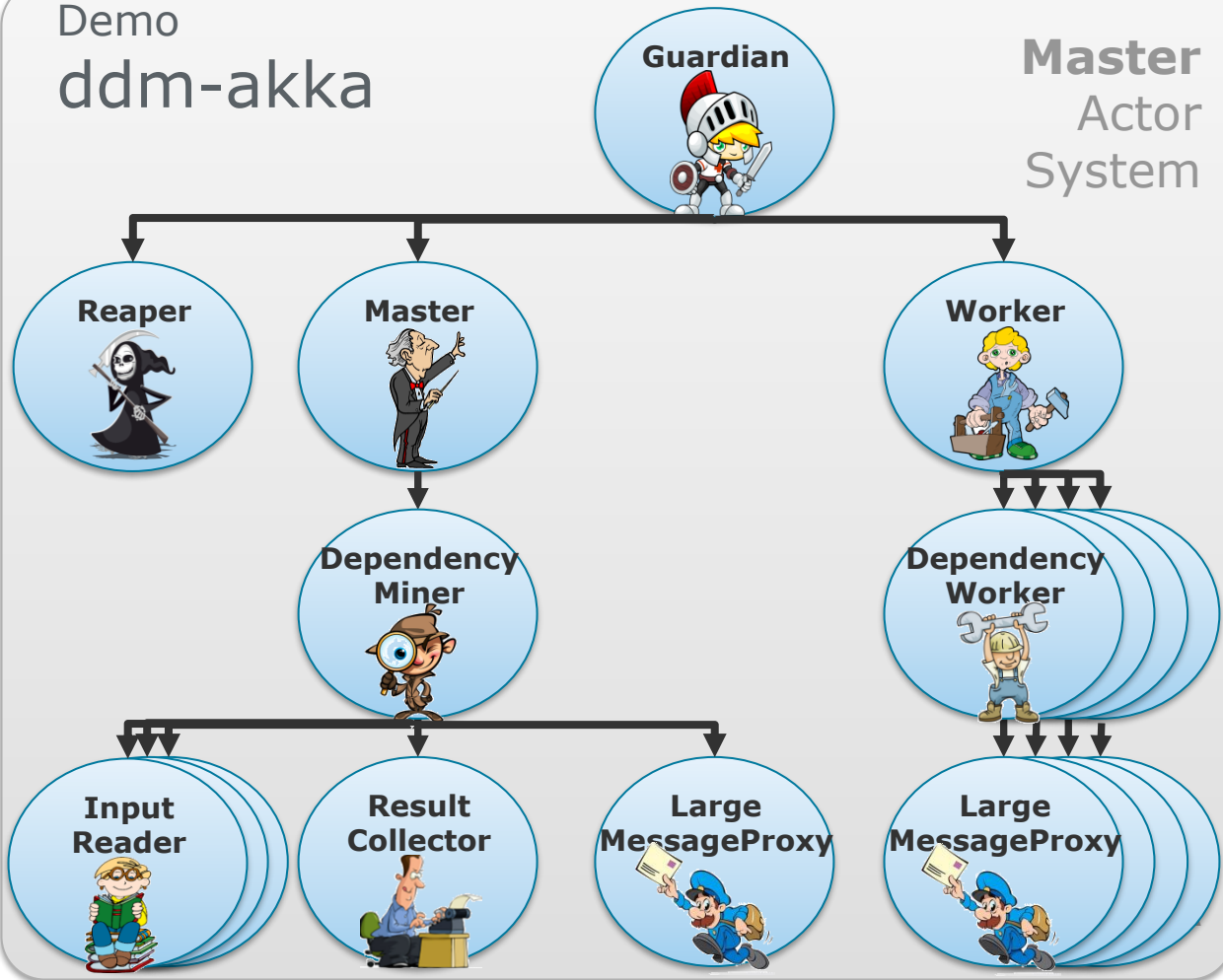
Contributors 2

 **thorsten-papenbrock** Thorsten Pape...

 **SeegerM** Seeger

<https://github.com/UMR-Big-Data-Analytics/ddm-akka>

Demo ddm-akka



Demo ddm-akka

Guardian

Master
Actor System

Reaper

Master

Worker

Solving the homework
requires modifying at
least these actors!

Dependency
Miner

Dependency
Worker

Input
Reader

Result
Collector

Large
MessageProxy

Large
MessageProxy

Worker
Actor System

Guardian

Reaper

Worker

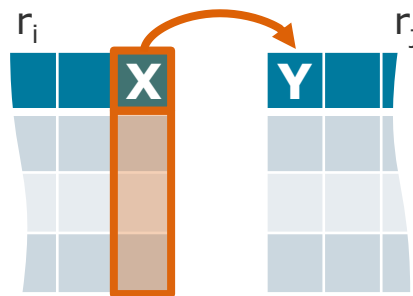
Dependency
Worker

Large
MessageProxy

Definition: Given two relational instances r_i and r_j for the schemata R_i and R_j , respectively. The **inclusion dependency (IND)** $R_i[X] \subseteq R_j[Y]$ (short $X \subseteq Y$) with $X \subseteq R_i$, $Y \subseteq R_j$ and $|X| = |Y|$ is valid, iff $\forall t_i[X] \in r_i, \exists t_j[Y] \in r_j : t_i[X] = t_j[Y]$.

“All values in X are also contained in Y ”

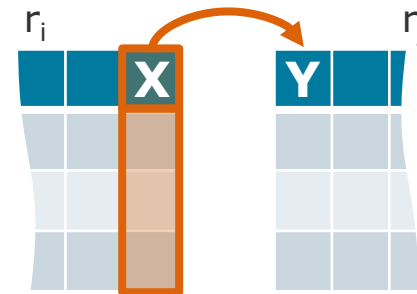
foreign-key candidates



A **unary inclusion dependency** $X \subseteq Y$ is an IND with $|X| = |Y| = 1$.
An **n-ary inclusion dependency** $X \subseteq Y$ is an IND with $|X| = |Y| > 1$.

Axioms

- Reflexivity: If $i = j$ and $X = Y$ then $R_i[X] \subseteq R_j[Y]$.
- Permutation: If $R_i[A_1, \dots, A_n] \subseteq R_j[B_1, \dots, B_n]$ then $R_i[A_{\sigma 1}, \dots, A_{\sigma n}] \subseteq R_j[B_{\sigma 1}, \dots, B_{\sigma n}]$ for each sequence $\sigma 1, \dots, \sigma n$ of distinct integers $\{1, \dots, m\}$.
- Transitivity: If $R_i[X] \subseteq R_j[Y]$ and $R_j[Y] \subseteq R_k[Z]$ then $R_i[X] \subseteq R_k[Z]$.

**Standardization**

- Permutation axiom: Applying the same permutation σ to the both attribute lists of an IND always results in the technically same IND:
e.g. $\{A, B, C\} \subseteq \{D, E, F\} = \{B, C, A\} \subseteq \{E, F, D\} = \{A, B, C\} \subseteq \{D, E, F\} = \dots$
- We define that the **dependent attributes shall be sorted by the index** in their schema, which avoids duplicate outputs in the result sets.

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **192**

Homework

Inclusion Dependency Discovery

Name	Type	Equatorial diameter	Mass	Orbital radius	Orbital period	Rotation period	Confirmed moons	Rings	Atmosphere
Mercury	Terrestrial	0.382	0.06	0.47	0.24	58.64	0	no	minimal
Venus	Terrestrial	0.949	0.82	0.72	0.62	-243.02	0	no	CO ₂ , N ₂
Earth	Terrestrial	1.000	1.00	1.00	1.00	1.00	1	no	N ₂ , O ₂ , Ar
Mars	Terrestrial	0.532	0.11	1.52	1.88	1.03	2	no	CO ₂ , N ₂ , Ar
Jupiter	Giant	11.209	317.8	5.20	11.86	0.41	67	yes	H ₂ , He
Saturn	Giant	9.449	95.2	9.54	29.46	0.43	62	yes	H ₂ , He
Uranus	Giant	4.007	14.6	19.22	84.01	-0.72	27	yes	H ₂ , He
Neptune	Giant	3.883	17.2	30.06	164.8	0.67	14	yes	H ₂ , He

Complexity: $O(n^2-n)$
for n attributes

Example:
10 attr ~ 90 checks
1,000 attr ~ 999,000 checks

- Name $\subseteq$ Type ?
- Name $\subseteq$ Equatorial_diameter ?
- Name $\subseteq$ Mass ?
- Name $\subseteq$ Orbital_radius ?
- Name $\subseteq$ Orbital_period ?
- Name $\subseteq$ Rotation_period ?
- Name $\subseteq$ Confirmed_moons ?
- Name $\subseteq$ Rings ?
- Name $\subseteq$ Atmosphere ?
- Type $\subseteq$ Name ?
- Type $\subseteq$ Equatorial_diameter ?
- Type $\subseteq$ Mass ?
- Type $\subseteq$ Orbital_radius ?
- Type $\subseteq$ Orbital_period ?
- Type $\subseteq$ Rotation_period ?
- Type $\subseteq$ Confirmed_moons ?
- Type $\subseteq$ Rings ?
- Type $\subseteq$ Atmosphere ?
- Mass $\subseteq$ Name ?
- Mass $\subseteq$ Type ?
- Mass $\subseteq$ Equatorial_diameter ?
- ...

Homework

Inclusion Dependency Discovery

Name	Type	Equatorial diameter	Mass	Orbital radius	Orbital period	Rotation period	Confirmed moons	Rings	Atmosphere	Planet	Rotation Period	Revolution Period	Symbol	Unicode	Glyph
Mercury	Terrestrial	0.382	0.06	0.47	0.24	58.64	0	no	minimal	Mercury	58.6 days	87.97 days		U+2609	☿
Venus	Terrestrial	0.949	0.82	0.72	0.62	-243.02	0	no	CO ₂ , N ₂	Venus	243 days	224.7 days		U+263D	♀
Earth	Terrestrial	1.000	1.00	1.00	1.00	1.00	1	no	N ₂ , O ₂ , Ar	Earth	0.99 days	365.26 days		U+263E	♁
Mars	Terrestrial	0.532	0.11	1.52	1.88	1.03	2	no	CO ₂ , N ₂ , Ar	Mars	1.03 days	1.88 years		U+2640	♂
Jupiter	Giant	11.209	317.8	5.20	11.86	0.41	67	yes	H ₂ , He	Jupiter	0.41 days	11.86 years		U+2642	♃
Saturn	Giant	9.449	95.2	9.54	29.46	0.43	62	yes	H ₂ , He	Saturn	0.45 days	29.46 years		U+2643	♄
Uranus	Giant	4.007	14.6	19.22	84.01	-0.72	27	yes	H ₂ , He	Uranus	0.72 days	84.01 years		U+2644	♅
Neptune	Giant	3.883	17.2	30.06	164.8	0.67	14	yes	H ₂ , He	Neptune	0.67 days	164.79 years		U+2645	♆
										Pluto	6.39 days	248.59 years		U+2646	♇

Planet	Synodic period	Synodic period (mean)	Days in retrograde
Mercury	116	3.8	~21
Venus	584	19.2	41
Mars	780	25.6	72
Jupiter	399	13.1	121
Saturn	378	12.4	138
Uranus	370	12.15	151
Neptune	367	12.07	158

Planet	Mean distance	Relative mean distance
Mercury	57.91	1
Venus	108.21	1.86859
Earth	149.6	1.3825
Mars	227.92	1.52353
Ceres	413.79	1.81552
Jupiter	778.57	1.88154
Saturn	1,433.53	1.84123
Uranus	2,872.46	2.00377
Neptune	4,495.06	1.56488
Pluto	5,869.66	1.3058

Planet	Calculated (in AU)	Observed (in AU)	Perfect octaves	Actual distance
Mercury	0.4	0.387	0	0
Venus	0.7	0.723	1	1
Earth	1	1	2	2
Mars	1.6	1.524	4	3.7
Asteroid belt	2.8	2.767	8	7.8
Jupiter	5.2	5.203	16	15.7
Saturn	10	9.539	32	29.9
Uranus	19.6	19.191	64	61.4
Neptune	38.8	30.061	96	-96.8
Pluto	77.2	39.529	128	127.7

Sign	House	Domicile	Detriment	Exaltation	Fall	Planetary Joy
Aries	1st House	Mars	Venus	Sun	Saturn	Mercury
Taurus	2nd House	Venus	Pluto	Moon	Uranus	Jupiter
Gemini	3rd House	Mercury	Jupiter	N/A	N/A	Saturn
Cancer	4th House	Moon	Saturn	Jupiter	Mars	Venus
Leo	5th House	Sun	Uranus	Neptune	Mercury	Mars
Virgo	6th House	Mercury	Neptune	Pluto, Mercury	Venus	Saturn
Libra	7th House	Venus	Mars	Saturn	Sun	Moon
Scorpio	8th House	Pluto	Venus	Uranus	Moon	Saturn
Sagittarius	9th House	Jupiter	Mercury	N/A	N/A	Sun
Capricorn	10th House	Saturn	Moon	Mars	Jupiter	Mercury
Aquarius	11th House	Uranus	Sun	Mercury	Neptune	Venus
Pisces	12th House	Neptune	Mercury	Venus	Pluto, Mercury	Moon

Symbol	Unicode	Glyph
	U+2609	☉
	U+263D	☾
	U+263E	☾
	U+263F	☿
	U+2640	♀
	U+1F728	🌍
	U+2642	♂
	U+2643	♃
	U+2644	♄
	U+2645	♅
	U+26E2	♅
	U+2646	♆
	≈ U+2641	♁
	≈ U+29EC	♁
	U+2647	♇
	not present	♇
	U+2648	♈
	U+2649	♉
	U+264A	♊
	U+264B	♋
	U+264C	♌
	U+264D	♍
	U+264E	♎
	U+264F	♏
	U+2650	♐
	U+2651	♑
	U+2651	♑
	U+2652	♒
	U+2653	♓
	U+260C	☌
...	...	...

Thorston

Slide :

<http://webdatacommons.org/>

Homework

Inclusion Dependency Discovery

unary inclusion dependency

Book

Title	Author	Price	Pages	Published
Database Systems	Ullman	214	1203	2007
Algorithms in Java	Sedgewick	130	768	2002
3D Computer Graphics	Watt	20	570	1999

Lending

ID	Name	Location	Student	Course
42	Database Systems	A-1.2	Miller	DBS 1
88	Database Systems	B-2.2	Miller	PT 1
73	Database Systems	A-1.2	Smith	DPDC
69	Algorithms in Java	C-E.1	Miller	PT 1
13	Algorithms in Java	C-E.1	Smith	DPDC

$\text{Name} \subseteq \text{Title}$

n-ary inclusion dependency

Student

Name	Lecture	Credit	Semester	Verified
Miller	DBS 1	20	2	false
Miller	PT 1	15	2	false
Smith	DPDC	10	6	true

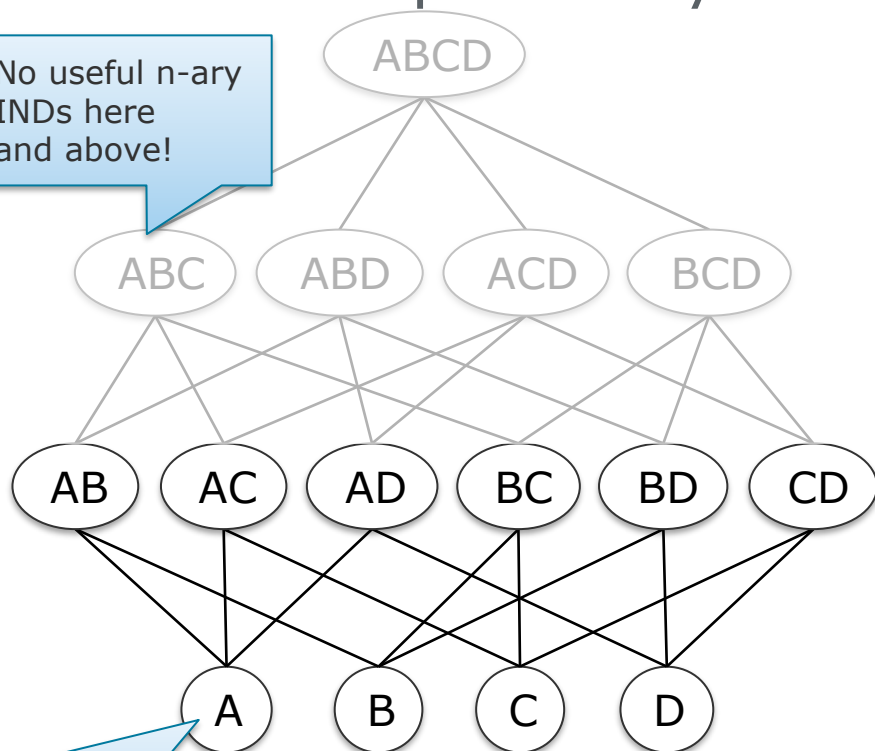
Lending

ID	Name	Location	Student	Course
42	Database Systems	A-1.2	Miller	DBS 1
88	Database Systems	B-2.2	Miller	PT 1
73	Database Systems	A-1.2	Smith	DPDC
69	Algorithms in Java	C-E.1	Miller	PT 1
13	Algorithms in Java	C-E.1	Smith	DPDC

$\text{Student, Course} \subseteq \text{Name, Lecture}$

Inclusion Dependency Discovery

No useful n-ary
INDs here
and above!



Test every combination
with all other combina-
tions of same size!

$$AB \subseteq CD$$

$$AC \subseteq BD$$

$$AD \subseteq BC$$

$$BC \subseteq AD$$

$$BD \subseteq AC$$

$$CD \subseteq AB$$

$$AB \subseteq DC$$

$$AC \subseteq DB$$

$$AD \subseteq CB$$

$$BC \subseteq DA$$

$$BD \subseteq CA$$

$$CD \subseteq BA$$

$$A \subseteq B$$

$$A \subseteq C$$

$$A \subseteq D$$

$$B \subseteq A$$

$$B \subseteq C$$

$$B \subseteq D$$

$$C \subseteq A$$

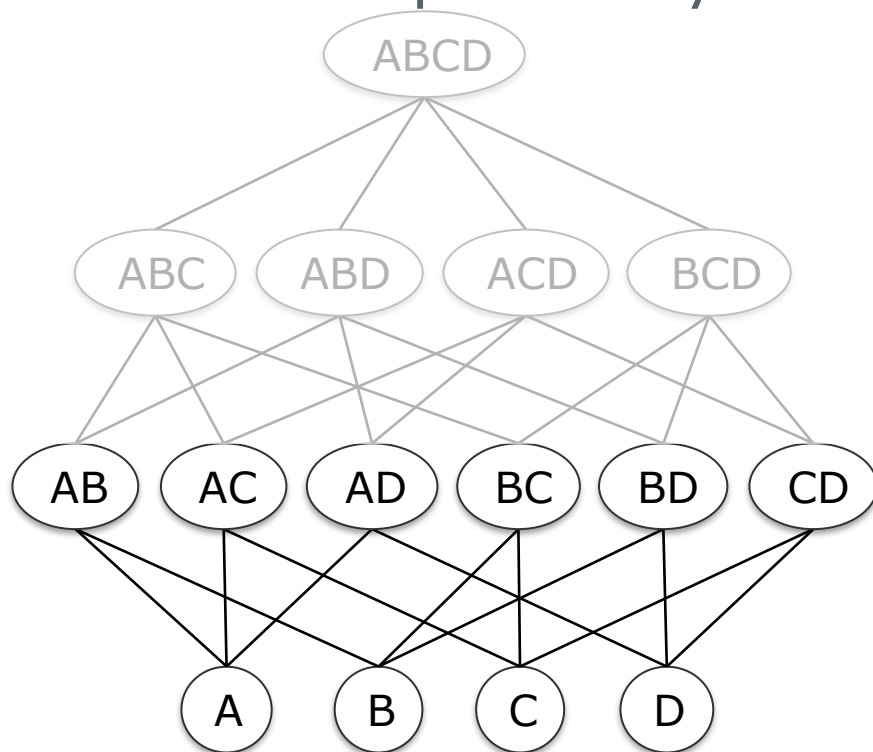
$$C \subseteq B$$

$$C \subseteq D$$

$$D \subseteq A$$

$$D \subseteq B$$

$$D \subseteq C$$



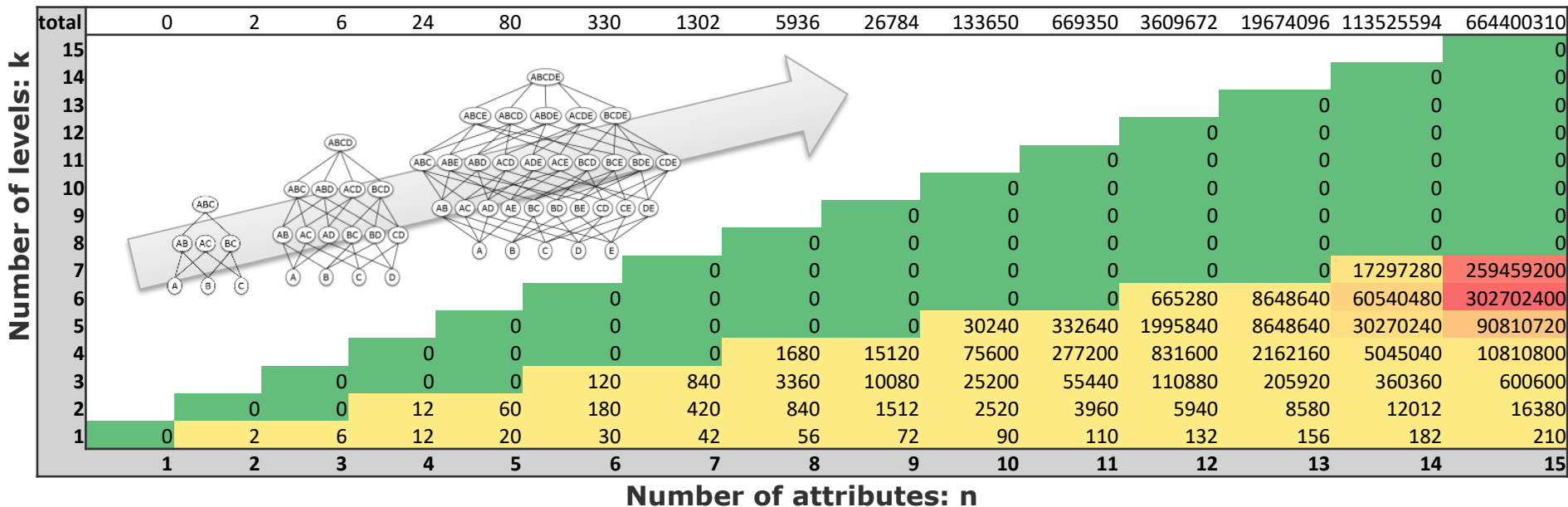
Pruning

- **Upwards:**
Generate $\{A, B\} \subseteq \{C, D\}$ only if $\{A\} \subseteq \{C\}$ and $\{B\} \subseteq \{D\}$ are both true!
- **Downwards:**
If $\{A, B\} \subseteq \{C, D\}$, then $\{A\} \subseteq \{C\}$ and $\{B\} \subseteq \{D\}$ must also be true.
- **Same level:**
If $\{A\} \subseteq \{B\}$ and $\{B\} \subseteq \{C\}$, then $\{A\} \subseteq \{C\}$ must also be true.

Homework

Inclusion Dependency Discovery

IND candidates



Unary IND detection:

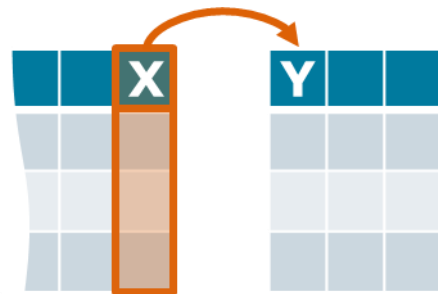
$O(n^2)$ for n attributes

N-ary IND detection:

$O(2^n \cdot n!)$ for n attributes

Assignment

- Complete the akka-ddm project such that it **discovers all unary INDs** in the input datasets (within and between all tables!).
- The input dataset may consist of multiple tables.
- The solution should run distributed and exploit all available resources.
- Optional bonus task: If you seek for a challenge, also implement the “hardMode” and **discover all n-ary INDs** as well.

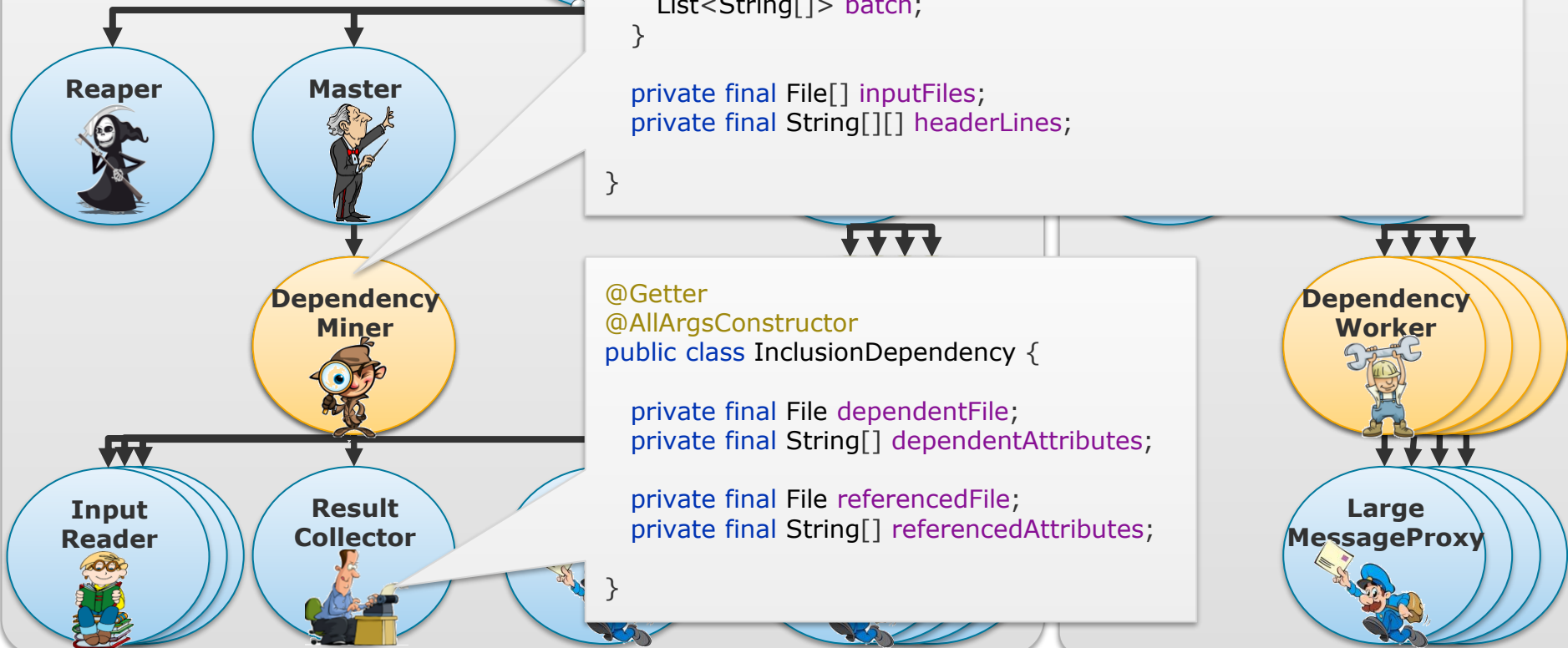


Dataset

- Use the TPCB dataset provided in the git repository and on our Ilias website.
- TPCB is a generated dataset often used to benchmark query performance.

www.tpc.org/tpch

Demo ddm-akka





News



Folders



To me



Recycle Bin



Links



Clients



Help

Home / Data



5



+ Name	Size	Modified by	Modified	
 MusicBrainz.zip	476.8 MB	Thorsten Papenbrock	1 minute ago	
 TPC.H.zip	92.9 MB	Thorsten Papenbrock	Just now	
 WDC.zip	8.7 KB	Thorsten Papenbrock	Just now	

<https://next.hessenbox.de/index.php/s/6zL45EzYnc6ZTyG>

Data

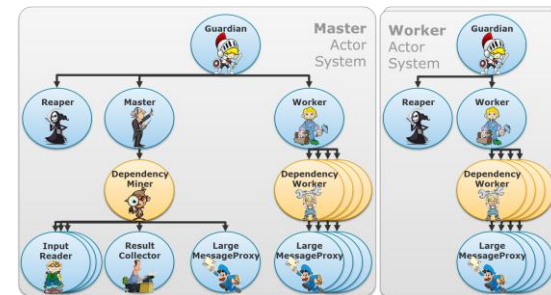
- 4 datasets of different sizes
- Algorithm needs to discover all inter- and intra-table INDs
- Datasets might not fit into the main memory of one machine
- Unzip the datasets into **ddm-akka/data** to process them

One node cannot store the entire dataset alone!

- The data needs to be sharded while it is being read.
- If the memory resources of all nodes are exhausted, wait for more worker nodes.

The data needs to be distributed, but the IND checks need to compare every pair of columns eventually!

- Your algorithm might need to move data on demand.
- Or the algorithm needs to replicate data.



Big Data Systems

Akka Actor
Programming

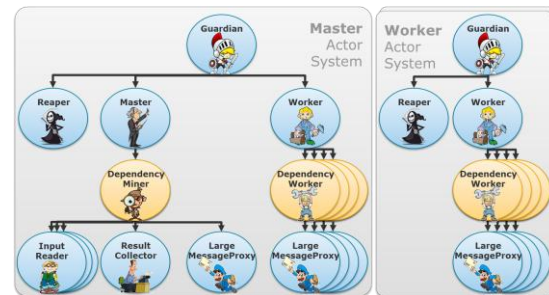
ThorstenPapenbrock
Slide **202**

What to change in the code?

- The main class should not send a shutdown on key-press, i.e. comment these lines (they are used for demo only):

```
// waitForInput(">>> Press ENTER to exit <<<");  
// guardian.tell(new Guardian.ShutdownMessage());
```

- Use the DependencyMiner and DependencyWorker to implement the IND discovery. Remove all code that generated random results or pretends workload.
 - Recommendation: Master/Worker pattern ;-)
- Somehow, you need to (intelligently) transfer data to the DependencyWorkers so that they can participate in the discovery.
 - Options: LargeMessageProxys, LargeMessageChannels, Message Batching, ...



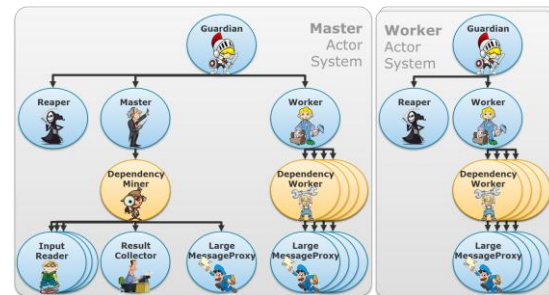
Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **203**

What **not to change** in the code?

- Do not change the time measurement:
It should start with the StartMessage and end when the DependencyMiner ends the discovery with the FinalizeMessage.
- Do not change:
 - the application name or packaging.
 - the application command-line interface
(i.e. the jcommander parts).
- Do not write anything else than the results to disk!
 - Distribute the data and do not spill it (temporarily) to disk.
 - Let the ResultCollector manage your outputs.



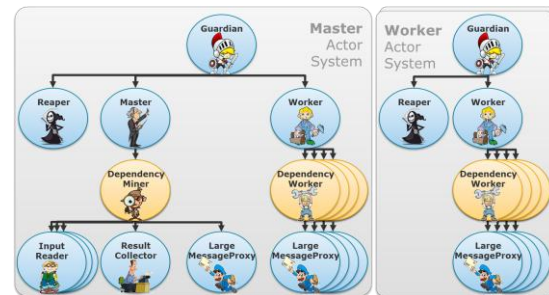
Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **204**

What **probably/maybe to change** in the code?

- Add more actors, such as
 - More (specialized) workers
 - Specialized child actors for the DependencyMiner
 - DataHolder or DataProvider actors for the sharding
 - ...
- Improve the shutdown protocol for a more sophisticated downing by e.g. forwarding the ShutdownMessage to children of the Master and Worker actors.
- Implement the "hardMode" and discover n-ary INDs when the flag is set.
- Optimize the application.conf to your needs.



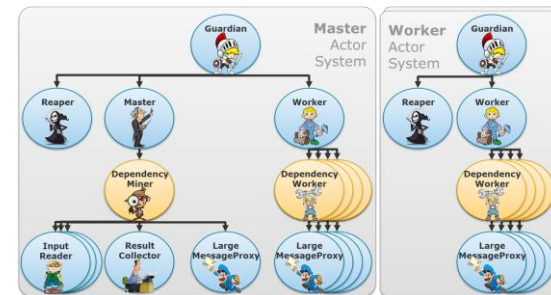
Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **205**

Challenges of the homework

1. Understand the available code and protocols.
2. Find an effective strategy and protocol for
 - data representation
(e.g. column vs. row)
 - data distribution
(e.g. dynamic vs. clever partitioning).
3. Find a clever candidate generation technique
(e.g. exploit pruning?).
4. Find an efficient strategy for candidate validation
(e.g. `valuesA.containsAll(valuesB)` might not be the fastest approach).



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **206**

Tasks

1. Form teams of **two** students and create a **private** GitHub repository according to the instructions at the beginning of this slide deck.
2. Follow the README.md instructions to **build**, **understand** and **test** the ddm-akka project; it does not do anything meaningful, yet, but the file reading, cluster setup and shutdown should already work.
3. Solve the homework by implementing the distributed IND discovery:
 - Discover all inter- and intra-table unary INDs without exhausting the memory!
4. Test your code:
 - On one machine by starting multiple local processes and on two machines by connecting your two computers.
5. Document your code:
 - Create (one or two) PowerPoint slides to present your solution and commit the slides into your repository.

Inclusion Dependency Discovery in Akka

Criteria to pass the assignment

1. Code does not contain plagiarism.
 2. Code compiles.
 3. Algorithm discovers all inter- and intra-table unary INDs correctly for any given relational input dataset.
 4. Algorithm can process datasets that do not fit into the memory of a single cluster node.
 1. Uses distribution
 2. Reads data only once
- Algorithm can process all three provided datasets on the PI cluster.

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **208**

DEADLINES:

- For your repository submission:

15.11.2025 23:59:00

- For your final Akka homework:

02.01.2025 23:59:00

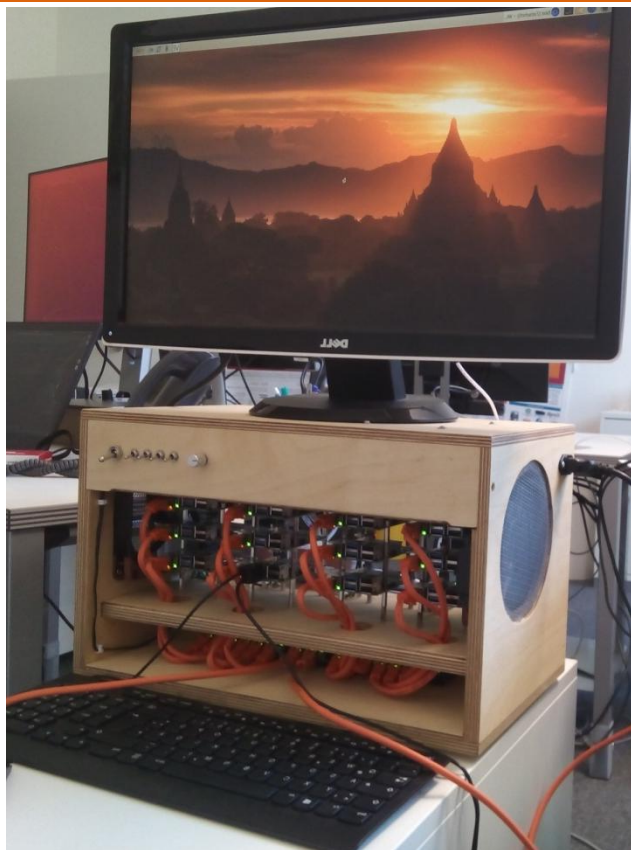
Note:
Solving the homework
is a **requirement** to be
permitted to the exam!

Workshop Days:

Date: Wednesdays

Time: 8:00 – 10:00

Room: 04C22a



Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **210**



Homework

Final Hints

Stick to established patterns!

- Consider the patterns we learned: master/worker, proxy, router, reaper, singleton, reactive waiting, receptionist and large message proxy.

Not everything is an actor!

- Use actors where concurrency is expected; basic OO otherwise.

Use custom dispatchers!

- Isolate long running actors on dispatchers with limited amounts of threads; otherwise they starve other actors for threads.

Expect change/elasticity!

- Prepare for late joining and exiting actor systems.

Avoid mutable states in messages!

- Check that the payload of all messages is immutable (or treated as if it was) (e.g. sending mutable Java Maps lets the target actor see mutable state!).

Big Data Systems

Akka Actor
Programming

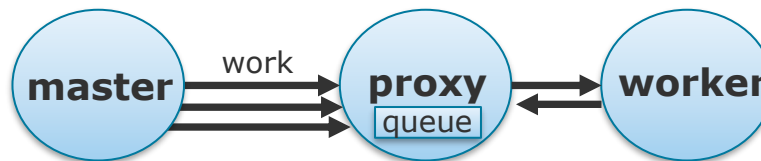
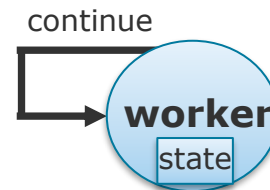
ThorstenPapenbrock
Slide **213**

Homework

Final Hints

Stay reactive!

- Actors that work on long running tasks are not reactive to e.g.
 - shutdown messages
 - pruning information
 - further work packages or requests
- Pause work frequently
 - Send a "ContinueMessage" to yourself to check your own mailbox.



- Organize help
 - Setup proxy actors to manage/queue your inbox while working long.

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **214**

Homework

Final Hints

Pruned work beats parallelized work!

- Do not waste energy by parallelizing subtasks blindly.
- Consider your pruning potential and react on partial results to win the game!

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **215**

Consider batching of messages!

- Group tasks into reasonable sized work packages.
 - Not too small (due to high messaging overhead)
 - Not too large (due to non-reactive workers)

Consider breaking of messages!

- Split large messages into reasonably sized sub-messages.
- Large refers to the message size and message computation costs.
 - Too large messages cannot be transferred (see LargeMessageProxy)
 - Too computation intensive messages make actor unresponsive (see task- and data-parallelism)

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **216**

Homework

Final Hints

Measure, log, record and test!

- Distributed systems are difficult to **debug** (no stop-the-world breakpoints)
 - **Actor hierarchies** for failure debugging
 - **Unit tests** for actors and code modules (see Akka's `TestKit`)
- Distributed systems are hard to **profile** (limited use of JVisualVM)
 - **Metrics collection** for runtime profiling (see `ClusterMetricsExtension`)
 - **Logging** of important messages and events (see `LoggingAdapter`)

Avoid premature optimization!

- Start simple and test correctness first.
- Then, carefully identify bottlenecks and where necessary:
 - Add concurrency e.g. via routers or further child actors.

Big Data Systems

Akka Actor
Programming

ThorstenPapenbrock
Slide **217**

Homework

Final Hints

Dear BDS students,

today was our first workshop and we collected some common mistakes and general hints that I would like to share with you:

- 1.) Remove the two lines in the Main class that let the main thread wait for user input and send a Shutdown message! As the task description says, there are two lines in the Main Class that are only for showcasing the algorithm frame. You have to send the Shutdown message in your algorithm yourself when the algorithm is done. ActorSystems to not terminate automatically - You have to terminate them when your computation is done!
- 2) You are not allowed to use the disk! Send data to Worker nodes and store partitions of the data in RAM there. You can go for row-wise partitioning or column-wise partitioning. If row-wise partitioning is easier for you to implement, you can do so and later change it to column-wise partitioning.
- 3) Your first concern should be to load the data into the cluster. Because Worker ActorSystems join gradually, you do not know in the beginning how many Worker actors you will ultimately get. Find a loading strategy that refines the partitioning during the loading process. Our slide deck on "Partitioning" can provide you with concrete ideas for dynamic partitioning strategies that relocate partitions when new nodes enter the system; we will have that respective lecture next week. The described techniques are exactly what you can do in your implementation as well: Start with an initial partitioning of the data and start loading; if new nodes enter, reassign partitions to them and continue the loading. (A more naïve approach would be to send all data to one worker until that worker signals that it is full; the continue sending further data to the next worker until also that worker is full; once you loaded all data, you can shuffle the loaded records again - this would be quite slow, but easy to implement.)
- 4) Study the command line arguments! Via the command line arguments, we provide, inter alia, a number of worker actors. This is a hint for how many worker actors we recommend the system to create (usually equal to the number of cores). Your solutions can create more or fewer worker actors. Decide yourself how many actors you would like to spawn and what you want to do with this parameter.
- 5) All actors in an ActorSystem share the same RAM! If an ActorSystem has 2GB RAM, not every (Worker) actor in that system has that much RAM. If you spawn 4 Worker actors in an ActorSystem, each work has only up to 1/4 of the ActorSystem's RAM. Monitor your resources that you do not run into OOM errors.
- 6) Because of 5) and 3), it might be a bit difficult to send the data to the workers directly and let all workers compete for the memory of their ActorSystem. A clever solution would create something like a DataProvider actor in every ActorSystem - a singleton that manages all data partitions of a local ActorSystem. If this is only one actor per ActorSystem, it is much easier to carefully handle local RAM (in contrast to a group of actors that compete for RAM). It is also easier to manage your data sharing protocols, which need to be implemented only in that particular DataProvider actor. Your Worker actors can then simply query their local DataProvider actor for the data that they require to validate IND candidates.

7) Do not perform too much work on your Master ActorSystem! If the master blocks all the threads and resources of its ActorSystem, the process will lose its connection with the rest of the cluster as heart beats will not come across. Try to do the hard work on the Worker nodes and use work/data pulling instead of work/data pushing whenever possible.

8) Test your solution frequently with multiple processes and different parameters and input datasets! Many issues can be seen quite early if you deliberately reduce the memory of your Java processes and increase/decrease the number of worker actors.

10) You can change almost everything in the ddm-akka algorithm frame! If you want to read larger input batches or no batches at all, just change it. Introducing new actors (such as a DataProvider) is something that might really make your life easier. Only the task description in the slides tells you, what you should not change.

11) Grouping by data types is dangerous! If you consider grouping the attributes by their data types to then only test IND candidates with the same type, you might lose some results. BOOLEAN cannot be in FLOAT (and vice versa), but any data type can be included in a STRING column and also INT can be in FLOAT. Do not lose valid results by excluding pairs that can actually be included in one another!

12) Fast solutions should be able to process the 400MB TPC-H dataset in about 300-400 seconds on the PI cluster. A setup with three processes on one modern desktop PC or laptop should be able to finish the profiling in less than that time.

13.) Java uses a garbage collector for memory management. Hence, memory is not directly freed when you give up on an object. Instead, Java will try to keep that object around for some time - once because garbage collection (gc) is expensive and second because sometimes it can reuse objects later on. For this reason, `ManagementFactory.getMemoryMXBean().getHeapMemoryUsage()` which is used in the MemoryUtils that the ddm-akka project provides you with is not really accurate. I therefore recommend a different strategy to track your memory consumption: Calculate "`workingMemory = MemoryUtils.bytesMax() - MemoryUtils.bytesUsed()`" before you start populating your memory with partitions of your input data. Then calculate the size X of all the partitions that you stored with `MemoryUtils.byteSizeOf(partition)`. You can then compare your workingMemory with that amount. Do not be too aggressive when using the workingMemory, because your system needs some working memory when it processes your data. But if you, for example, use 70% of your workingMemory for partition data, and if you do not allocate other large structures, you should be fine. Also, read up on Java Memory Management - this is a very important aspect of Java!

14.) The lines "`if (System.currentTimeMillis() - this.startTime > 2000000) this.end();`" is of course something that you cannot leave in your code. Call "`this.end()`" if you are really done with the profiling and not when a timeout occurs!

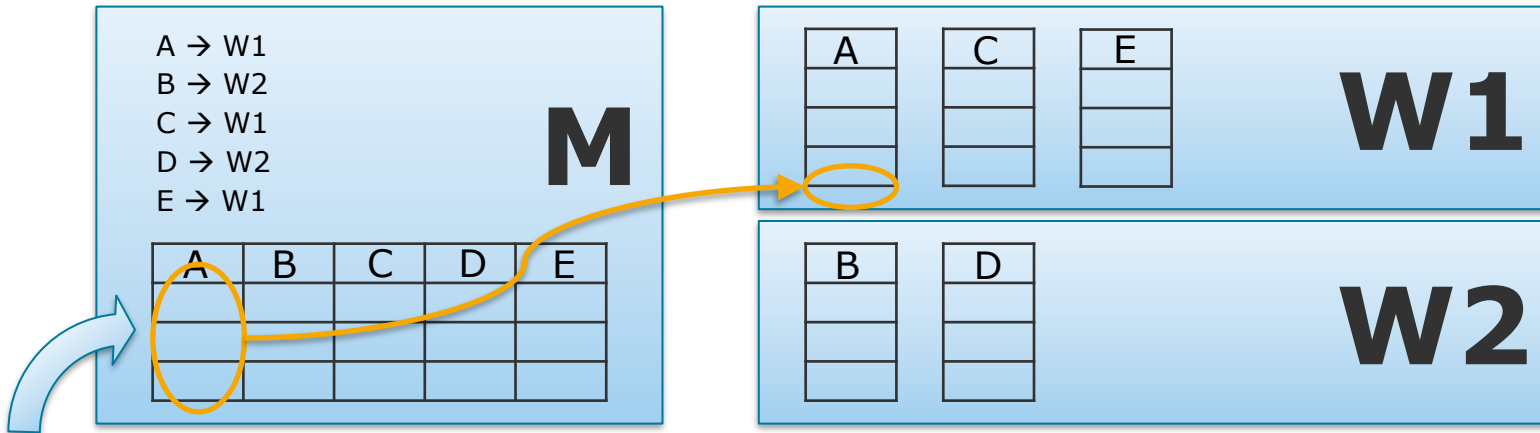
15.) You cannot assume that a single table fits into the memory of one machine entirely! The best strategy is to read the data batch-wise and then get rid of the batch directly by sending its data to the worker ActorSystems. The worker ActorSystems need to apply back-pressure if their memory is exhausted, nodes that join the cluster might need to steal partitions, the introduction of a singleton "DataProvider" actor on every ActorSystem is highly recommended, the DependencyMiner might need to pause the reading of batches if it cannot pass their data on to worker ActorSystems, and you cannot start validating IND candidates until all data has been loaded.

Best,

Thorsten

Homework

Final Hints



Homework Final Hints

"Fixed Number of Partitions" Partitioning

A → W1

B → W2

C → W1

D → W2

E → W1

M



A	B	C	D	E

A

C

E

W1

B

D

W2

W3

Homework Final Hints

"Fixed Number of Partitions" Partitioning

A → W1
B → W2
C → W1
D → W2
E → W3 !

M



A	B	C	D	E

A

C

F

W1

B

D

W2

F

W3

Steal!

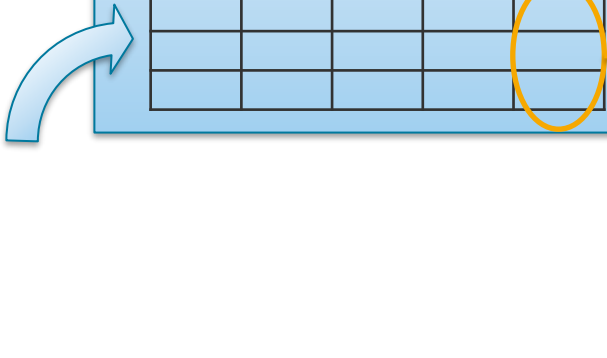


Homework Final Hints

"Fixed Number of Partitions" Partitioning

A → W1
B → W2
C → W1
D → W2
E → W3 !

M



A	B	C	D	E

A

C

W1

B

D

W2

E

W3

Homework Final Hints

"Fixed Number of Partitions" Partitioning

A → W1
B → W2
C → W1
D → W2
E → W3 !

M



A	B	C	D	E

A

C

F

W1

B

D

W2

F

W3

Segmentation?

Homework

Final Hints

DependencyMiner

$A \rightarrow W1$
 $B \rightarrow W2$
 $C \rightarrow W1$
 $D \rightarrow W2$
 $E \rightarrow W3$



A	B	C	D	E

DataProvider

A	C

$A \subseteq C ?$

$C \subseteq A ?$

Worker

Worker

Worker

W1

B	D

Worker

Worker

Worker

W2

E

Worker

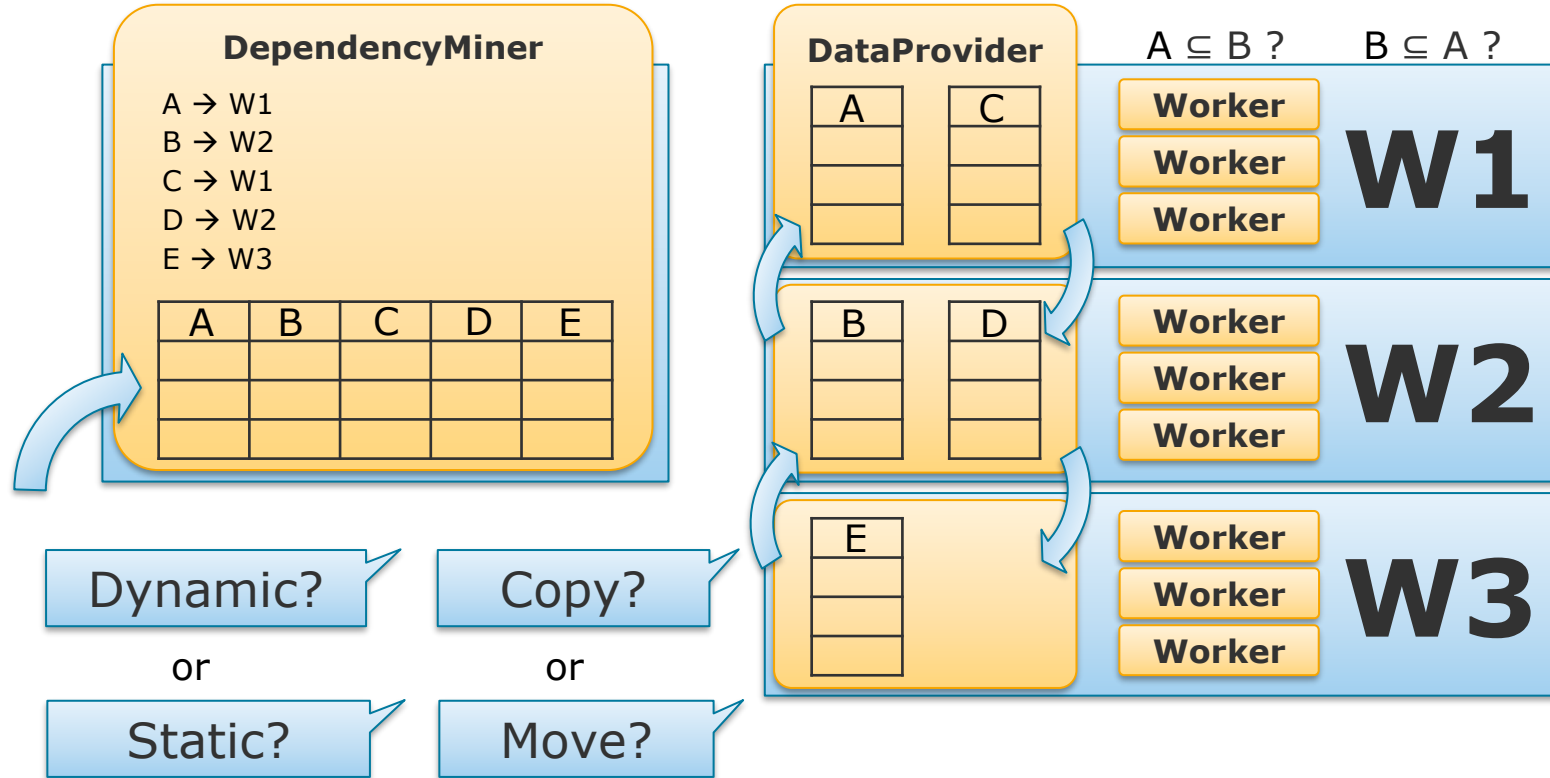
Worker

Worker

W3

Homework

Final Hints





"I wait for green"

"Road ahead is free!"

"I wait for crossing traffic"

"I accelerate!"

"You are not in my path!"

"Attention, I break!"